

STM32C5 Workshop

Mena Roumbakis

Technical Marketing

Daniel Isacson

Application Engineer

October 2026

Agenda

1. STM32 overview
2. Introduction to STM32C5
3. STM32C5 target applications
4. Security on STM32C5
5. Ecosystem
6. Demonstration on STM32C5

Labs

1. Lab1 Blink LED with Timer
2. Lab 2 ADC implementation
3. Lab 3 EXTI and USART
4. Lab 4 Emulated EEPROM
5. Optional Lab FDCAN
6. Q&A

STM32 overview

The STM32 portfolio

Five product categories

Wireless MCU

Short- and long-range connectivity

Ultra-low-power MCU

Mainstream MCU

High-performance MCU

32-bit general-purpose microcontrollers: from 75 to 5,072 CoreMark score

Embedded MPU

32- and 64-bit microprocessors







Enabling edge AI solutions

Scalable security

MPU portfolio

MCU portfolio

Introduction to STM32C5

A modern platform

Built on Arm® Cortex® -M33 core

Powered by ST's 40nm process

Enabling a smooth design journey

- **No external accelerators required**
  - Integrated floating-point unit simplifies software development & hardware division
  - Enhanced native DSP and math instructions
- Easier peripheral separation and software management thanks to **more flexible** interrupt vectors
- **Easier debugging** with improved trace features
- Boosted edge AI algorithms
- Cortex® -M33 is **supported by all major compilers**

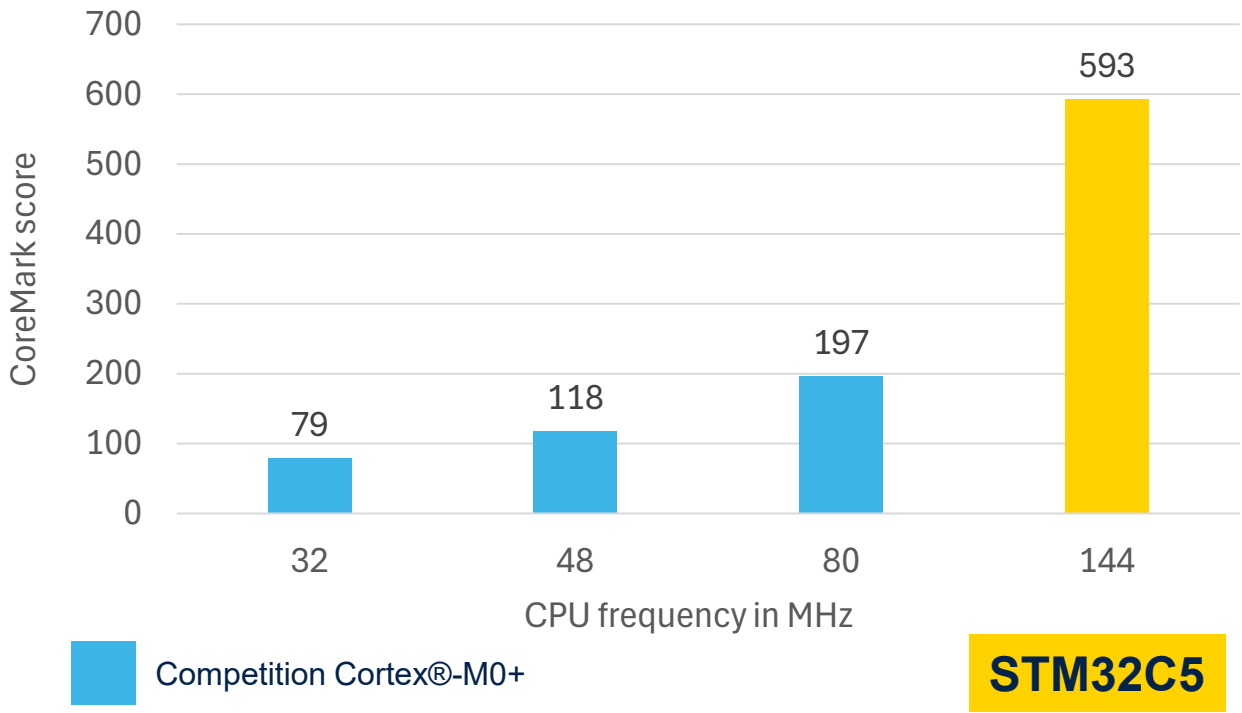


512 Kbytes of flash, **128 Kbytes** of SRAM,
64 pins, starts at **$0.87** for **10k** units

A modern core providing a performance leap for the entry range

	Cortex®-M0+	Cortex®-M33
Instruction set	Arm V6	Arm V8
FPU	-	Yes
Hardware divide	-	Yes
DSP instructions	-	Yes
CoreMark/MHz	2.46	4.12
FIR filter (float32) perf	Reference	40x more performance
FIR filter (int32) perf	Reference	20x more performance
Flexible interrupt vectors	Up to 32 inputs	Up to 480 inputs

STM32C5 provides **three times more performance** than the competition Cortex®-M0+ in the market



Reliable performance in industrial environments



A robust platform and precise clock system

- Arm Cortex®-M33 core up to **144 MHz** with FPU, MPU, DSP, and flexible NVIC
- Up to **125°C** ambient temperature without performance penalty
- **1%** factory-calibrated internal oscillator between wide ambient temperature range from **-20** to **105°C**
- Multiple external clock sources including high-speed (HSE), low-speed (LSE), and internal oscillators for flexibility and redundancy
- Versatile analog features available (op amp, DAC, 12-bit ADC, comparator)

Industrial interfaces and flexible memory

- More RAM available compared to typical Cortex-M0 +-based applications
- **OctoSPI** for external memory expansion
- Industrial-grade communication interfaces: **Ethernet**, **FDCAN**, **I3C**

Security & safety

- Hardware security with side channel attack resistant crypto
- PSA Certified Level 3 & SESIP3 target certifications
- Ready for functional safety requirements

Certified functional safety packages for STM32C5



www.st.com/x-cube-stl

Robust performance and reliability in harsh conditions

- ECC on RAM and flash
- Brown-Out Reset
- Dual Watchdog

SIL functional safety

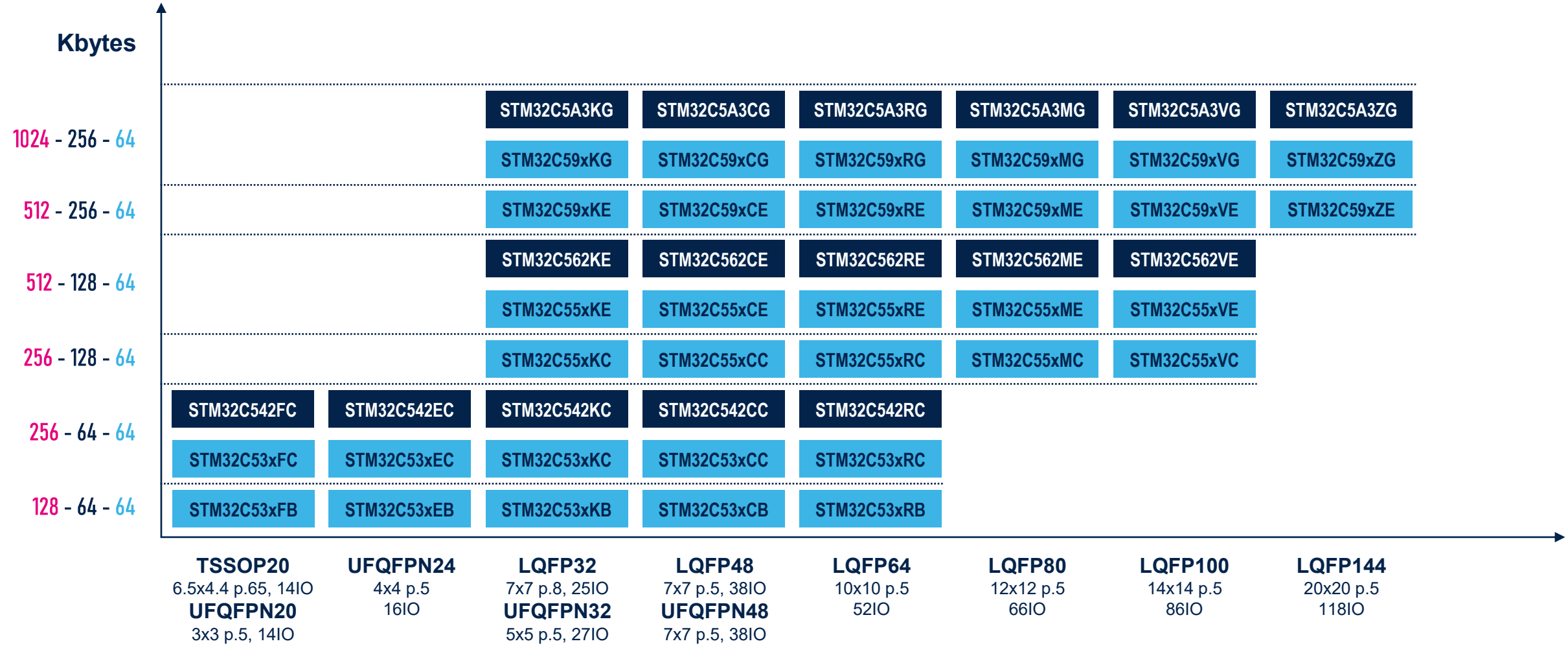
- Ready for IEC 61508 systems up to SIL3
- FMEDA & FMEA, safety manual
- Certified STM32 Self-Test library

Class B functional safety

- IEC 60335-1/60730-1 Class-B
- Household electrical appliances
- Certified STM32 Self-Test library

One new STM32 series. Multiple lines.

Flash - RAM - Flash area for software EEPROM emulation



Commercial part number availability in $T_{ambient}$ = -40 to 85/125°C

Crypto line | Non-crypto

Find the right fit for your design thanks to a scalable portfolio

	STM32C531	STM32C532	STM32C542	STM32C551	STM32C552	STM32C562	STM32C591	STM32C593	STM32C5A3
Flash size with ECC	128 to 256 KB	128 to 256 KB	256 KB	256 to 512 KB	256 to 512 KB	512 KB	512 KB to 1 MB	512 KB to 1 MB	1 MB
RAM size with ECC	64 KB	64 KB	64 KB	128 KB	128 KB	128 KB	256 KB	256 KB	256 KB
12-bit ADC	1	1	1	2	2	2	3	3	3
DAC/COMP	2/2	2/2	2/2	1/1	1/1	1/1	1/1	1/1	1/1
OPAMP	1	1	1						
Memory IF							1x OctoSPI	1x OctoSPI	1x OctoSPI
FDCAN		2	2		1	1		2	2
Ethernet 10/100Mbit								Yes	Yes
AES			Yes			Yes			Yes
HASH	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
SAES/SPKA, HUK, SCA, CCB							SPKA verify	SPKA verify	Full config

STM32C59x/A3 block diagram

Arm® Cortex®-M33
144 MHz
593 CoreMark

FPU, MPU, DSP, ETM
SW debug

8KB - ICACHE

Memory

1 MB flash
Dual Bank w/ ECC

256 KB SRAM w/ ECC

64 KB flash area for software emulated EEPROM

4.5 KB OTP	128B bkp RAM

System

VDD 2.7-3.6V

32kHz LSE, LSI, HSE
144MHz HSI with internal 1% accuracy Ta=-20-105°C

RTC

Connectivity

4x USART, 3x UART
1x LPUART

3x SPI

2x I2C, 1x I3C

1x USB FS

2x FDCAN (2)

1x OCTOSPI

ETH 10-100Mb (2)

Security

SAES, AES 128/256 (1)

TRNG

HASH, OTP, HUK
SPKA, CCB

96-bits unique ID

Accelerators

2x DMA (8+8)

CORDIC, CRC

Analog

3x 12-bit ADC
2.25MSPS

1x DAC12
w/ Buffer

1x COMP

Timers

4x 32-bit GPTIM

2x 16-bit 7ch MCTIM

4x 16-bit GPTIM

2x 16-bit basic TIM

1x 16-bit LPTIM

IWDG, WWDG
Systick

Operation modes

Run, Sleep, STOP,
Standby

Bootloader

UART, SPI, FDCAN, USB

Operating temperature

$T_{ambient}$ -40 to 125°C

$T_{junction}$ 140°C

Packages

QFP (0.5p) 48, 64, 80, 100, 144
QFN (0.5p) 32, 48

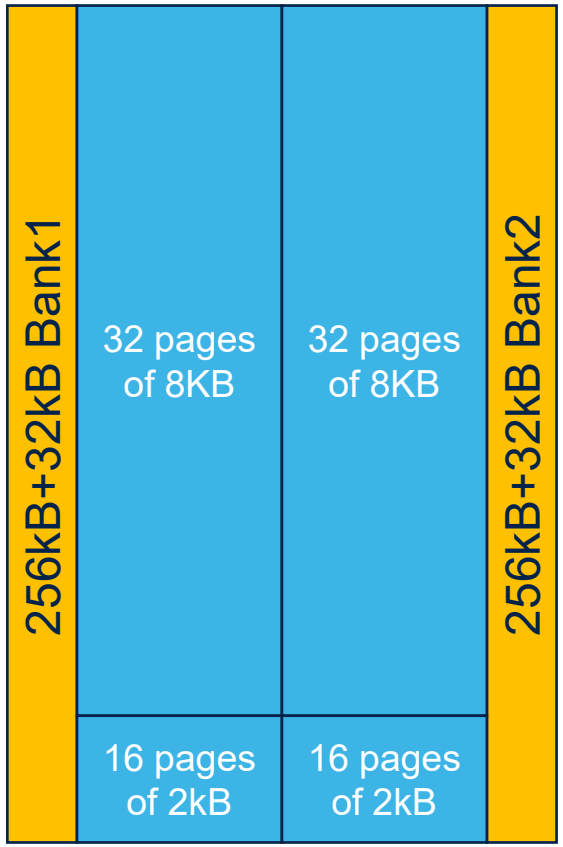
Cortex-M33 for all entry products

Extensive memory

More connectivity

(1) Available on STM32C5A3 lines
(2) Available on STM32C593 and STM32C5A3 lines

STM32C55x Flash



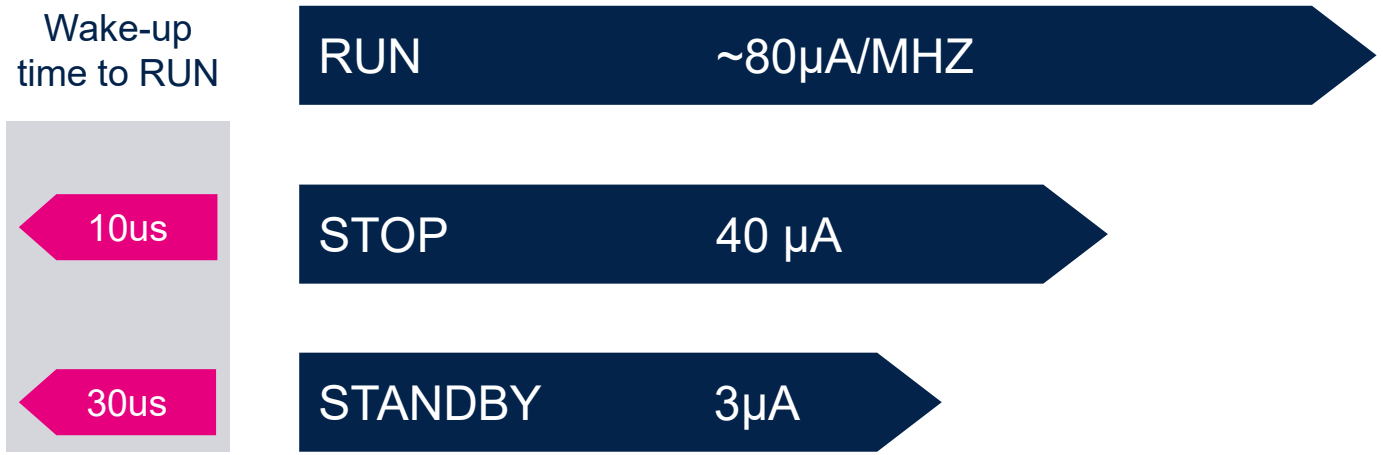
- 512kB program flash with ECC, split into dual bank
  - 8kB flash pages, 10k erase cycles
  - Linear address space
- 64kB data flash, split into dual bank
  - 2kB data pages, 10k erase cycles
  - EEPROM emulation
  - Linear address space
- Read-while-write dual bank

$$writeCycles = \frac{32kB_data_flash_per_bank \ * 10k_flash_erase_cycles}{appBytesToWrite}$$

Number of application bytes to store in EEPROM	Number of effective write cycles
256 Bytes	1.3M cycles
1k Bytes	320k cycles
8k Bytes	40k cycles

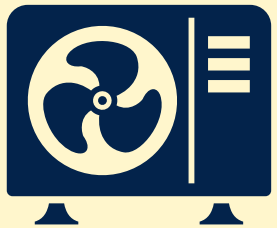
STM32C5

STM32C5X target current consumption



STM32C5 target applications

STM32C5 Target Applications

Smart homes		Industrial devices	Consumer devices	
				
Fridges Ovens Coffee machines	Fan control Circuit breakers	Industrial pumps Control Panels	Smoke detectors Fire detectors Alarms	PC peripherals Printers & accessories

Security on STM32C5

STM32C5 target certifications

STM32C5 256k	• PSA L1 certification • SESIP/PSA RoT Level3
STM32C5 512k	• PSA L1 certification • SESIP/PSA RoT Level3
STM32C5 1M	• PSA L1 certification • SESIP/PSA SOC Level3 (with extra Security Functional Requirements related to CCB/Crypto IPs)

RoT (HW) : RDP/Lifecycle/HW isolation

STM32C5 security overview

Increasing security with digital identities

Memory protections
against illegal access control

OTP, HDP, WRP, RDP, MPU
Passive tamper

Cryptography
for hardware robustness

(*) SAES, SPKA, SHA512
SHA, SHA1, SHA224/256
TRNG
Hardware unique key (HUK)

Platform protection
during product lifecycle

RDP: 2 protection level states
Password based regression

Code isolation
for runtime protection

Temporal isolation – HDP

Security services

Secure boot & secure updates

Secure KeyStore with CCB*

NIST - CAVP certified X-CUBE-CRYPTOLIB
(**) Flexible Crypto Framework

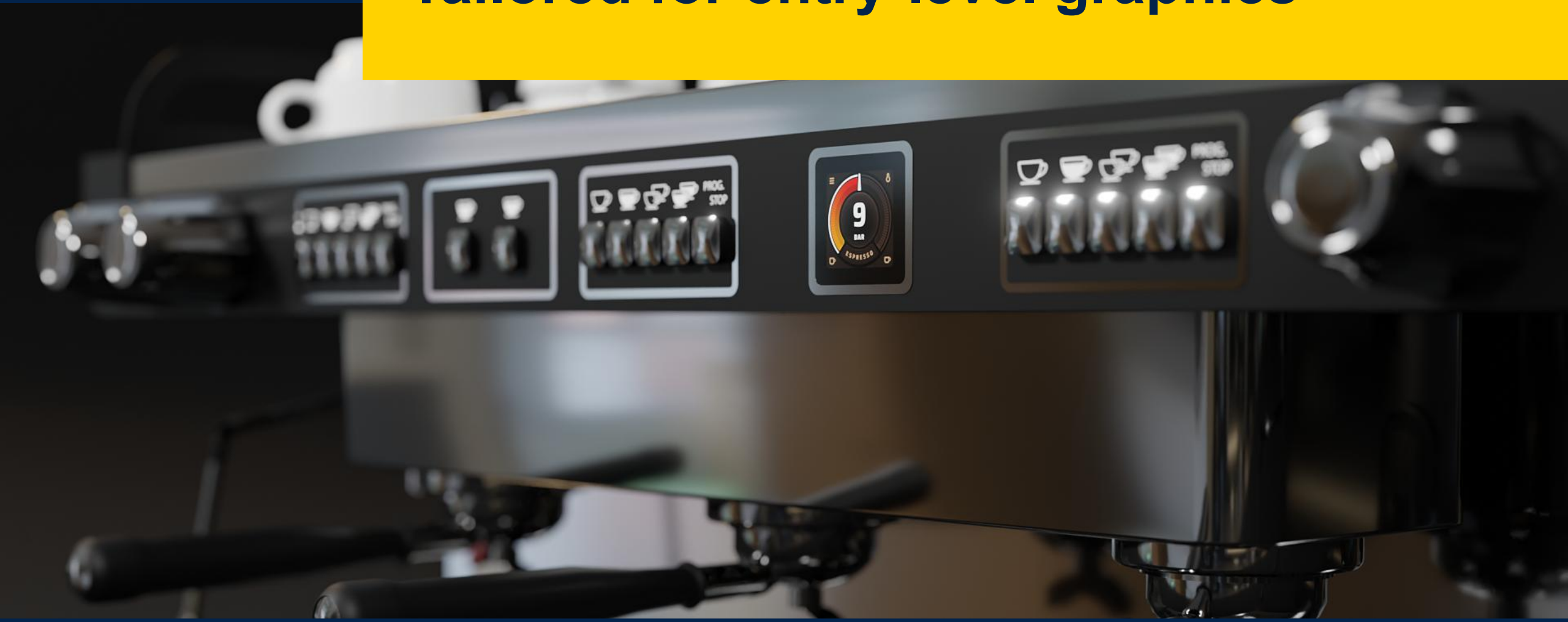
State-of-the-art security assurance level

target certifications

*Available only on STM32C5-1M. SAES, SPKA side channel resistant
Available Q2'2026

** X-CUBE-CRYPTOLIB V5 supports Classic + PQC
Available Q2'2026

Tailored for entry-level graphics



STM32 hardware for entry-level graphics

Graphic performance	ENTRY LEVEL							
Display	SPI/Quad/OctoSPI and parallel display IF, G-RAM, touch, and no-touch							
STM32 MCU	**STM32U0**	**STM32U3**	**STM32C0**	**STM32G0**	**STM32H503**	**STM32C5**	**STM32WBA**	**STM32WB**
Development kits-TouchGFX supported	**Nucleo-U083RC** + X-NUCLEO-GFX01M2	**Nucleo-U385RG-Q +** RVA15MD*	**Nucleo-C092RC** + RVA15MD*/ X-NUCLEO-GFX01M2	**Nucleo-G071RB** + X-NUCLEO-GFX01M2	**Nucleo-H503RB** + X-NUCLEO-GFX01M2	**NUCLEO-C5A3ZG** + RVA15MD	**Nucleo-WBA65RI** + RVA15MD*	**Nucleo-WB55RG** + X-NUCLEO-GFX01M2
Main features *Max memory listed*	CM0+, 56 MHz 40 KB RAM 256 KB flash	CM33, 96 MHz 256 KB RAM 1000 KB flash	CM0+, 48 MHz 36 KB RAM 256 KB flash	CM0+, 64 MHz 144 KB RAM 512 KB flash	CM33, 250 MHz 32 KB RAM 128 KB flash	CM33, 144 MHz 64-256 KB RAM 128-1024 KB flash	CM33, 100 MHz 512 KB RAM 1000 KB flash	CM4, 64 MHz 256 KB RAM 1000 KB flash
Framebuffer details	Partial in internal RAM, starting from 6 KB					Full or partial in internal RAM		
Asset storage (flash memory)	Internal or External SPI	Internal or Externa(Q)SPI	Internal or External SPI	Internal or External SPI	Internal or Externa(Q)SPI	Internal or External (O)SPI	Internal or External SPI	Internal or Externa(Q)SPI

**Riverdi add-on kit: RVA15MD, order from Riverdi sales channels: [1.54" STM32 Nucleo-64 display kit - Riverdi]*

STM32C5 tailored for entry-level GUIs

STM32 hardware



All STM32C5 Nucleo kits

+



Extension display from Riverdi





TSD Authorized Partner

One-chip module from TSD

STM32 software

TouchGFX + STM32 Cube



A complete STM32 software offer, free of charge

STM32C5 hardware development tools

STM32C5

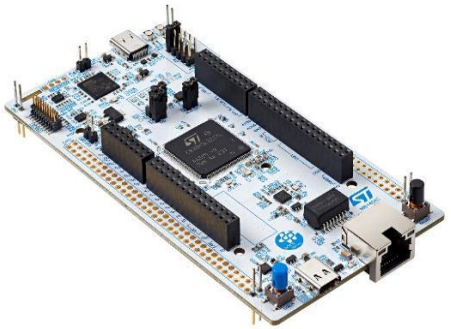
Development tools for STM32C5



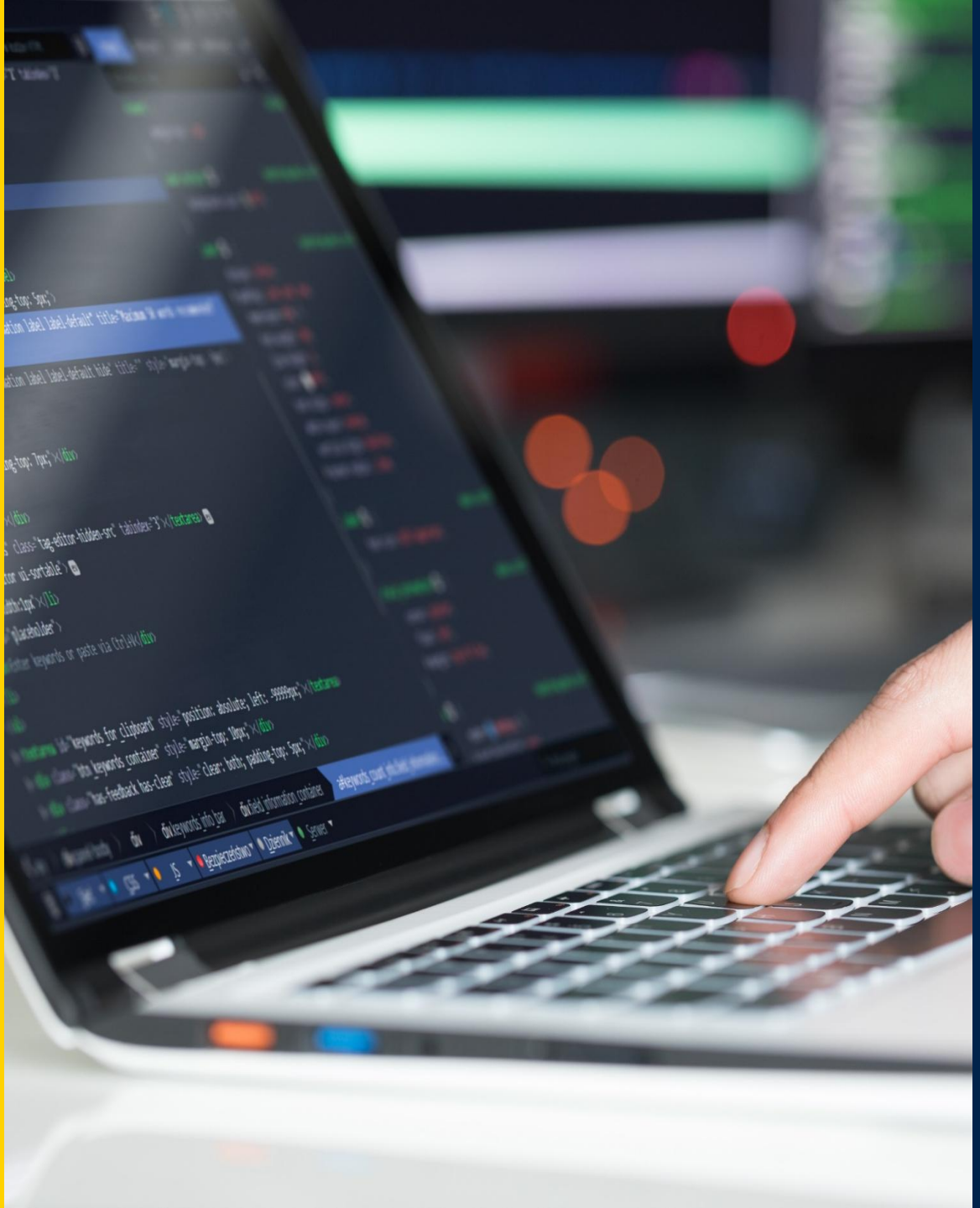
NUCLEO-C542RC
FDCAN USB FS 64-pin MCU Arduino IF



NUCLEO-C562RE
FDCAN USB FS 64-pin MCU Arduino IF



NUCLEO-C5A3ZG
Ethernet (RMII) USB FS FDCAN footprint 144-pin MCU Arduino IF



Ecosystem

Inside the STM32Cube ecosystem

STM32C5 ECOSYSTEM

STM32 CubeMX2

STM32 CubeEmbSW

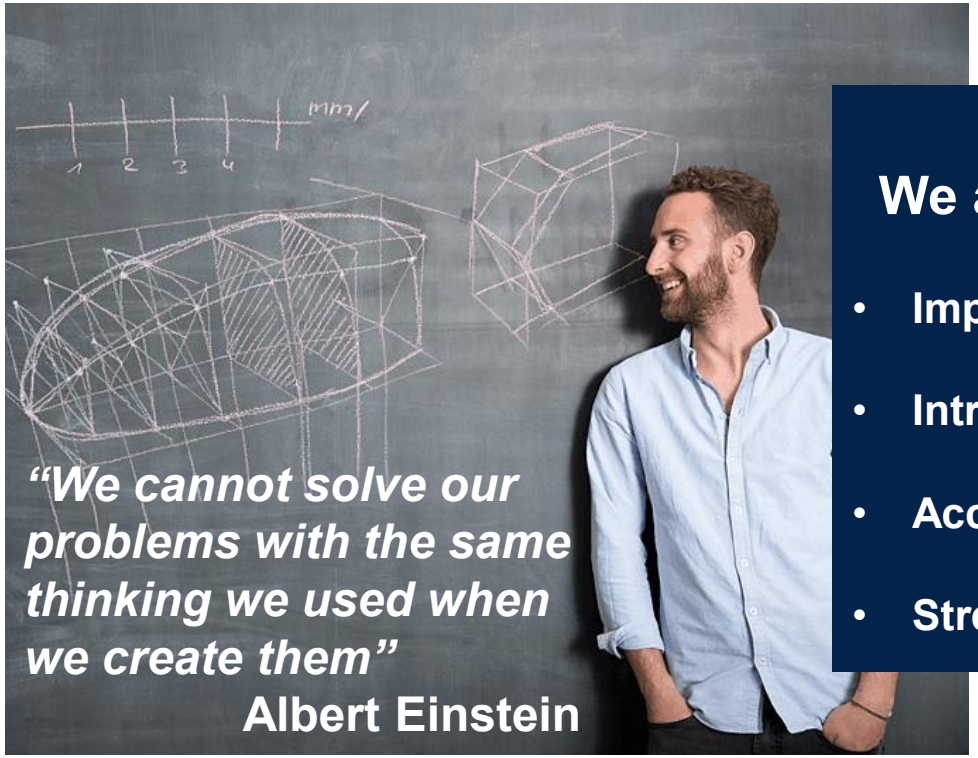
STM32 CubeIDE for VS Code



Demonstration on STM32C5

STM32Cube Ecosystem

STM32Cube Ecosystem is evolving to help you even more



We are upgrading our STM32Cube Ecosystem to:

- Implement developers’ expected enhancements and fixes
- Introduce more advanced features and deploy them at scale
- Accelerate our reactivity and release cycles
- Strengthen our foundation to support richer STM32 MCUs roadmap

STM32Cube Ecosystem

STM32Cube Ecosystem is evolving to help you even more

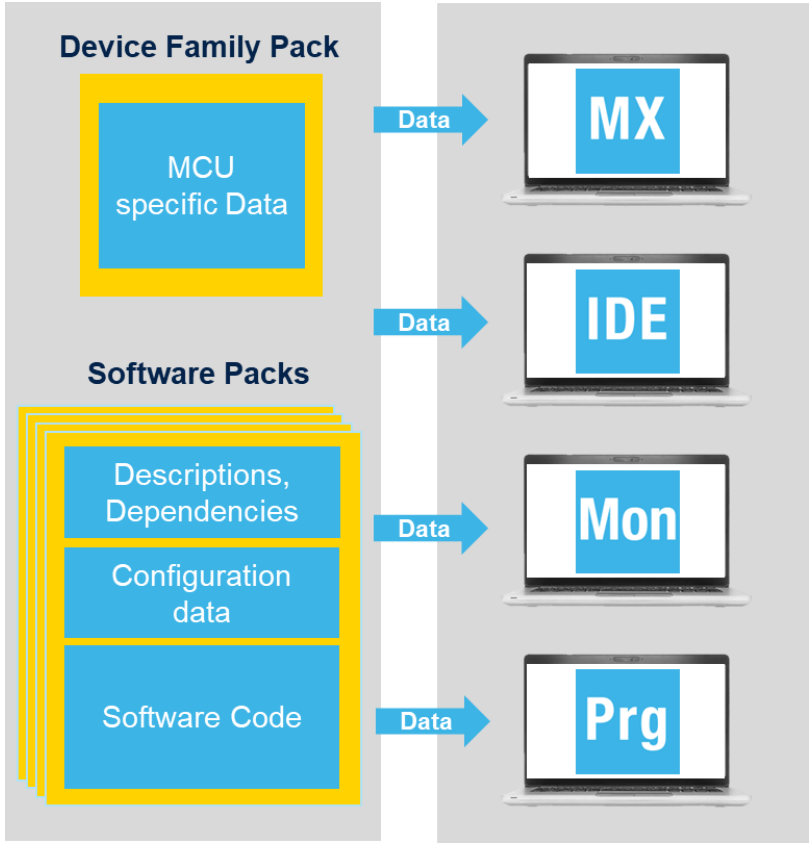


Our new architecture principles

- Decoupled embedded software and tools
  - Modular and configurable embedded software components
  - Data-centric tools, STM32 MCUs agnostic
- Inter-operable tools instead of tools integration

STM32Cube Ecosystem

STM32Cube Ecosystem new architecture principles



- **Device Family Pack (DFP):**
  - Data Describing each MCU (Pinout, clock, SVD, Flash, startup)
- **Modular software components (Software Packs)**
  - 1 driver pack for each series
  - 1 pack for each Middleware, example, utility and external part drivers
- **Open-CMSIS-PACK format for each pack description**
  - Data describing software licenses, versions, dependencies and configuration
- **Hardware-Independent tools**
  - Software tools use DFP and Software Packs data as standard input
  - Capacity to automatically fetch and add dependencies.

STM32CubeMCU Packages Evolution

HAL drivers are upgraded, LL drivers are unchanged

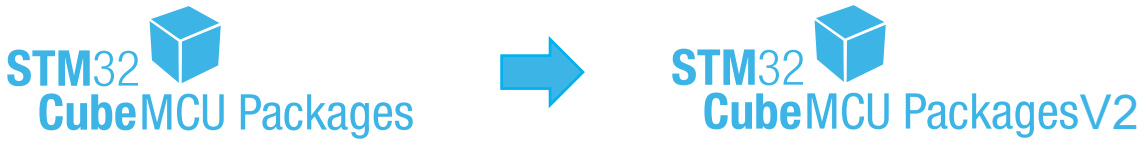
HAL ➡ HAL2.x

- **Optimized footprint:** up to 59%* lower flash size → **Saving memory and cost**
- **Improved API:** simplified APIs, faster execution, added missing functions → **Shorter design, higher perf.**
- **Improved portability:** compatibility across STM32 series → **Accelerated migration**
- **Enhanced quality:** IEC61508/SC2 and MISRA C compliant, statical analysis → **More reuse, Safety ready code**
- **HAL2 built on top of LL:** HAL2 now calling LL APIs → **Easier and faster optimization**

*measured on RCC on STM32U5

STM32CubeMCU Packages Evolution

Our embedded software packages are reworked



- **Modular software components:** unitary, OPEN-CMSIS-PACK based
- **More examples:** reference use of all STM32 MCU features
- **Improved example's structure** made boards and IDEs agnostic
- **Enhanced BSP architecture:** based on part driver's layer

- **More flexibility, easier update**
- **Faster development and debug**
- **Easier reuse in end application**
- **Accelerates application bring-up**

STM32CubeMX Evolution

Faster development, alignment with embedded SW evolution



• **Data centric:** MCU agnostic, using CMSIS-Packs components as input	• **Flexibility on tool used versions**
• **Upgraded Configuration:** Enhanced pinout, clock and IPs settings, more flexible code generation	• **Fast and optimized configuration**
• **Porting support:** Supporting inter and cross series migration	• **Accelerated porting**
• **New Code preview:** Live view of generated code with easy copy	• **Easier code access and reuse**
• **Improved UX:** New views and highlights, simplified journeys	• **Faster and easier configuration**
• **Enhanced CLI:** full access to functionalities through command line	• **Automated CI/CD support**

STM32 IDE Evolution

Faster development, alignment with embedded SW evolution



• **Data Centric:** MCU agnostic, using CMSIS-Packs components as input	• **Flexibility on tool used versions**
• **VSCode based, decoupled from MX:** lighter, faster, more evolutive	• **Faster install and development**
• **CMake based, GCC and LLVM support:** decoupling projects and build	• **More flexibility, faster build**
• **Modular features addition:** flexible install with ‘Bundle manager’ feature	• **Flexible install, disk space saving**
• **Enhanced UX:** simple journeys, internationalization support	• **Easier to use tool**
• **Separated CLI:** full access to functionalities through command line	• **Automated CI/CD support**

STM32CubeMX2: GUI and Features



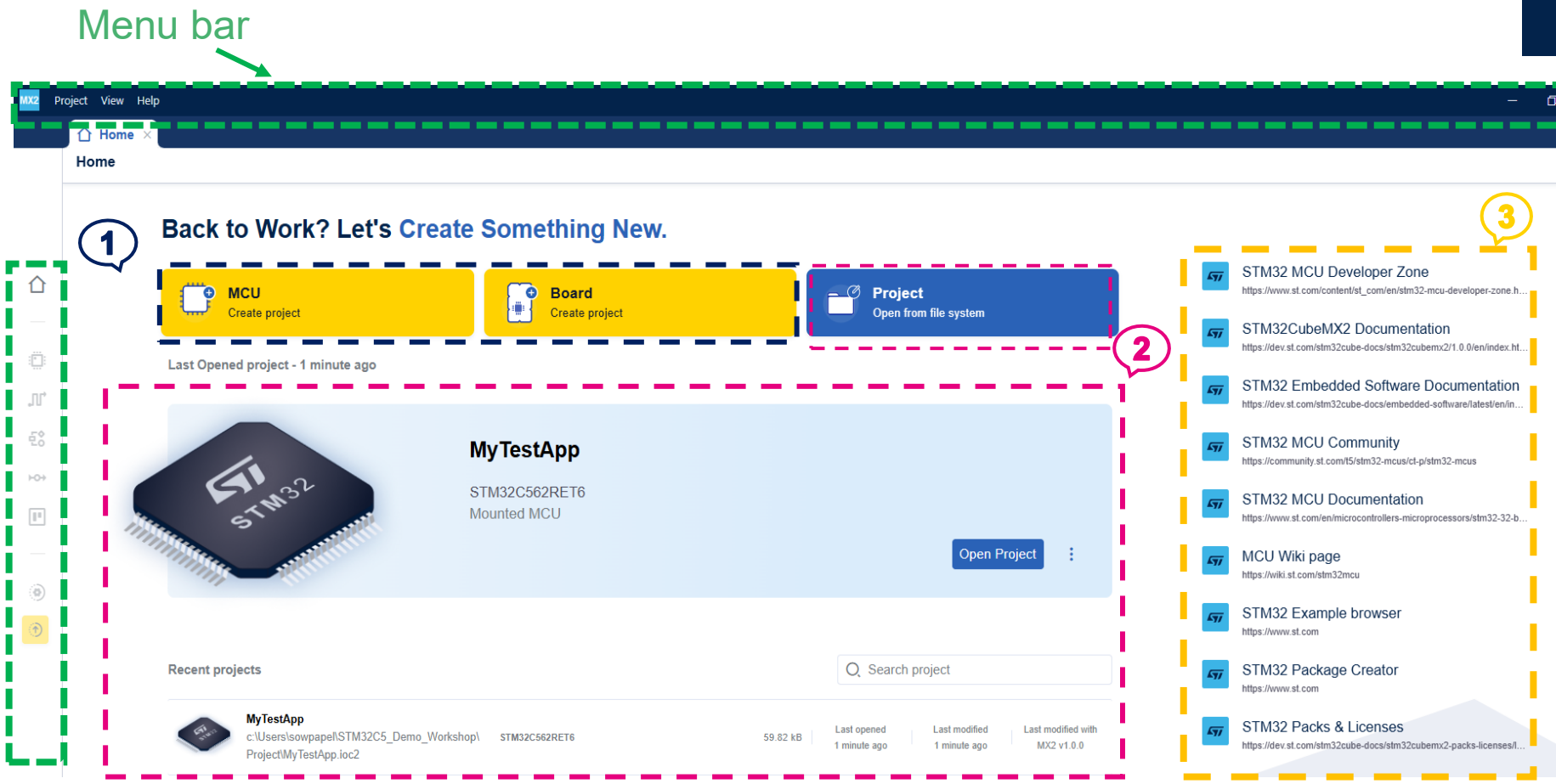
STM32CubeMX2 GUI: Home

Main Features:

Start a new project (1)
- From an MCU
- From a board

Open an existing project (2)
- From the file system
- From the list of current/ recent projects

Access useful links and documentation (3)



STM32CubeMX2 features:
Pinout, Clock, Code preview



STM32CubeMX2 features: **Pinout**



Pinout Interface: Graphic view

Main Features:

Configure and manage the MCU's pin assignments and properties (1)

- Assign pin to a signal
- Add/ Remove Pin label
- Access to configuration
- Exclude pin
- Reset pin state

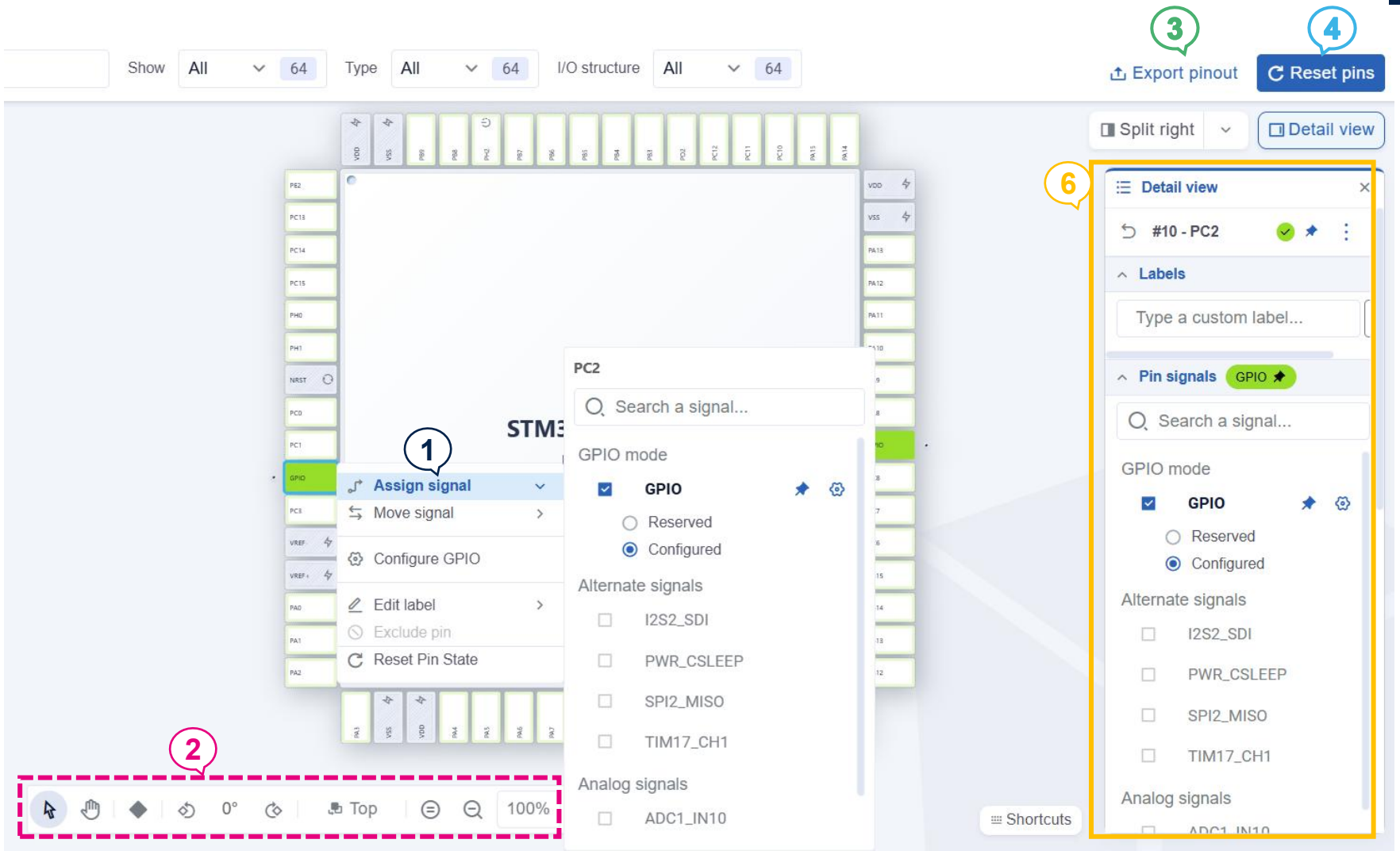
Toolbar (2)

- Flip package view
- Move/Edit
- Zoom in/out

Export pinout (4)

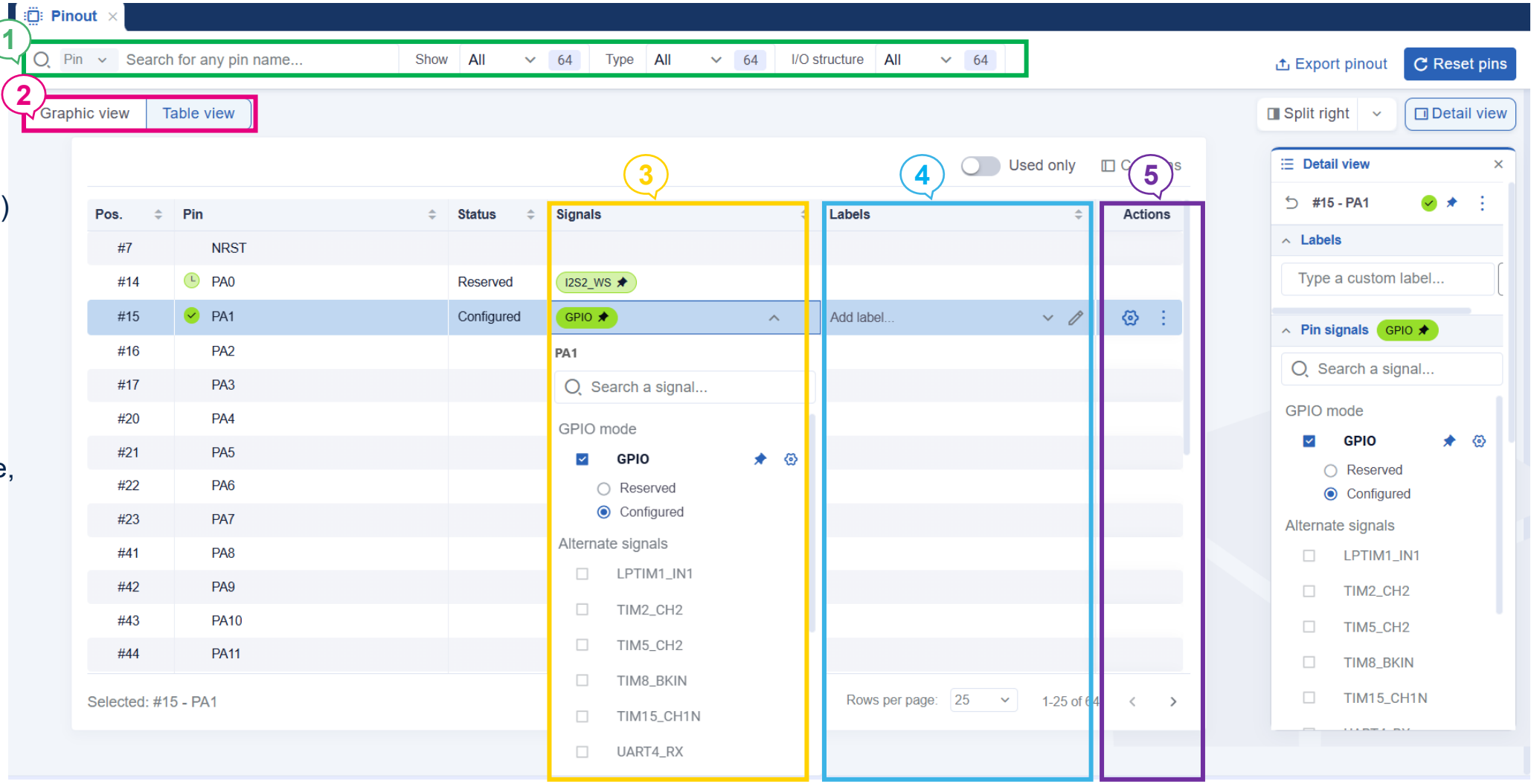
Reset pinout (4)

Detail view (6)



Pinout interface: Table view

- Search & filter options (1)
- Switch button (2)
- Pin assignment (3)
- Add/remove Labels (4)
- Configure, reset pin state, move signal, exclude (5)



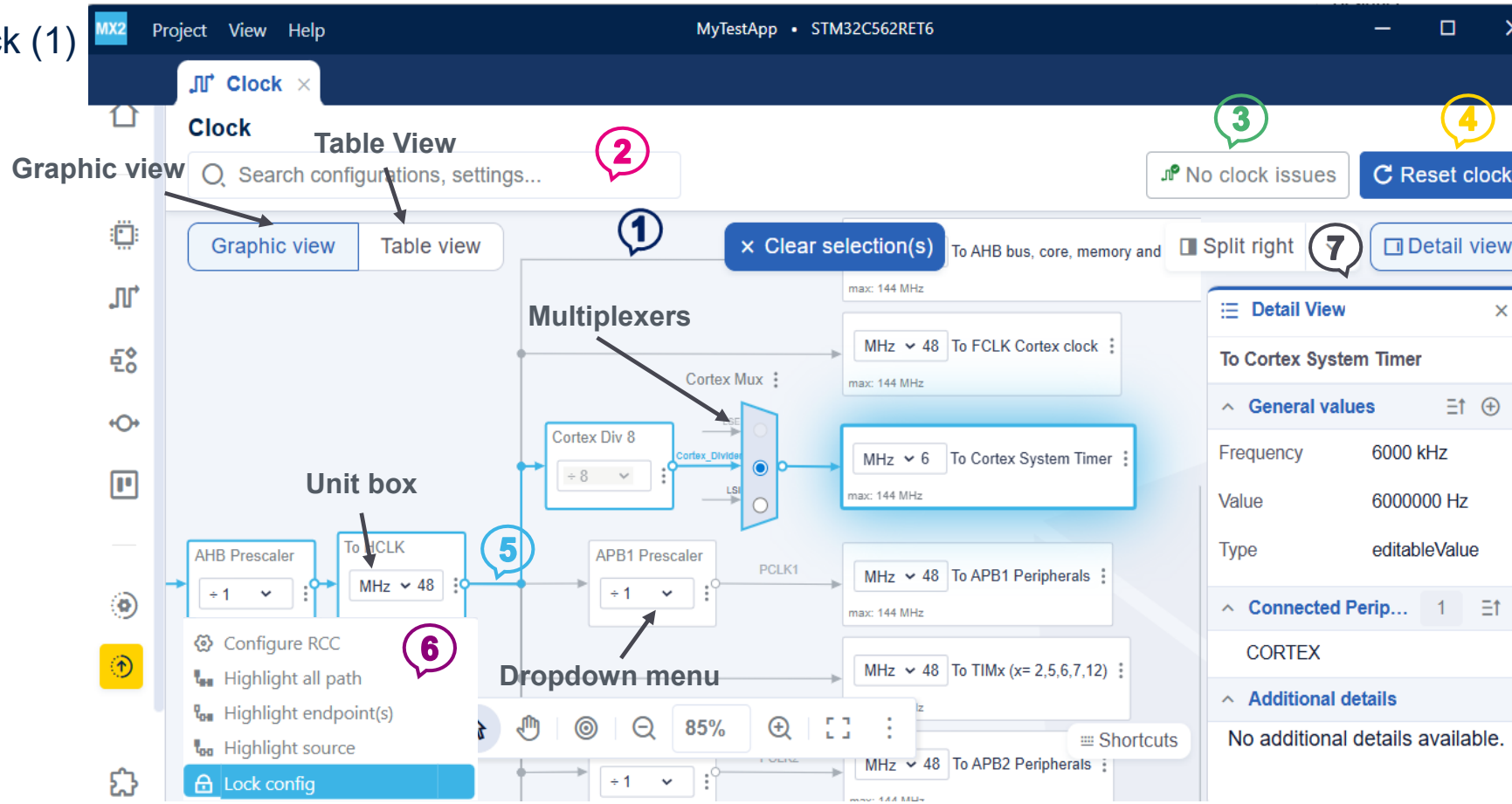
STM32CubeMX2 features: **Clock**



Clock : Graphic View

Main Features:

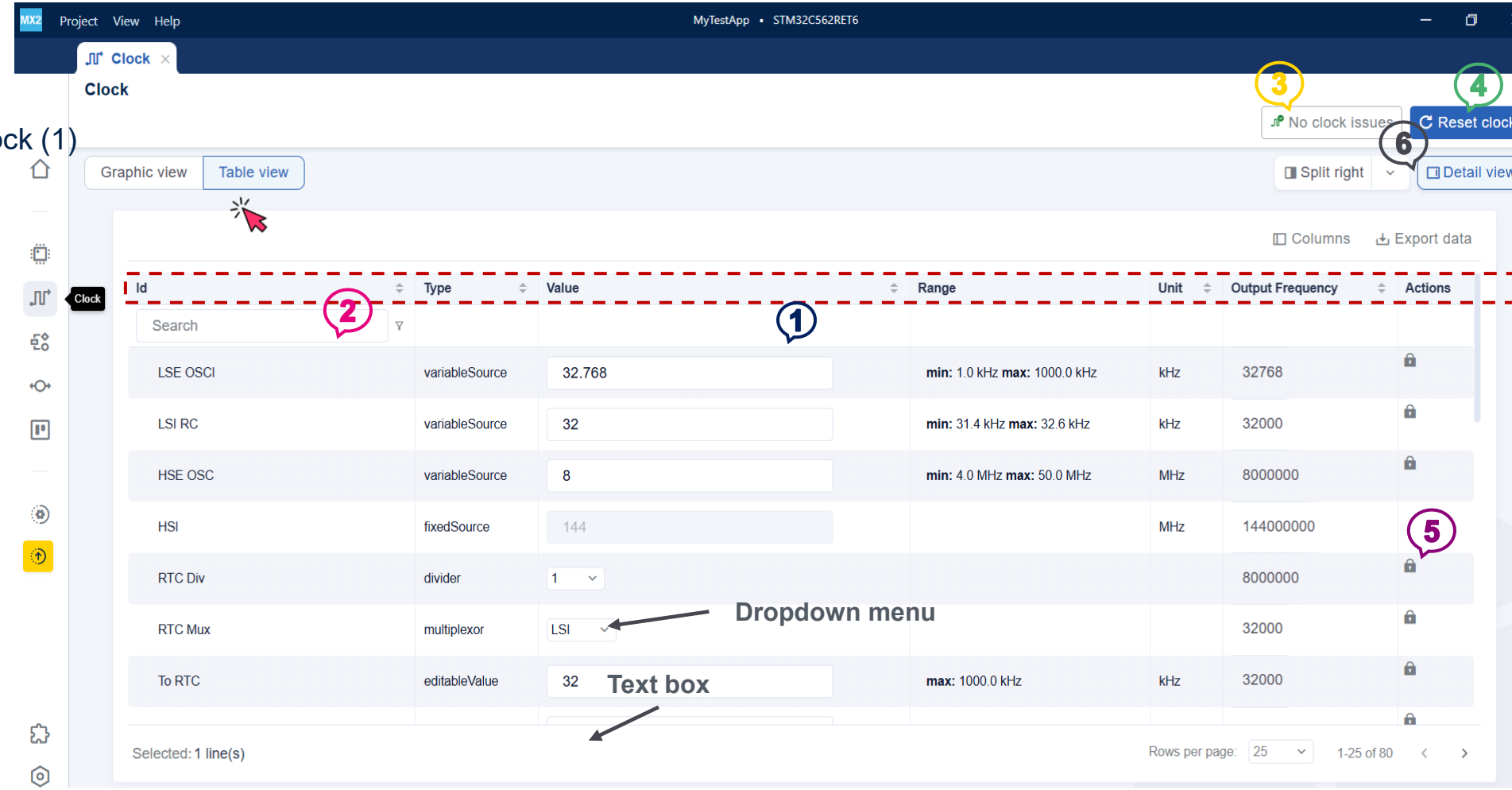
- Configure and manage the MCU's clock (1)
- Search clock (2)
- Resolve clock issues (3)
- Reset clock (4)
- Highlight path (5)
- Configuration RCC, Lock/ Unlock (6)
- Toggle details view (7)



Clock: Table View

Main Features:

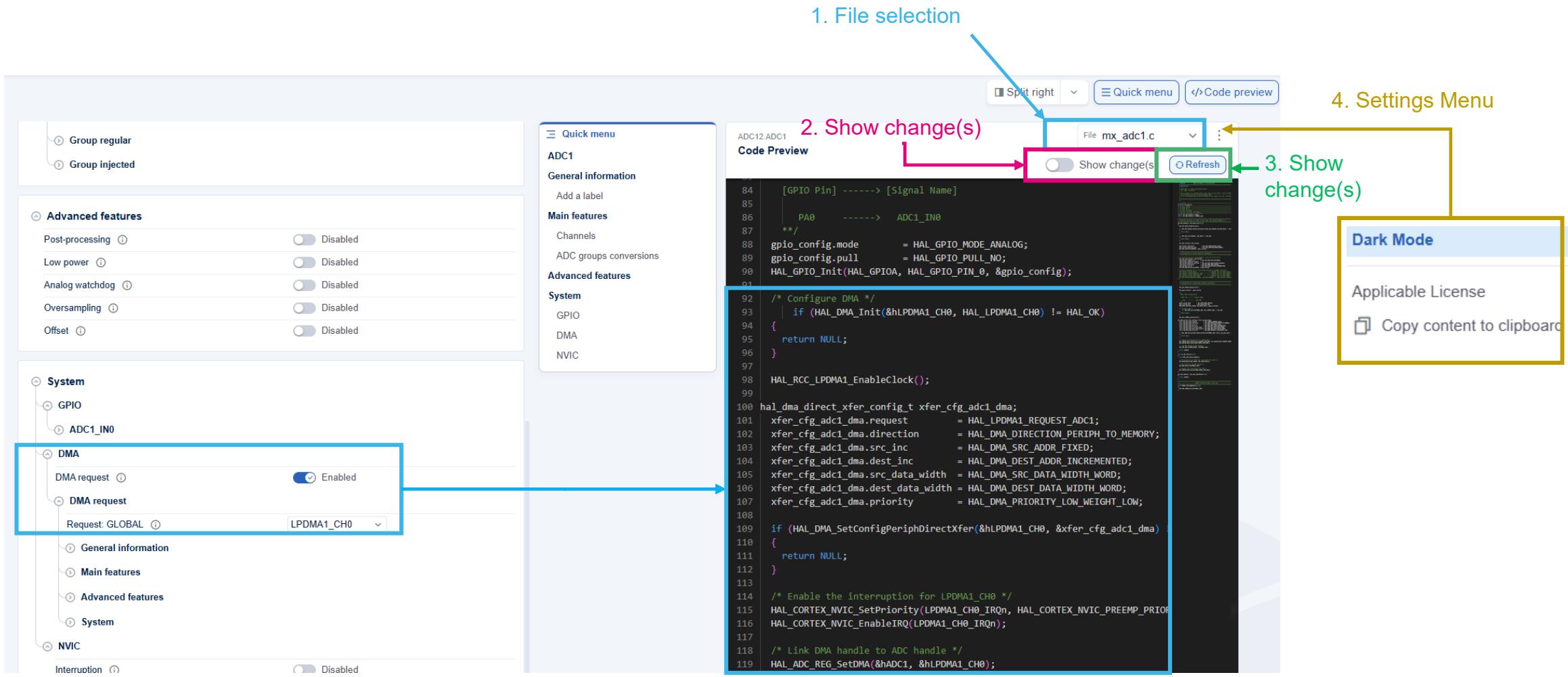
- Configure/ Manage the MCU's clock (1)
- Search clock (2)
- Resolve clock issues (3)
- Reset clock (4)
- Lock/ Unlock configuration (5)
- Toggle details view (6)



Use case : **Code Preview**



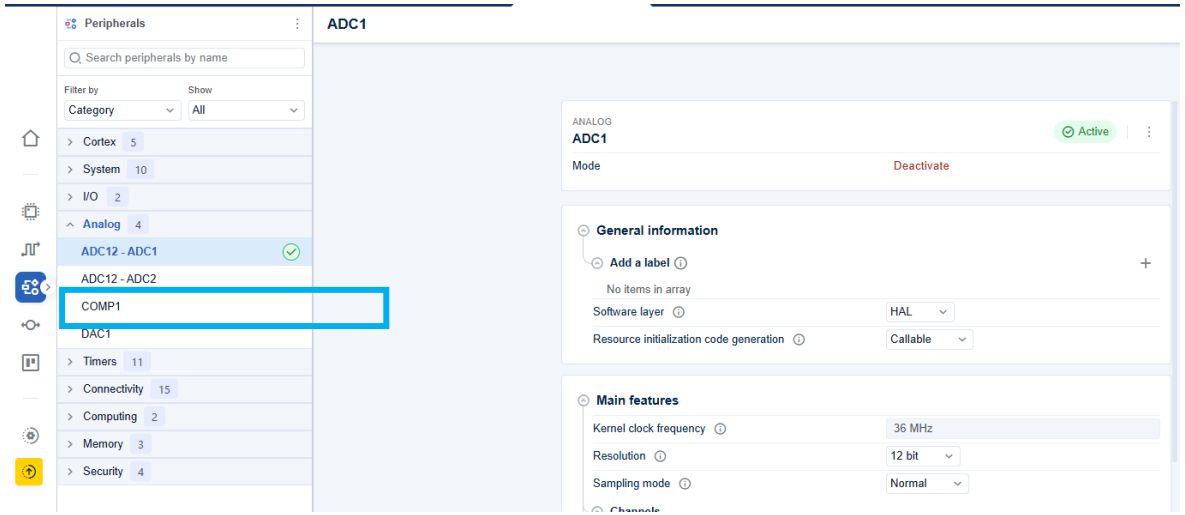
STM32CubeMX2 features: ***Code preview***



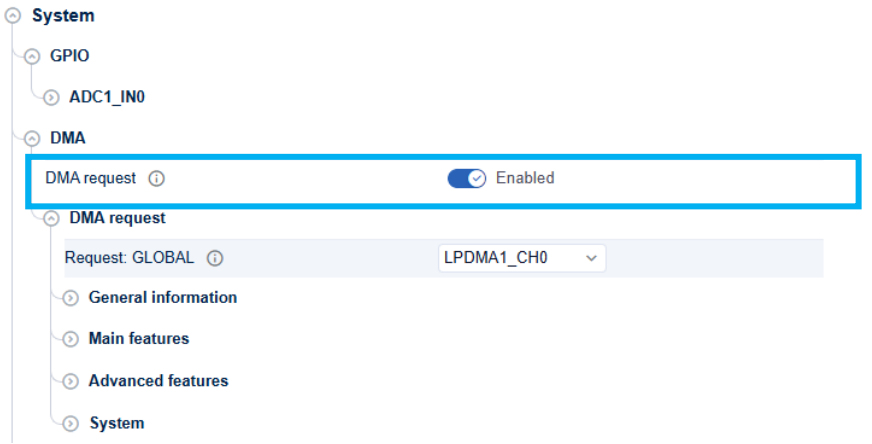
Code preview use case

How can I associate my ADC with a DMA in my project without regenerating code?

1 Go to peripheral section and click on “ADC12 – ADC1” section

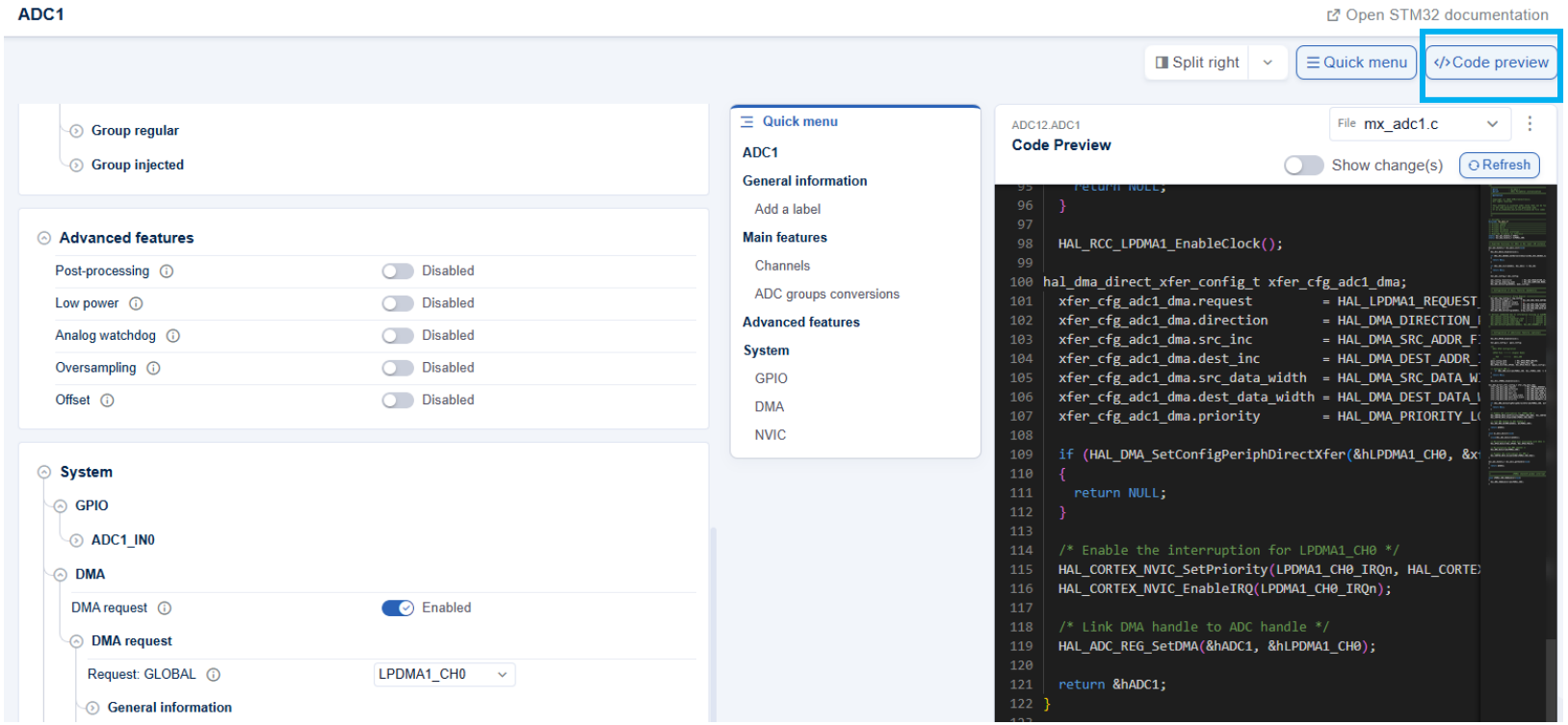


2 Go to system section and enabled DMA request



Code preview use case

3 Click on “Code preview” section



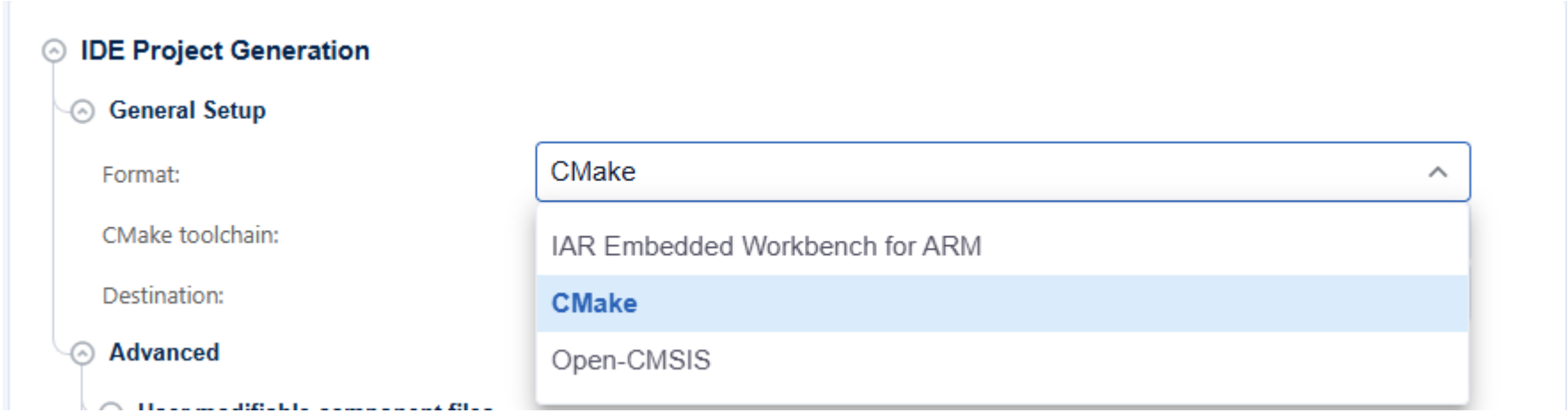
STM32CubeMX2 features: **Project settings**



Project settings: Available formats

STM32CubeMX2 offers the possibility to generate code for these three format

- **IAR Embedded Workbench for ARM**
- **CMake** (required for STM32Cube for VSCode and STM32CubeIDE v2.1.0)
- **Open-CMSIS (**compatible with Keil µVision and Keil Studio)

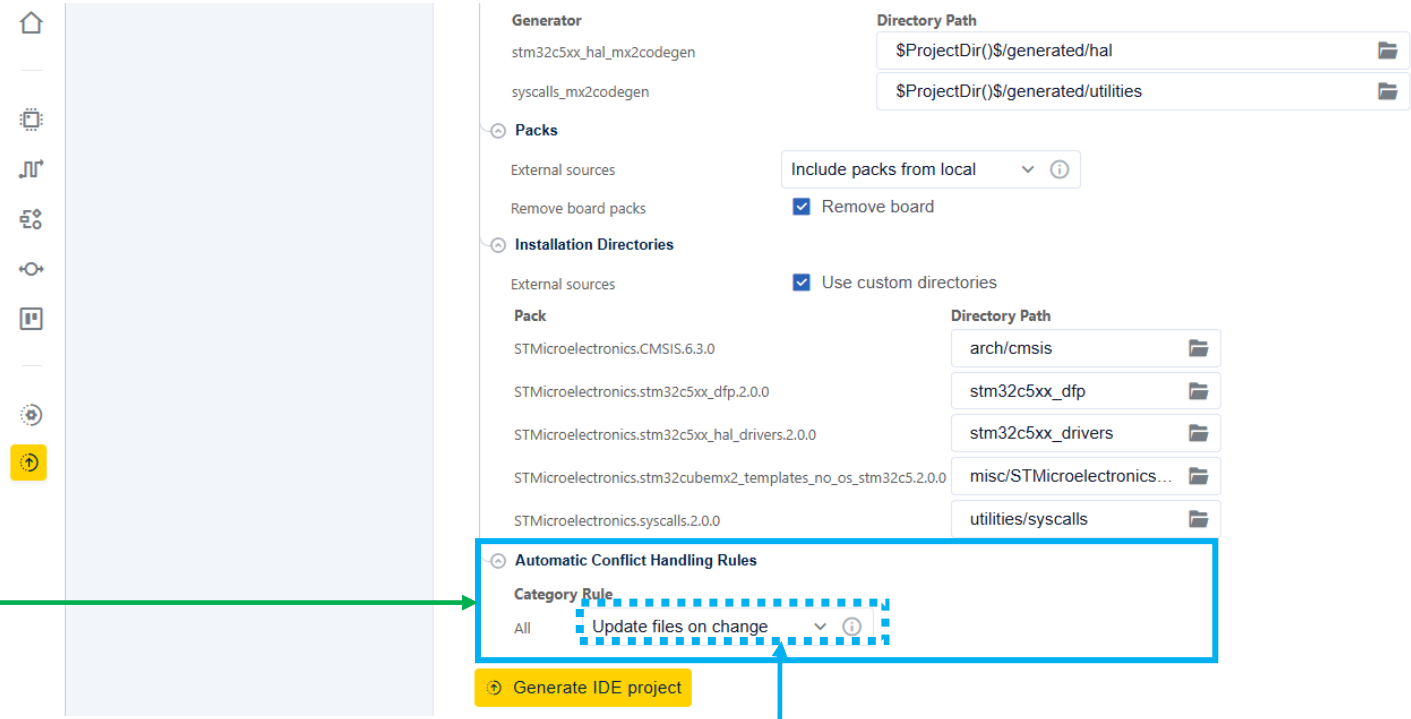


Project settings: *Automatic Conflict Handling Rules*

STM32CubeMX “**user code”** sections are removed and replaced by the “***Automatic Conflict Handling Rules***” feature

```
/* USER CODE BEGIN X */


/* USER CODE END X */
```

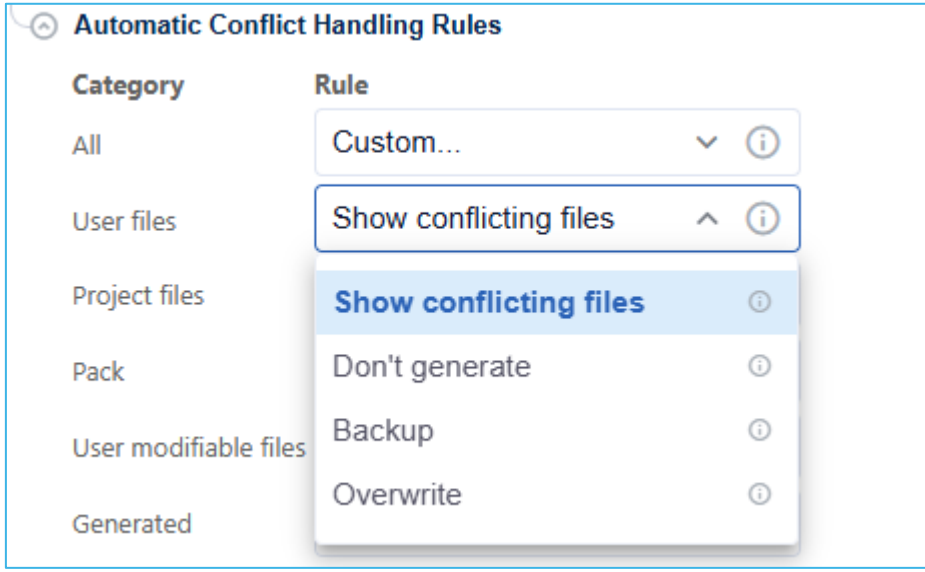


By default, all files are updated on change

Project settings: *Automatic Conflict Handling Rules*

Instead of "***update files on change***" option, the "***custom***" one can be used
➔to handle different types of **conflict resolution**

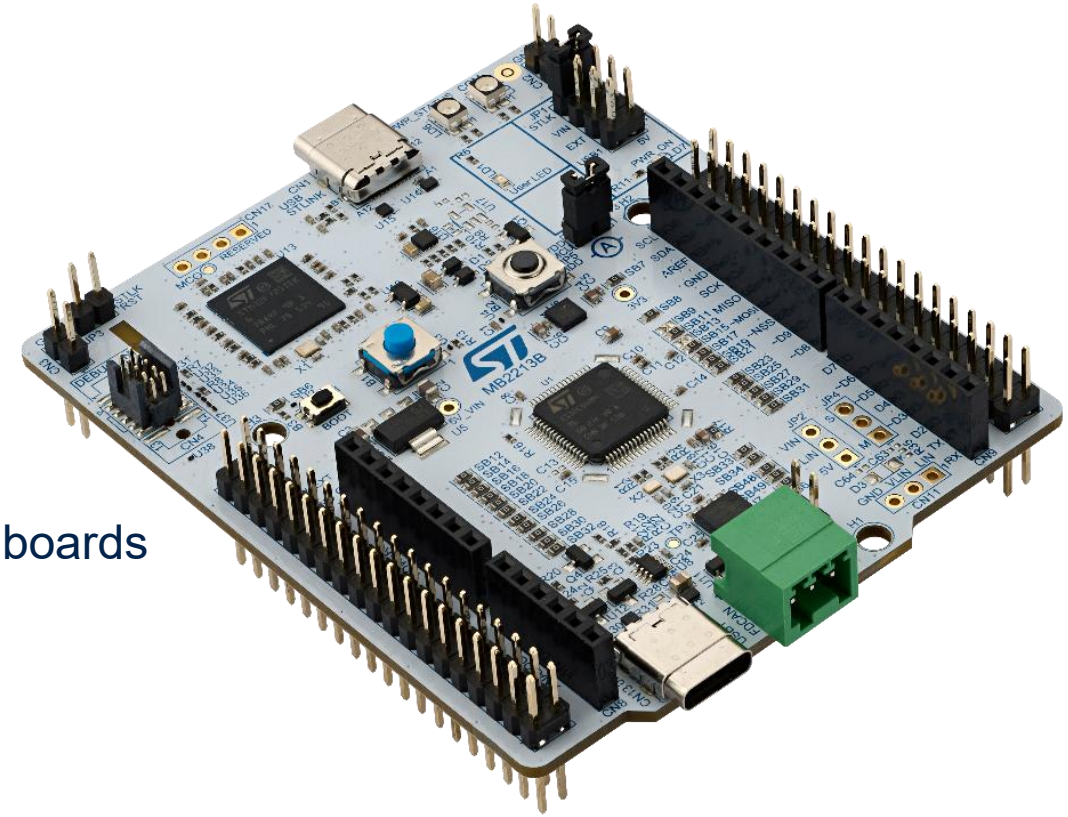
- **Show conflicting files** : launch the conflict resolution tool to handle each case
- **Don't generate** : Files will be kept unchanged
- **Backup** : Creates backup of existing files (with **_Bckp** suffix) and overwrites them
- **Overwrite** : Overwrites the existing



Hands-On Demo

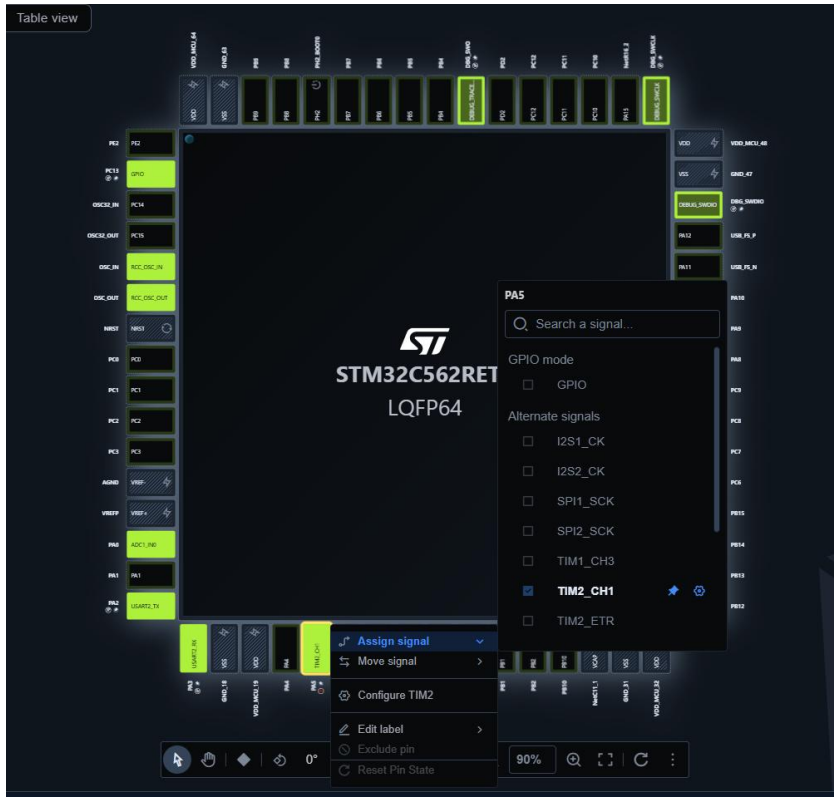
STM32C5/STM32CubeMX2 Demo

- In today's demo, we will
  - Use STM32CubeMX2 to configure the STM32C5
  - Use the HAL2 Drivers to:
    - Blink LED using PWM
    - Use ADC in Circular DMA mode
    - Use input button to trigger GPIO EXTI
    - Use UART Terminal to output status message
    - Use EEPROM Emulation to store data in Data Flash
    - Use FDCAN to transmit and receive message between boards



STM32C5/STM32CubeMX2 Demo

- LED/TIM2 Configuration
  - Use TIM2 to blink LED (PA5)
  - Enable PWM to blink the LED at a 50% duty cycle



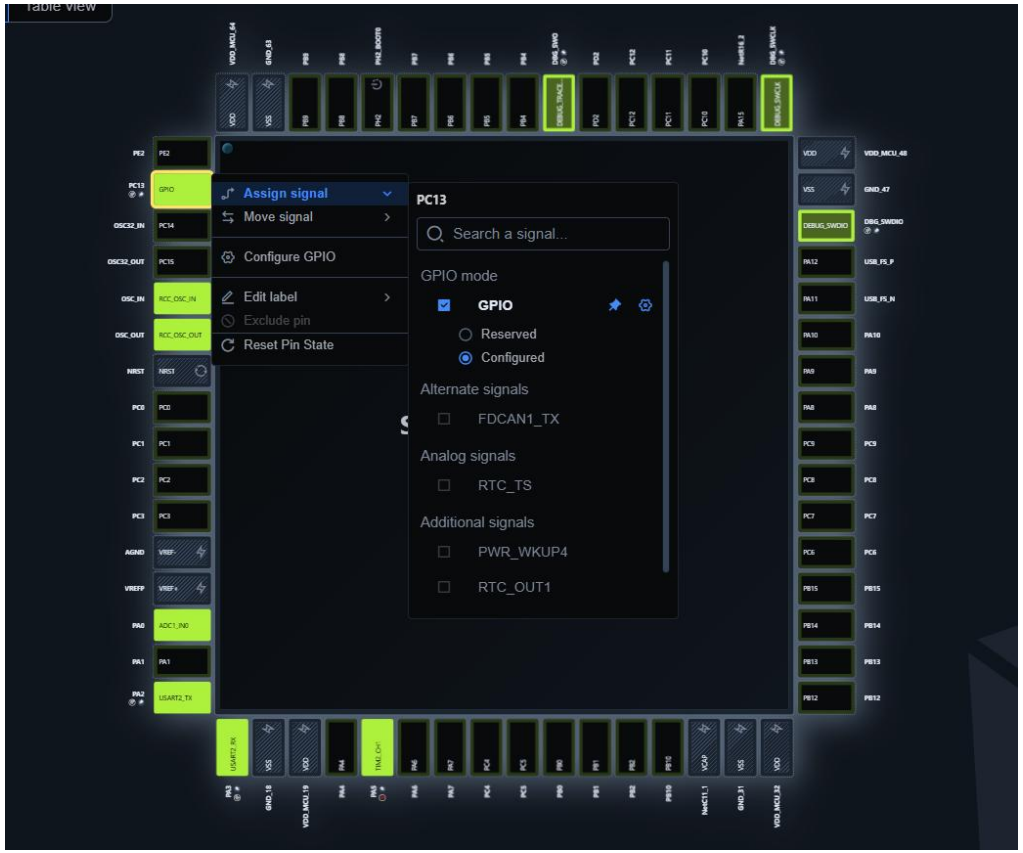
STM32C5/STM32CubeMX2 Demo

- ADC Configuration
  - Use ADC1 Channel 0 (PA0)
  - Transfer data continuously/circularly into DMA buffer



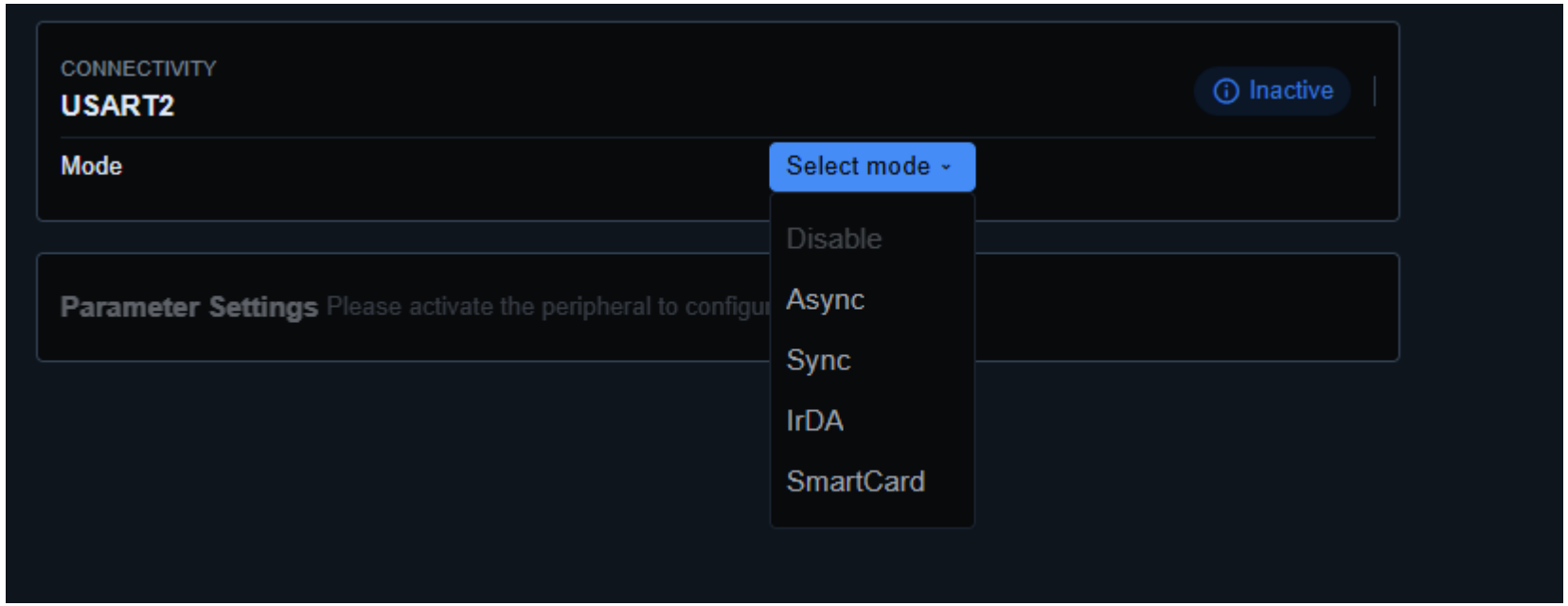
STM32C5/STM32CubeMX2 Demo

- GPIO Configuration
  - Configure GPIO as EXTI (PC13)
  - When the button is pressed the ADC buffer will be stored in the Emulated EEPROM



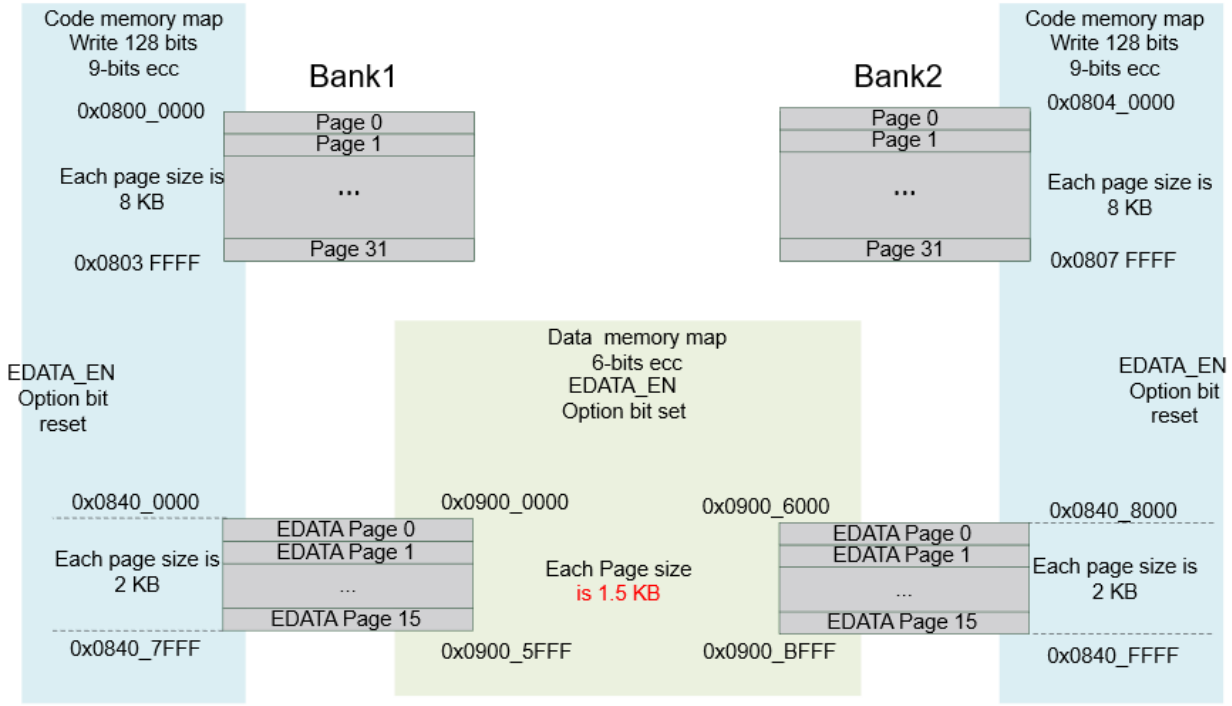
STM32C5/STM32CubeMX2 Demo

- USART
  - Enable USART to print status messages
  - We will redirect the printf function to USART in code



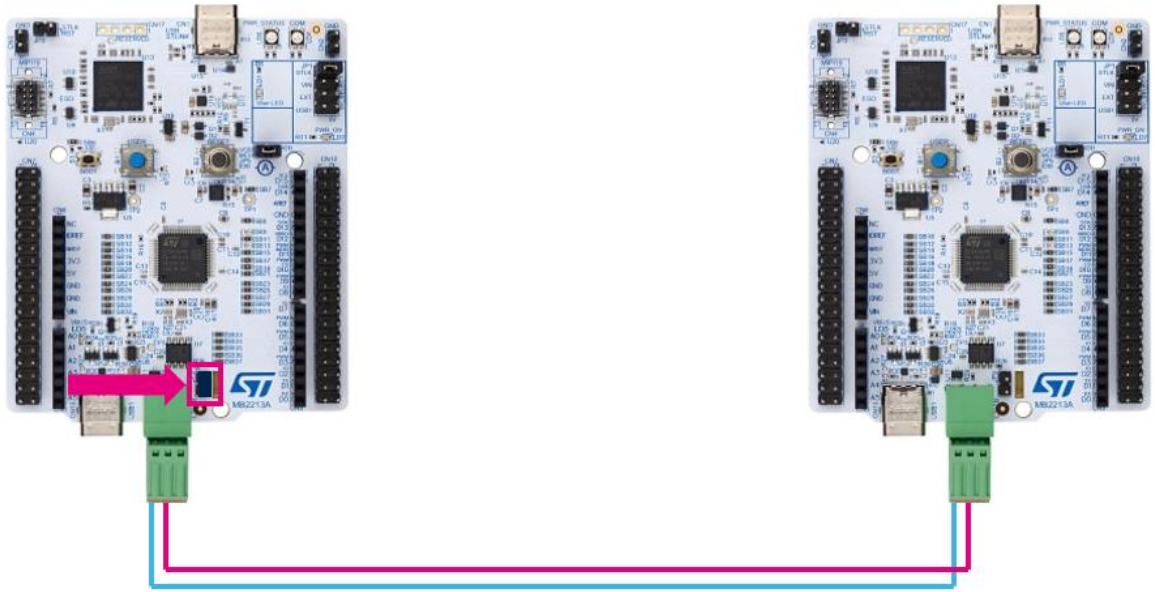
STM32C5/STM32CubeMX2 Demo

- Emulated EEPROM
  - Enable FLASH peripheral
  - Configure the MPU to make the EDATA address range non-cacheable
  - Set EDATA_EN Option Byte to 1



STM32C5/STM32CubeMX2 Demo

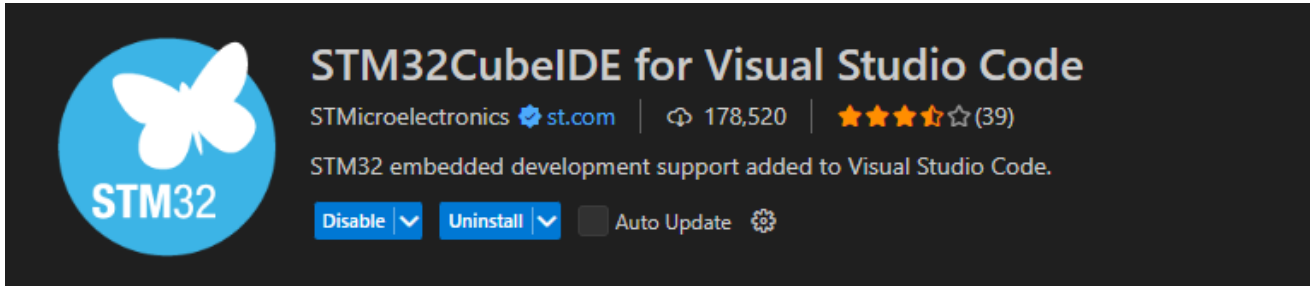
- FDCAN
  - Assign FDCAN signals to pins
  - Configure the external clock
  - Configure LED as GPIO
  - Configure FDCAN peripheral to transmit and receive data



Materials

- You will need:
  - STM32CubeMX2
  - VSCode with STM32CubeIDE Extension
  - STM32CubeProgrammer
  - NUCLEO-C562RE board
  - USB-C Cable
  - 1 M2M jumper cable to change ADC value
  - 1 F2F jumper cable for FDCAN

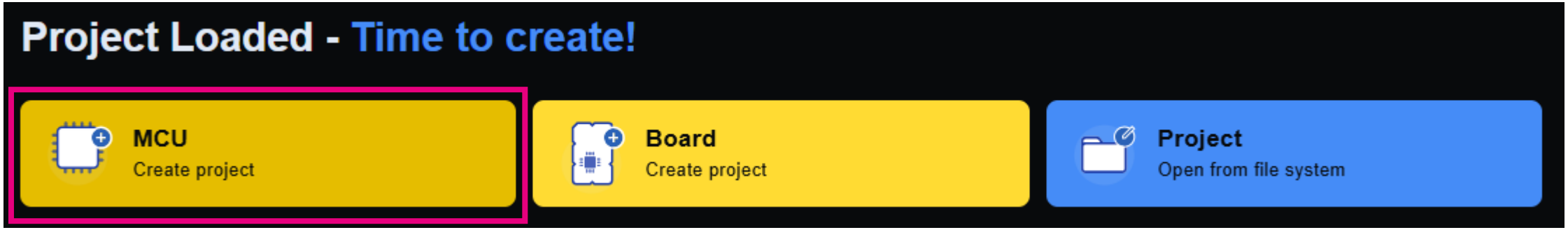




Creating a Project in CubeMX2

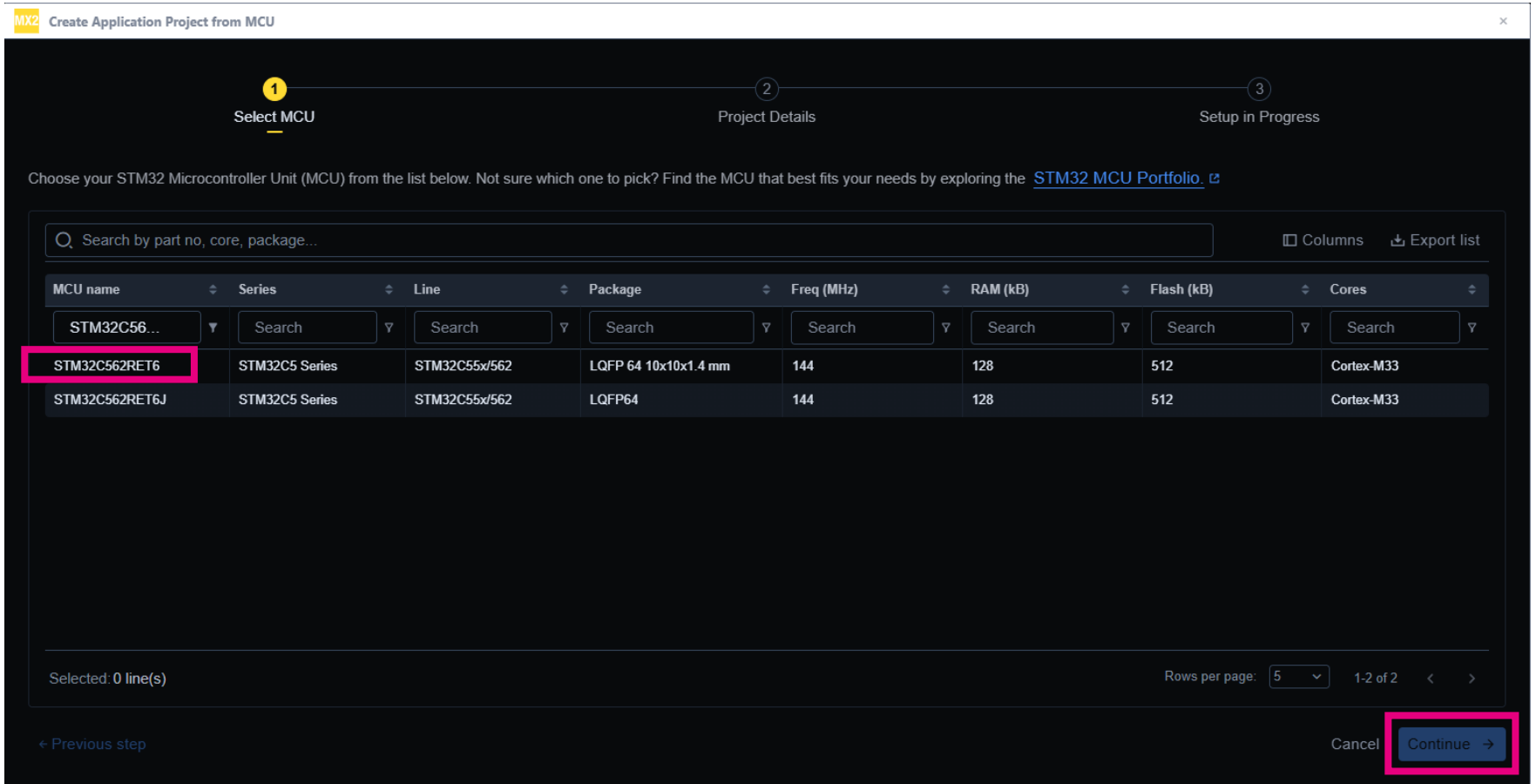
Creating MX2 Project

- Select **MCU** when creating a new project to reference the NUCLEO board pinout



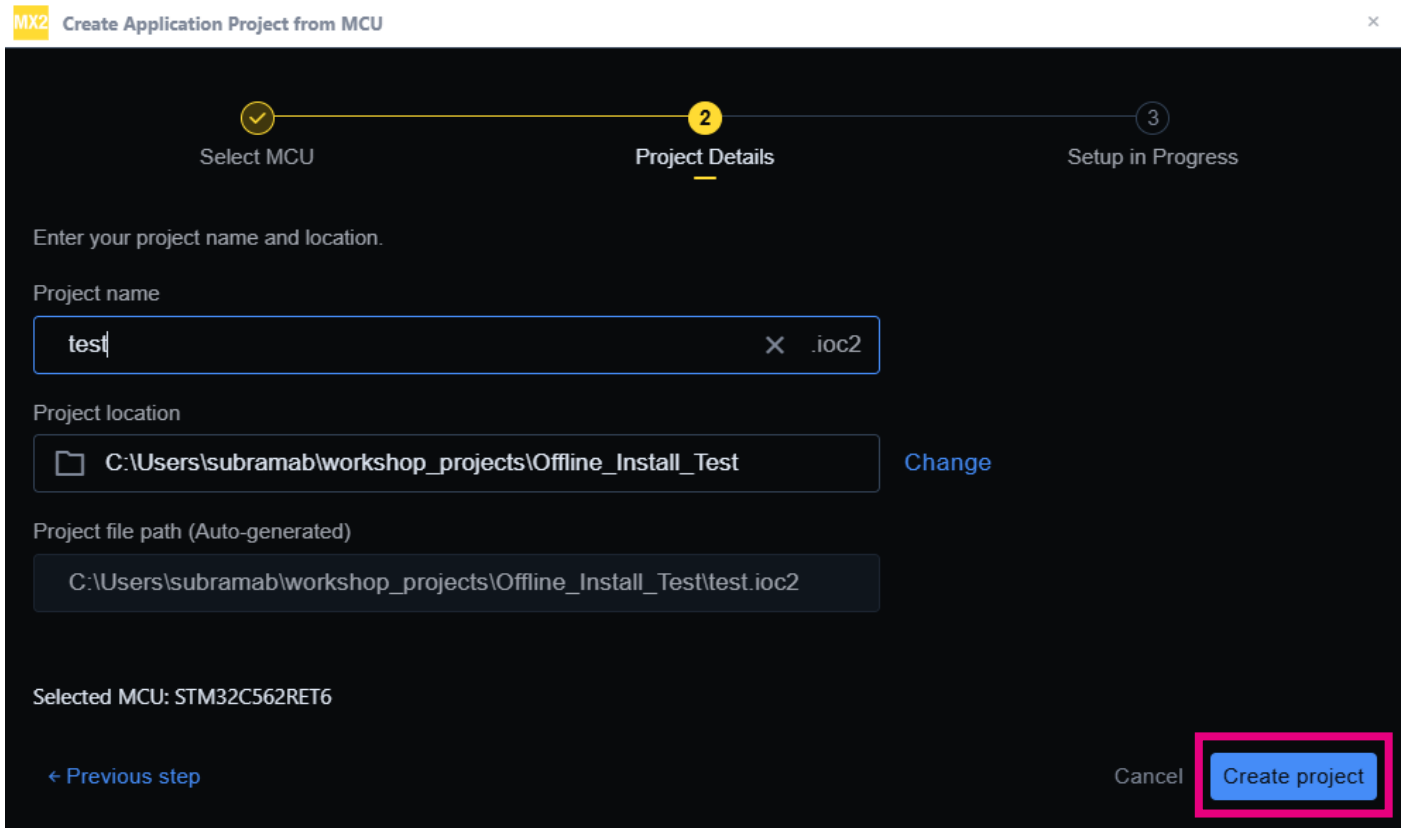
Creating MX2 Project

- Under MCU name select **STM32C562RET6**
- Once you have done that hit **Continue**



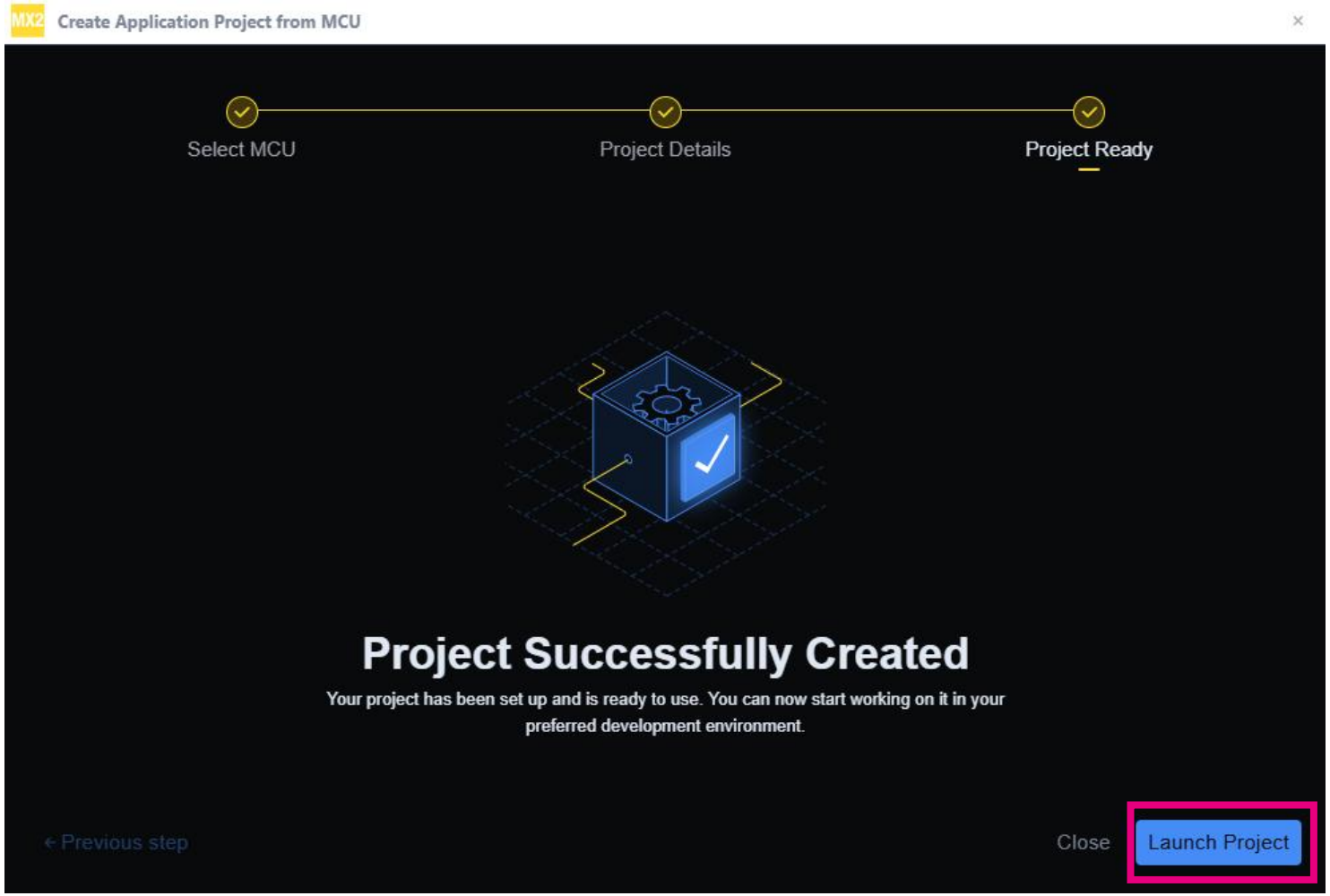
Creating MX2 Project

- Name your project and select the project location to save it in
- Once you have done that, select **Create Project**



Creating MX2 Project

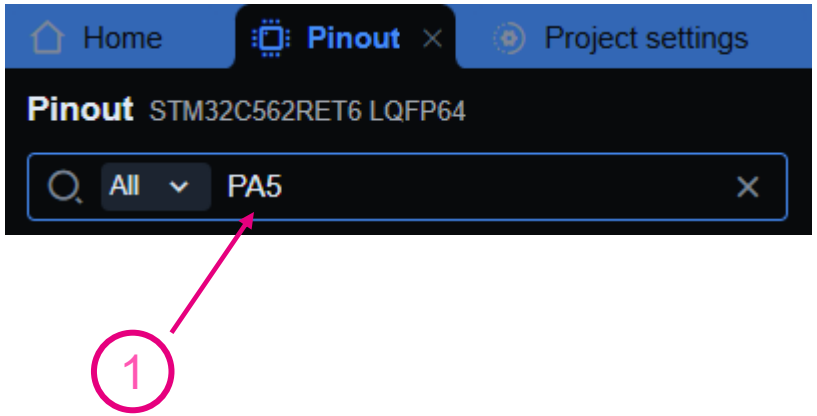
- Once the project has been successfully created you can select **Launch Project**

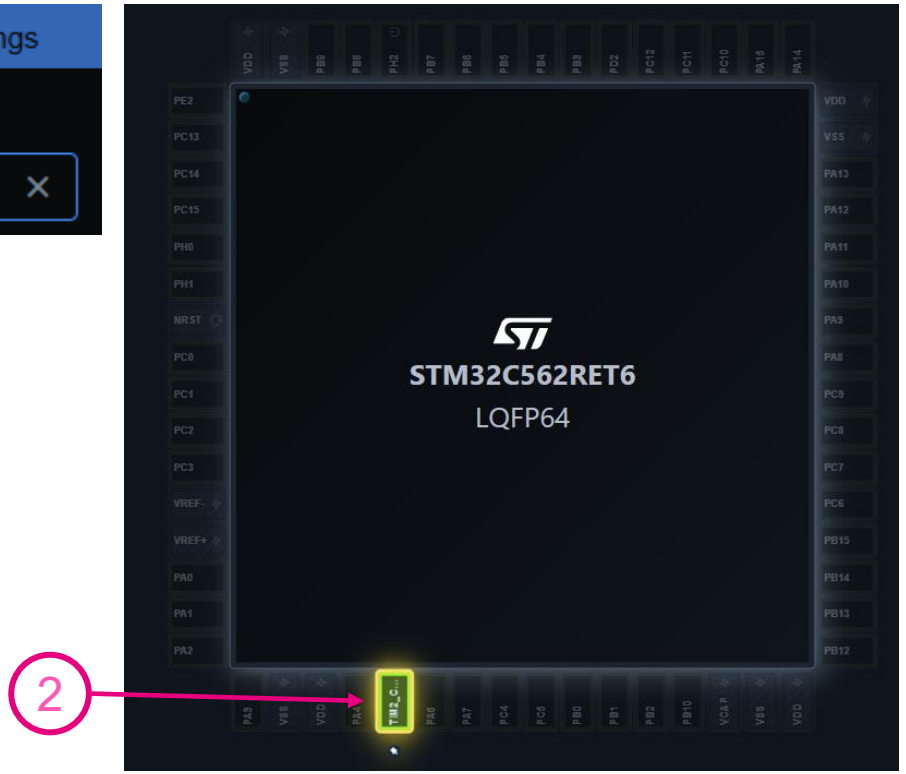


Lab 1: Blink LED with Timer

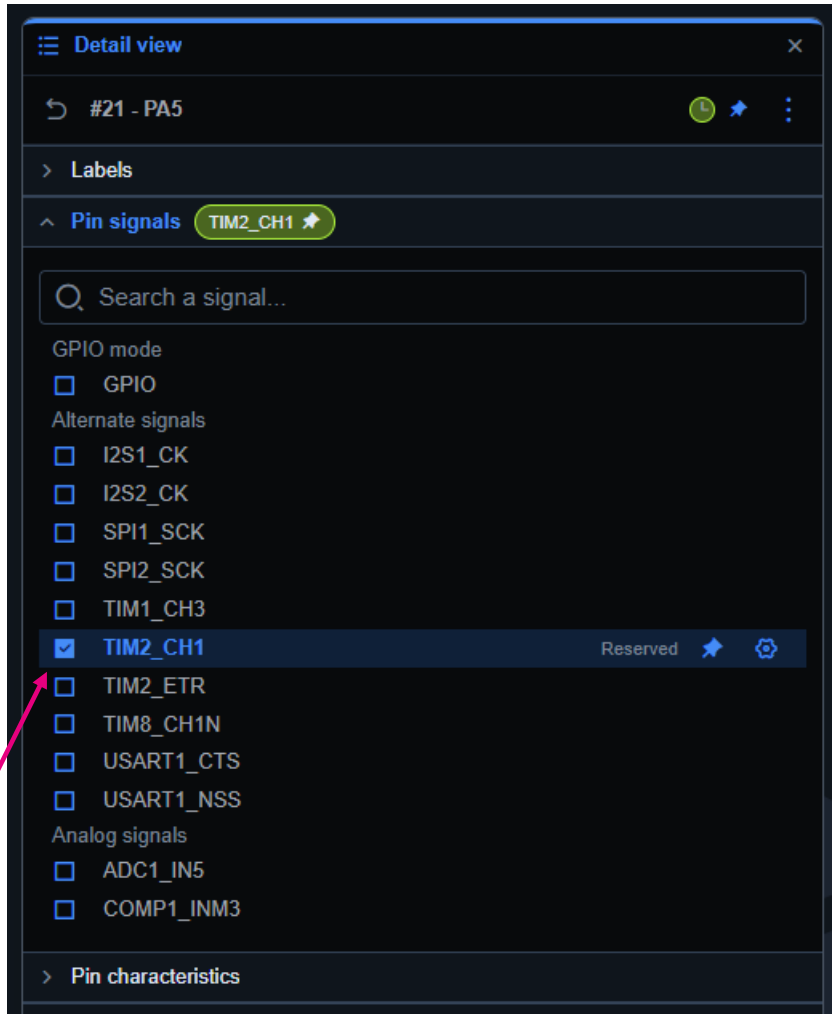
STM32CubeMX2 Timer Configuration

- Enable Timer 2 Channel 1 and output PWM signal on PA5.



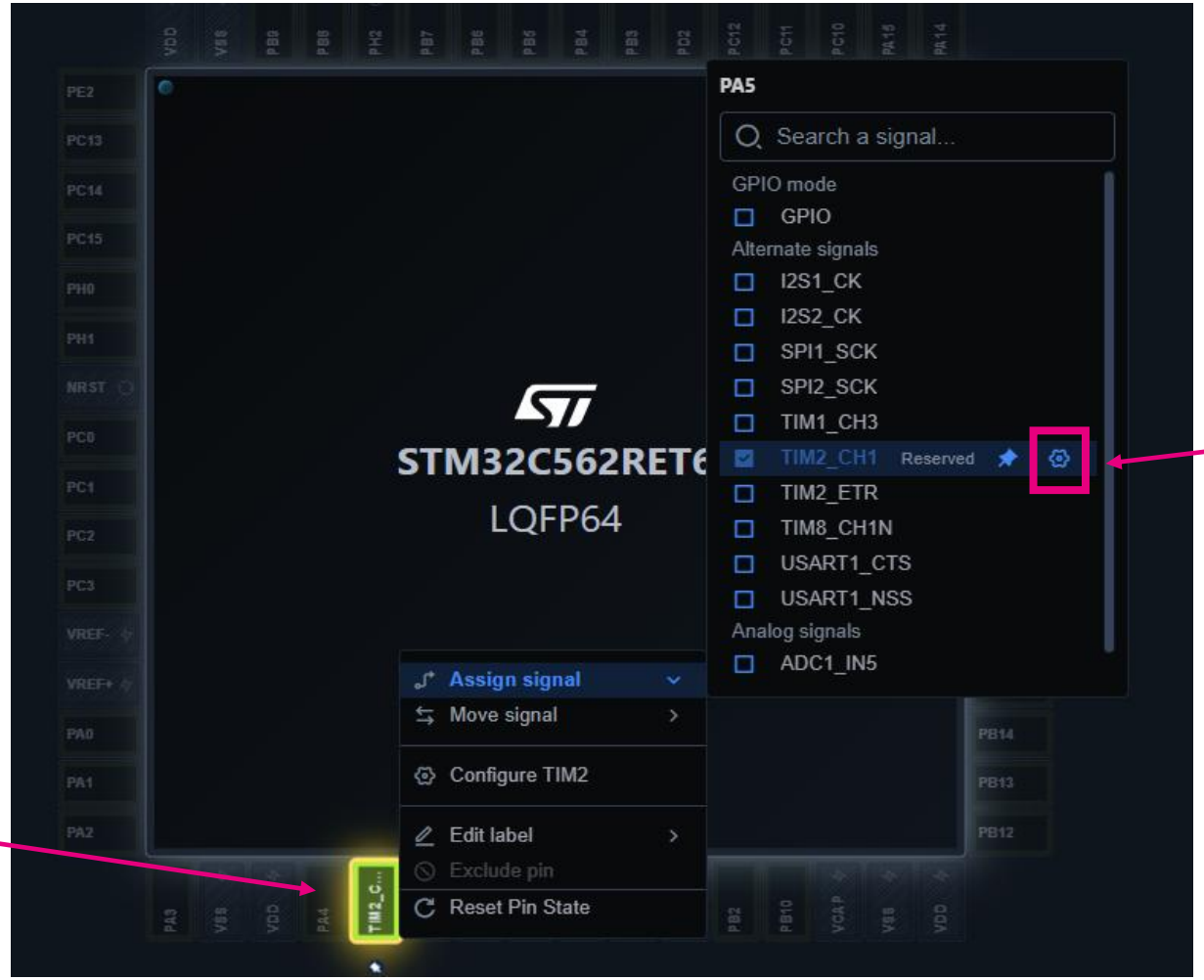


Right - click on pin



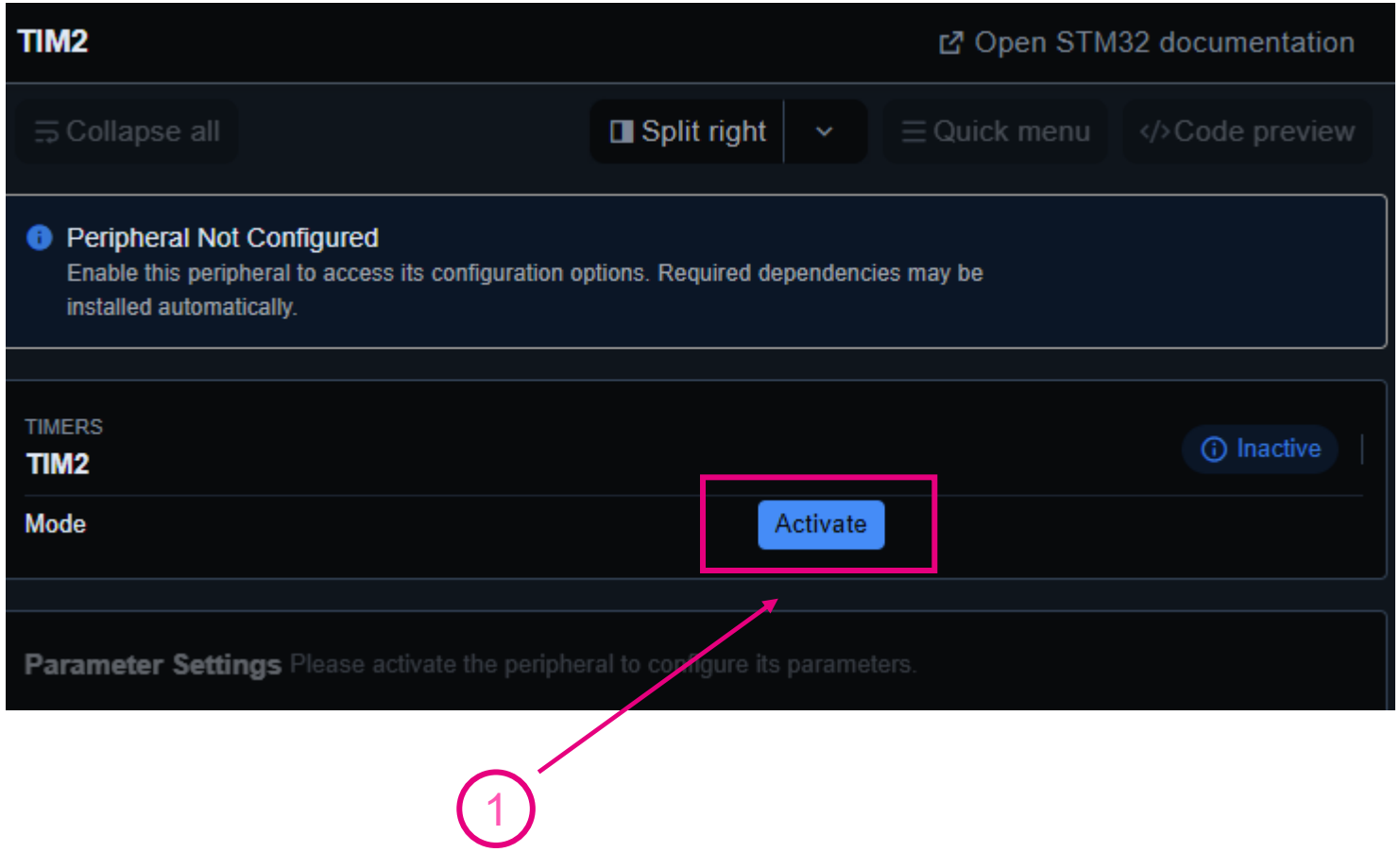
Timer Configuration

- Right-click on **PA5.** Under assign signal **TIM2_CH1** output, click on the gear icon to make further changes to the TIM2 peripheral



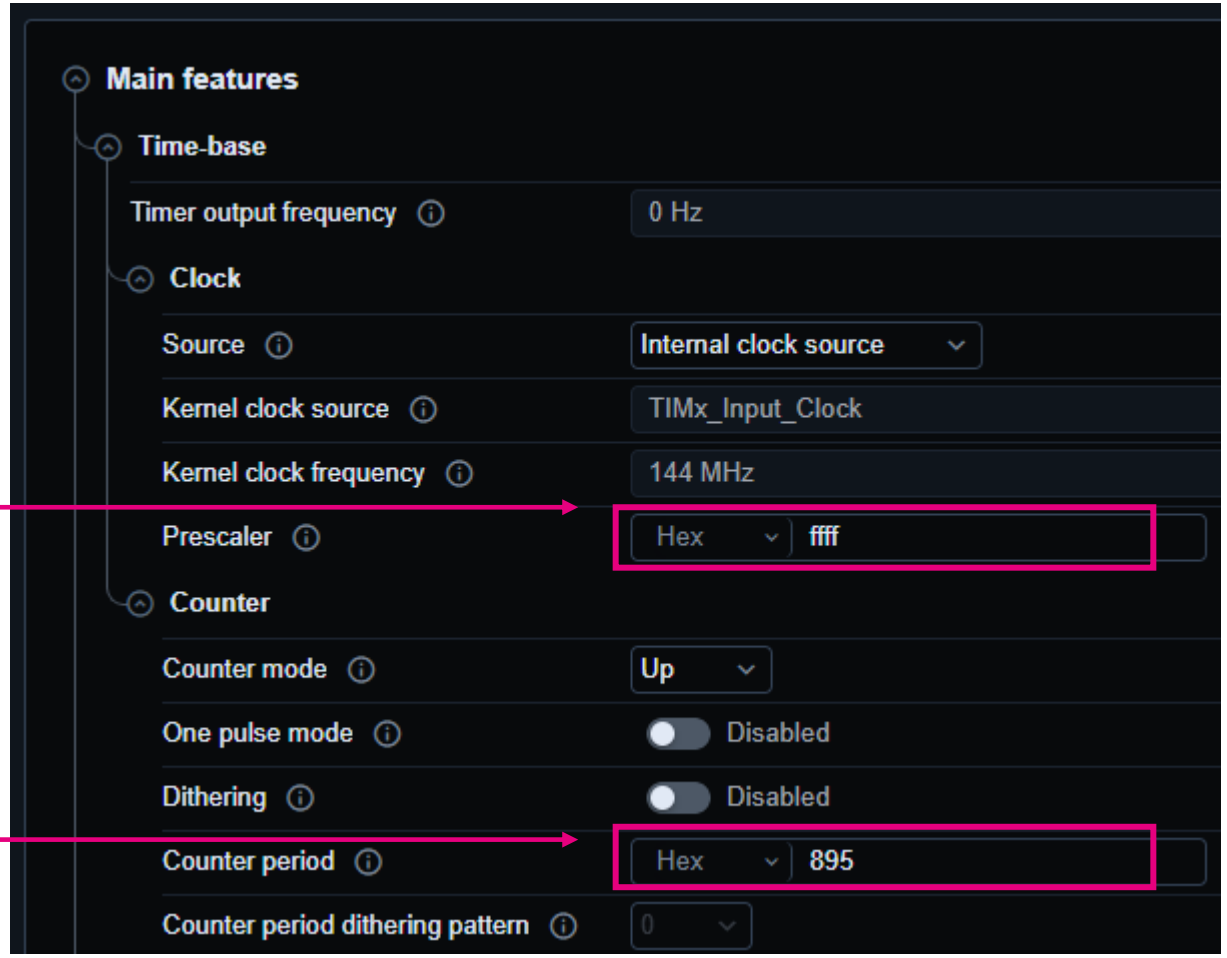
Timer Configuration

- Click on Activate



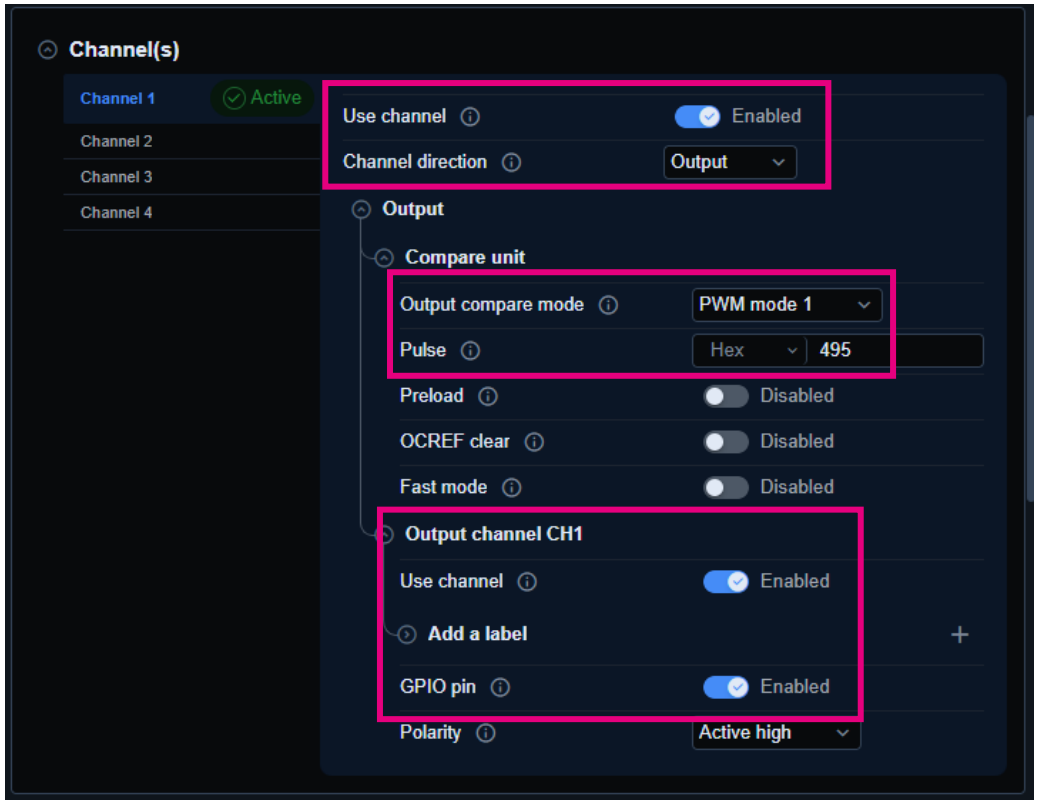
Timer Configuration

- **TIM2** Configure Time-base with the below parameters:
  - Time-base - Clock - Prescaler: **0xffff**
  - Time-base - Counter - Counter Period: **0x895**



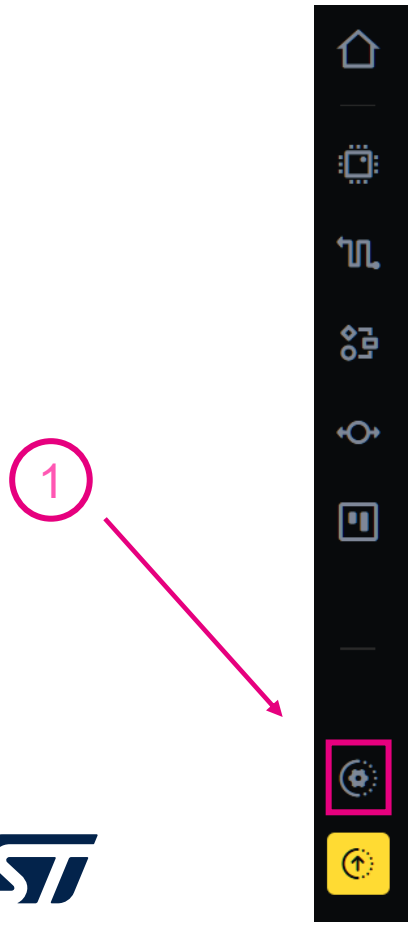
Timer Configuration

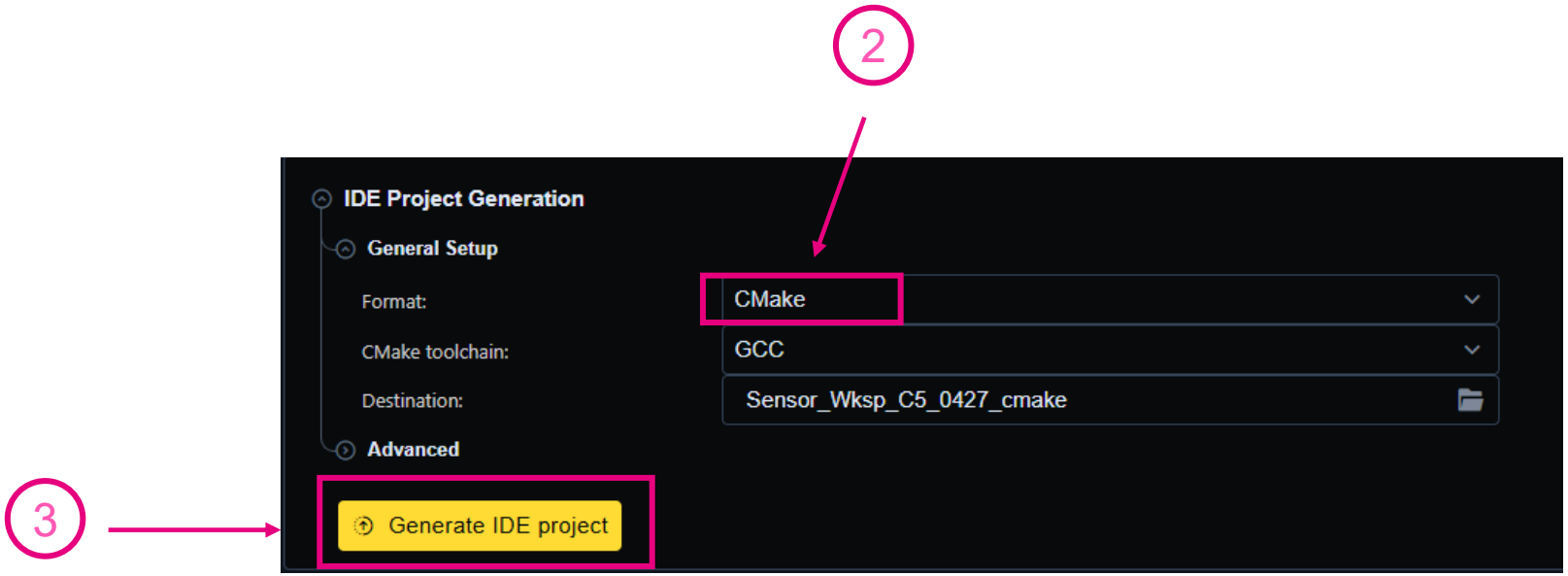
- Configure **Channel 1** with the below parameters:
  - Use channel: **Enabled**
  - Channel direction: **Output**
  - Output compare mode: **PWM mode 1**
  - Pulse: **0x495**
  - Output channel CH1 > Use channel: **Enabled**
  - Output channel CH1 > GPIO pin: **Enabled**



Generating CMake Project

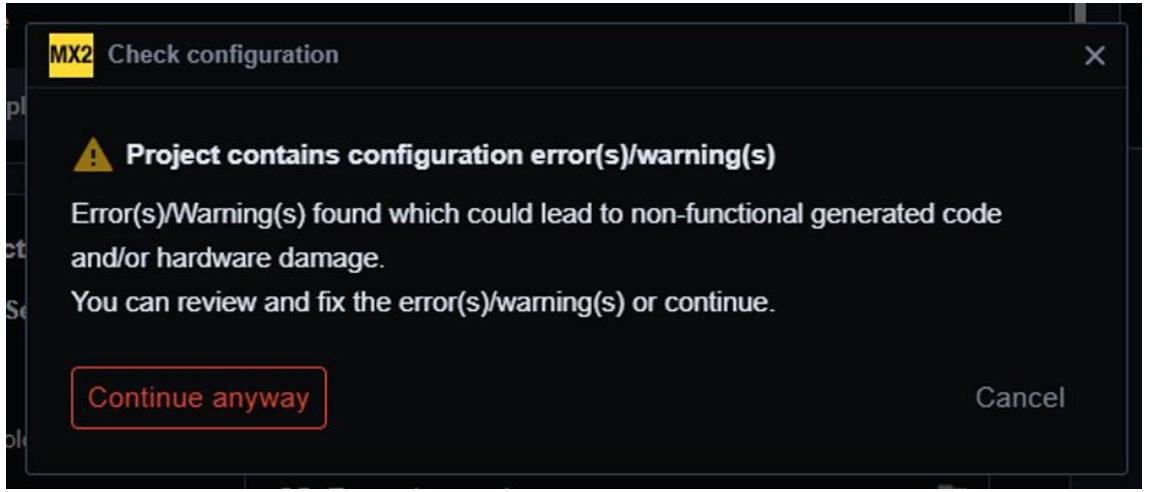
- Return to project settings, make sure **CMake** is selected as the format and click **Generate IDE project**



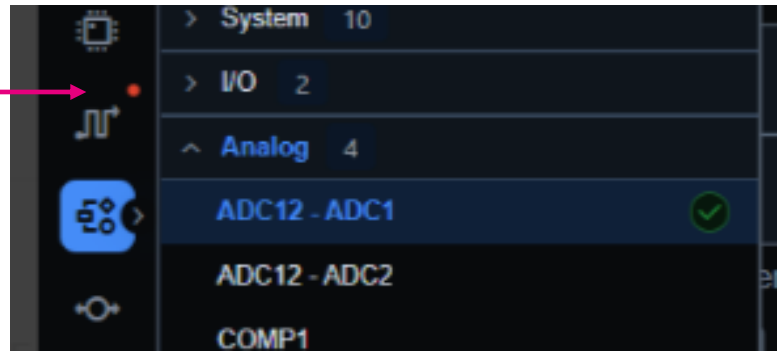


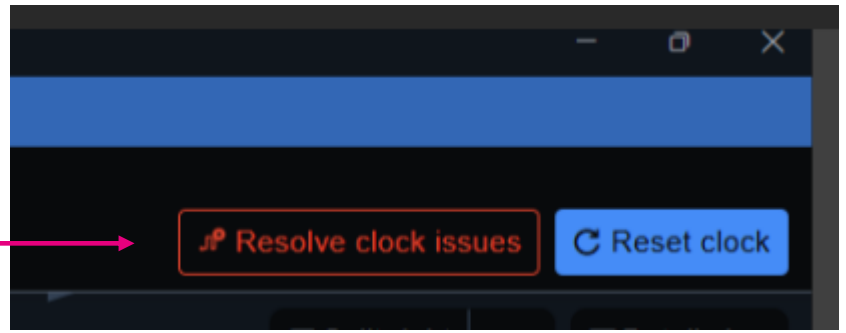
Generating CMake Project

- If seeing an error message:



- Check for a red dot by the clocks and fix by entering clock window and selecting resolve clock issues near top right corner.

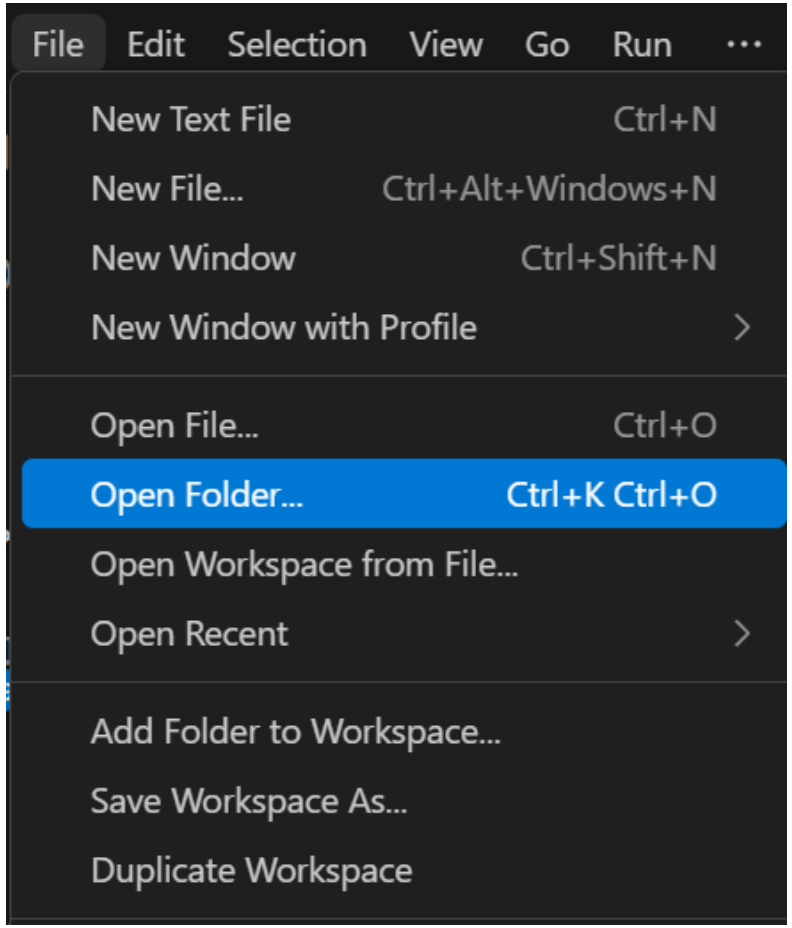




VSCode Project Setup

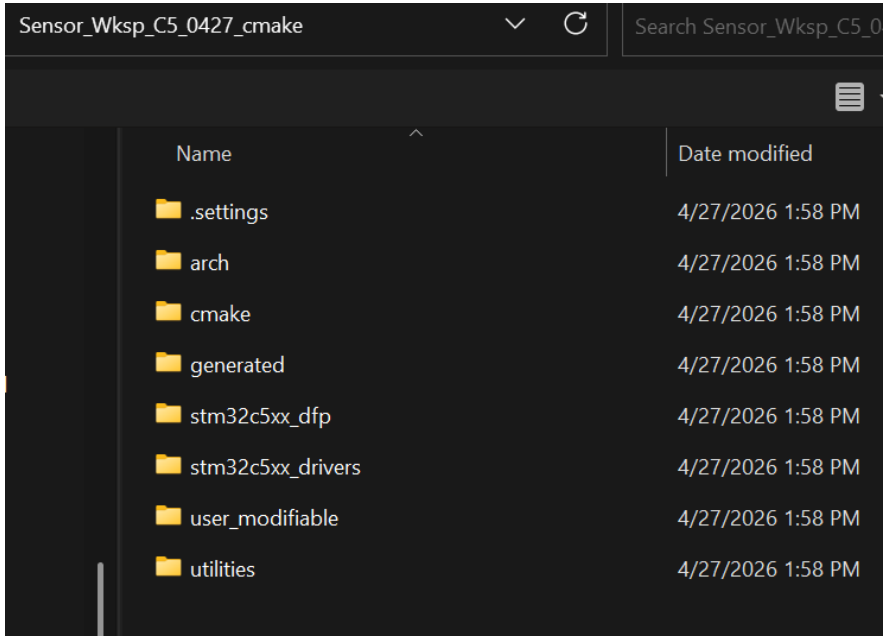
Project setup in VSCode

- In the VSCode application go to **File > Open Folder**



Project setup in VSCode

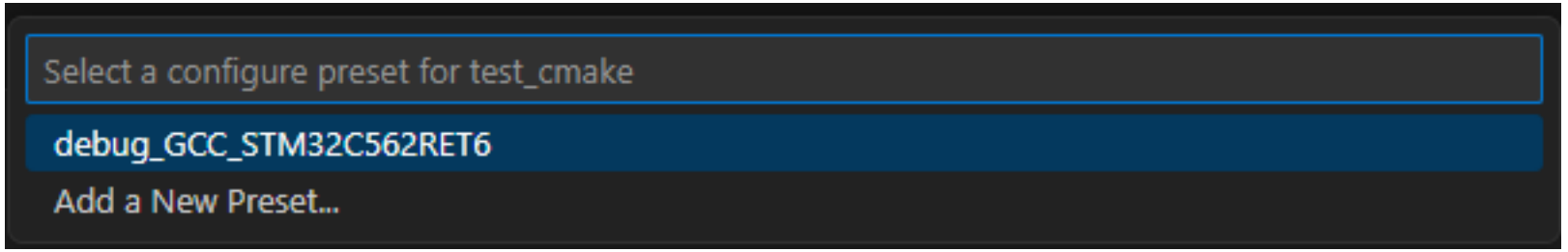
- Select the folder generated by STM32CubeMX2 ending in _cmake



NOTE: check under the VSCode Extensions tab to ensure that your extensions are not restricted

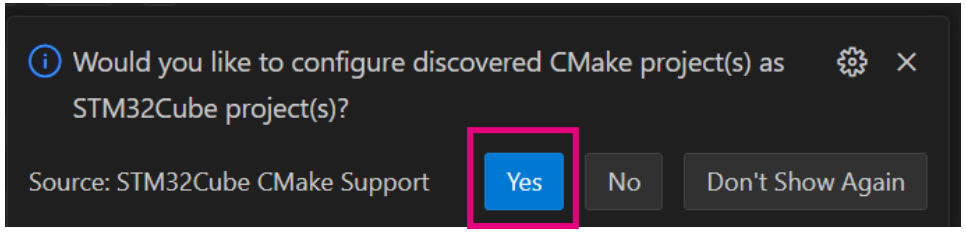
Project setup in VSCode

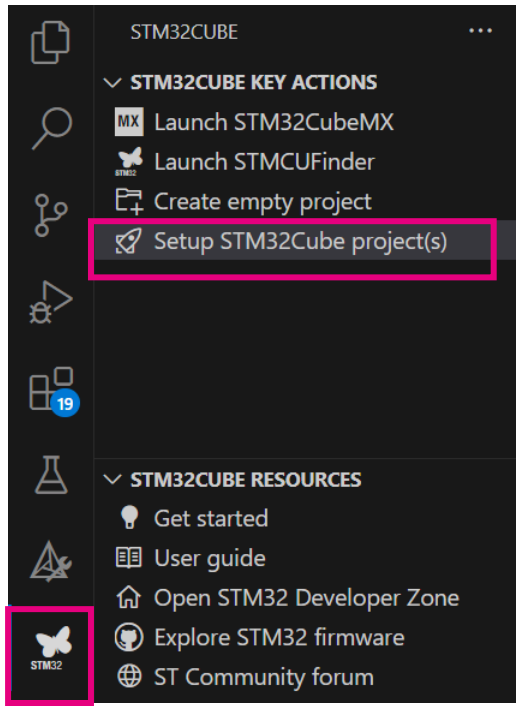
- During the project set up, you will see a drop down to select a configure preset. Select **debug_GCC_STM32C562RET6**



Project setup in VSCode

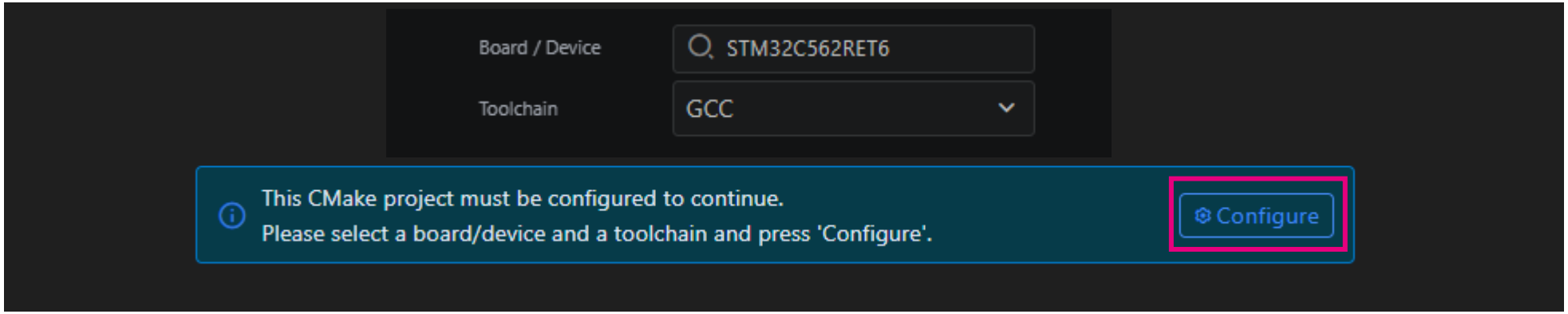
- You may be prompted on the bottom right to setup your STM32Cube Project
  - If you are not prompted on the bottom, you can access the page from **Setup STM32Cube project(s)** in the **STM32 VSCode Extension** Tab





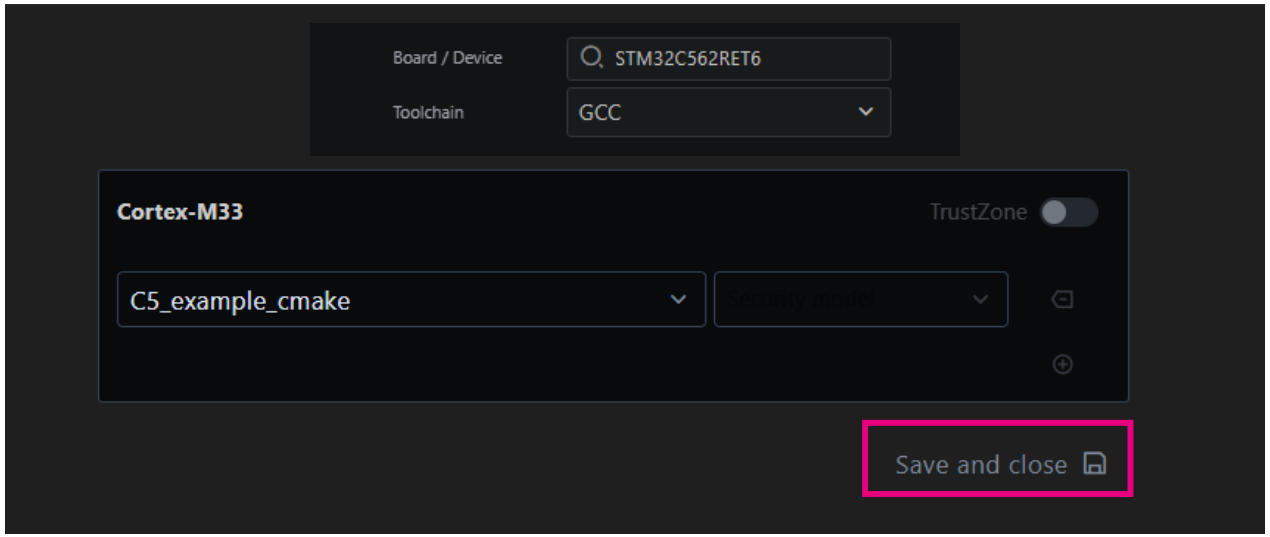
Project setup in VSCode

- From the Setup Tab set the Board/Device to **STM32C562RET6** and select **Configure**



Project setup in VSCode

- Wait for the **Save and close** button to appear, and select **Save and close**



Adding Application Code

- Before **main()**
  - Add the timer handle declaration

```
#include "main.h"

/* Private typedef -----------------------------------------------------------*/
/* Private define ------------------------------------------------------------*/
/* Private macro -------------------------------------------------------------*/
/* Private variables ---------------------------------------------------------*/
hal_tim_handle_t* htim2;
/* Private functions prototype -----------------------------------------------*/

/**
  * brief:  The application entry point.
  * retval: none but we specify int to comply with C99 standard
  */
```

Adding Application Code

- In **main()**
  - Get the timer peripheral's handle using gethandle()
  - Start Channel1 of TIM2
  - Start TIM2

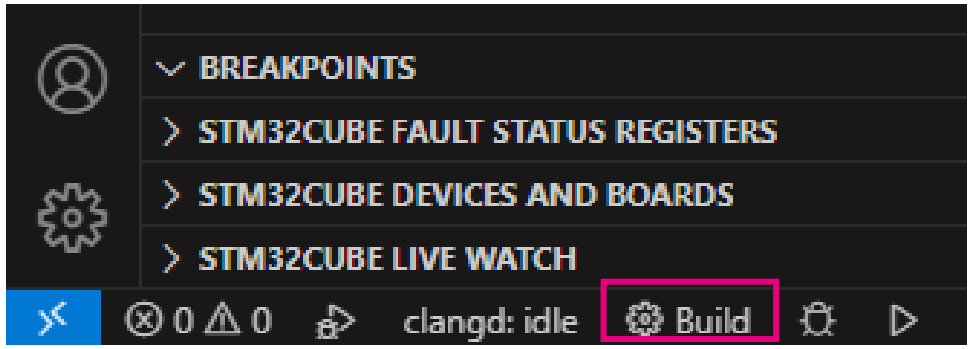
```
int main(void)
{
  /** System Init: this code placed in targets folder
    * It calls the initialization (and sets the initial
    * You can use STM32CubeMX to generate and call this
    * It also contains the HAL initialization and the
    */
  if (mx_system_init() != SYSTEM_OK)
  {
    return (-1);
  }
  else
  {
    /*
      * You can start your application code here
      */
    htim2 = mx_tim2_gethandle();

    HAL_TIM_OC_StartChannel(htim2, HAL_TIM_CHANNEL_1);
    HAL_TIM_Start(htim2);

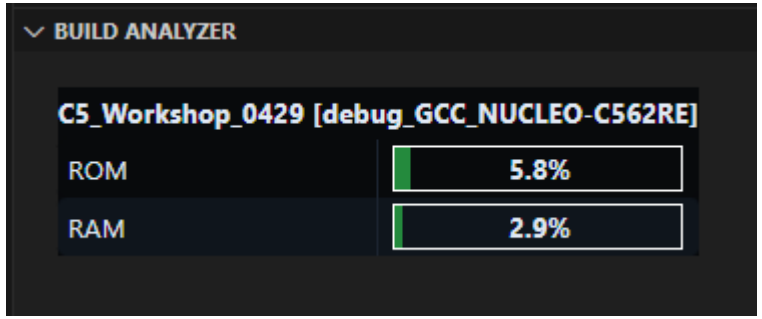
    while (1) {}
  }
} /* end main */
```

Building your project

- Select **Build** in the bottom left-hand corner of the screen

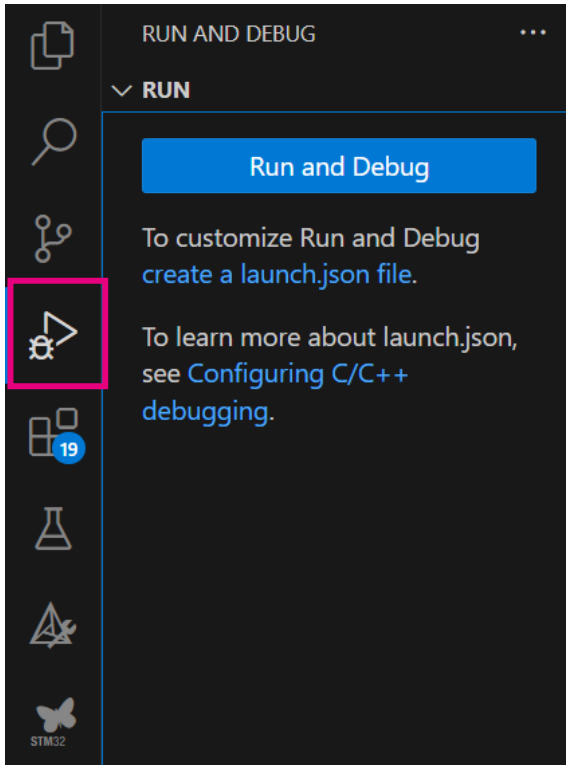


- Once you build you will see the Build Analyzer display ROM and RAM usage



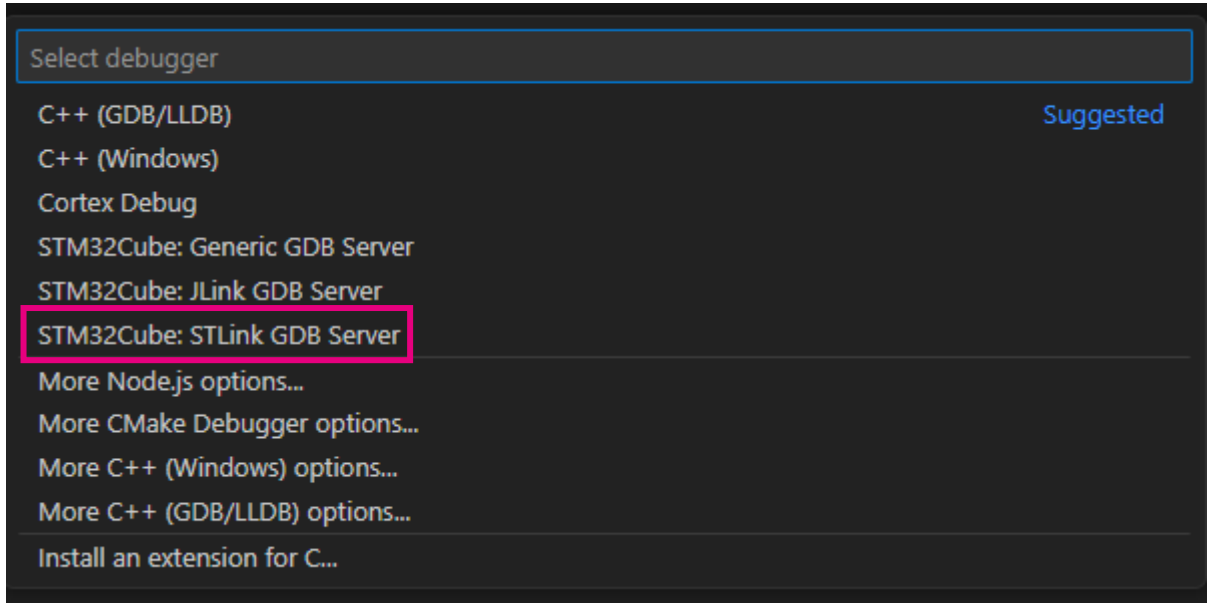
Running and debugging your project

- To run and debug your project select **Run and Debug** on the left-side toolbar and click the blue button



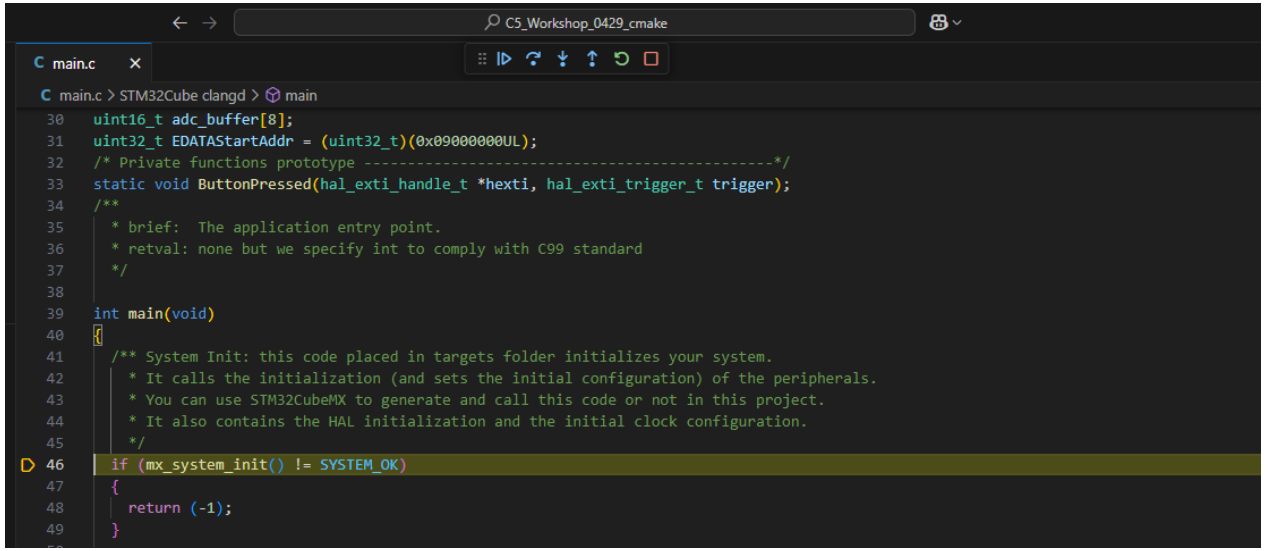
Running and debugging your project

- Select **STM32Cube: STLink GDB Server** as the debugger



Running and debugging your project

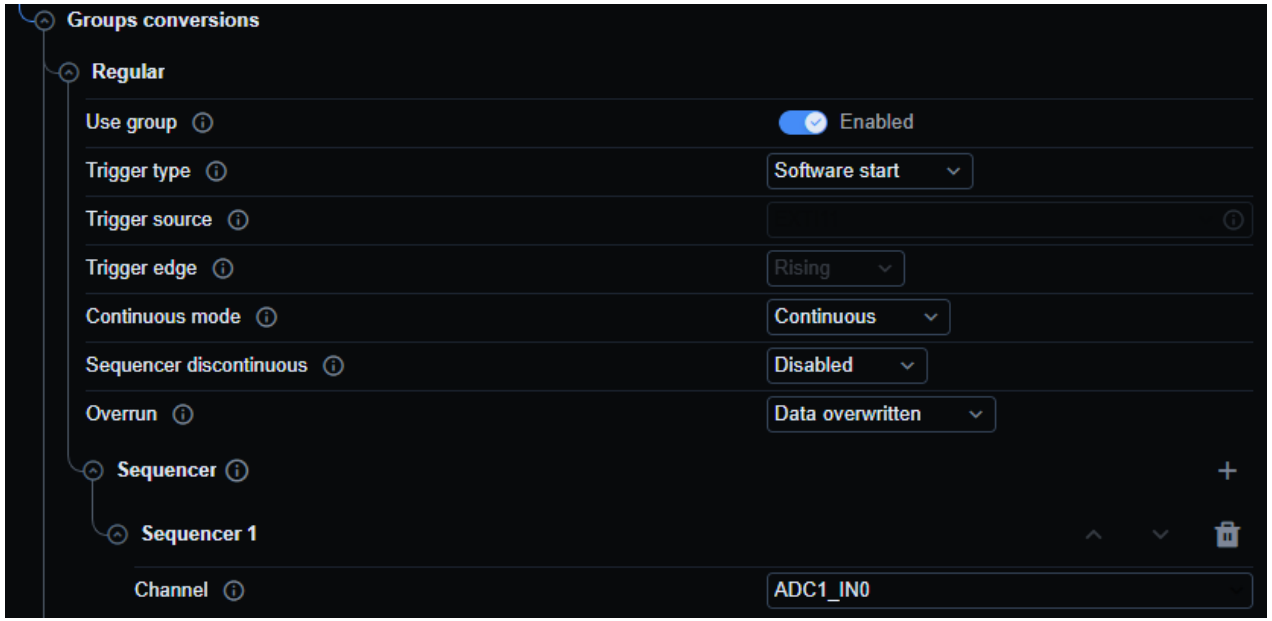
- Your project will stop at the initial breakpoint
- Hit **Continue** to resume execution
- You should see the LED toggling



Lab 2: ADC implementation

STM32CubeMX2 ADC Configuration

- Configure the ADC to continuously update the circular DMA buffer

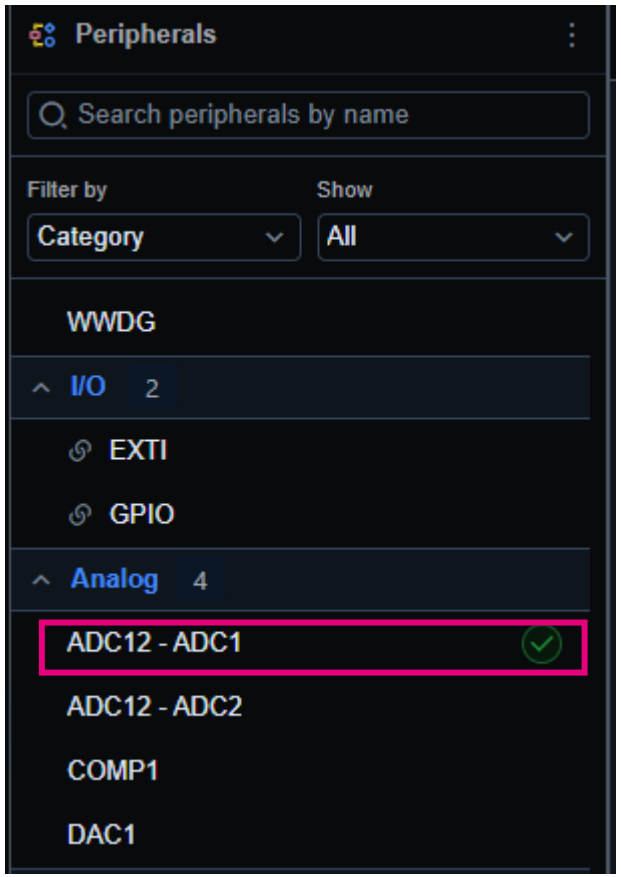




ADC Configuration

- Select **ADC1** under **Peripherals > Analog** to configure the ADC

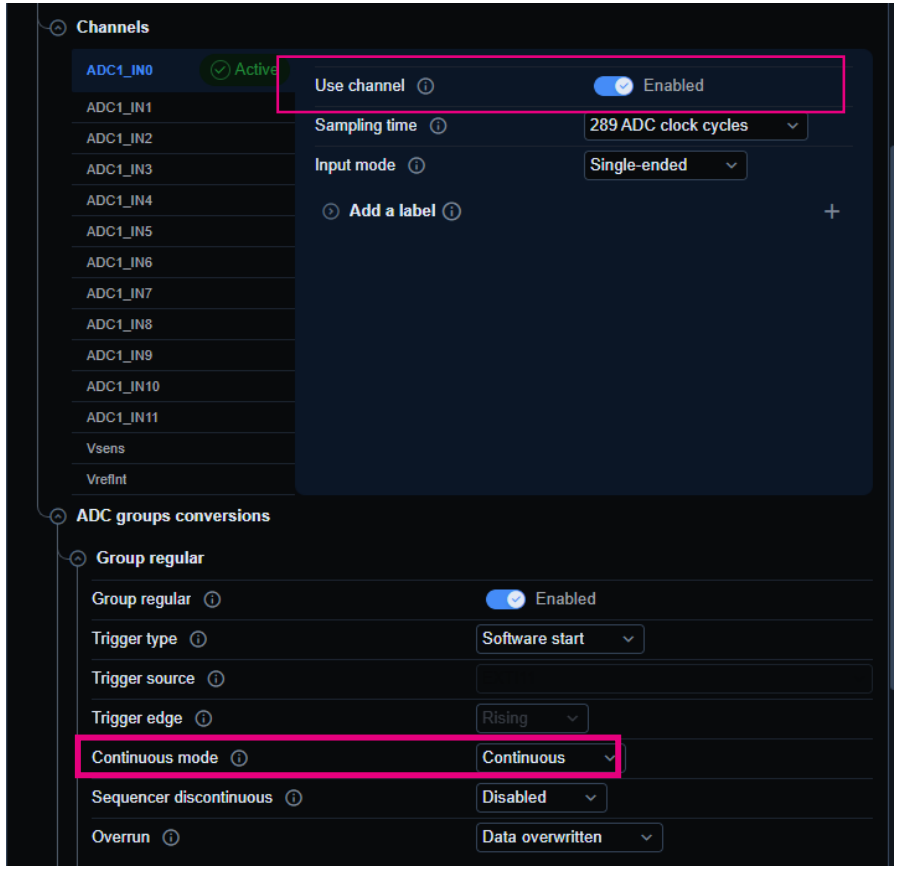






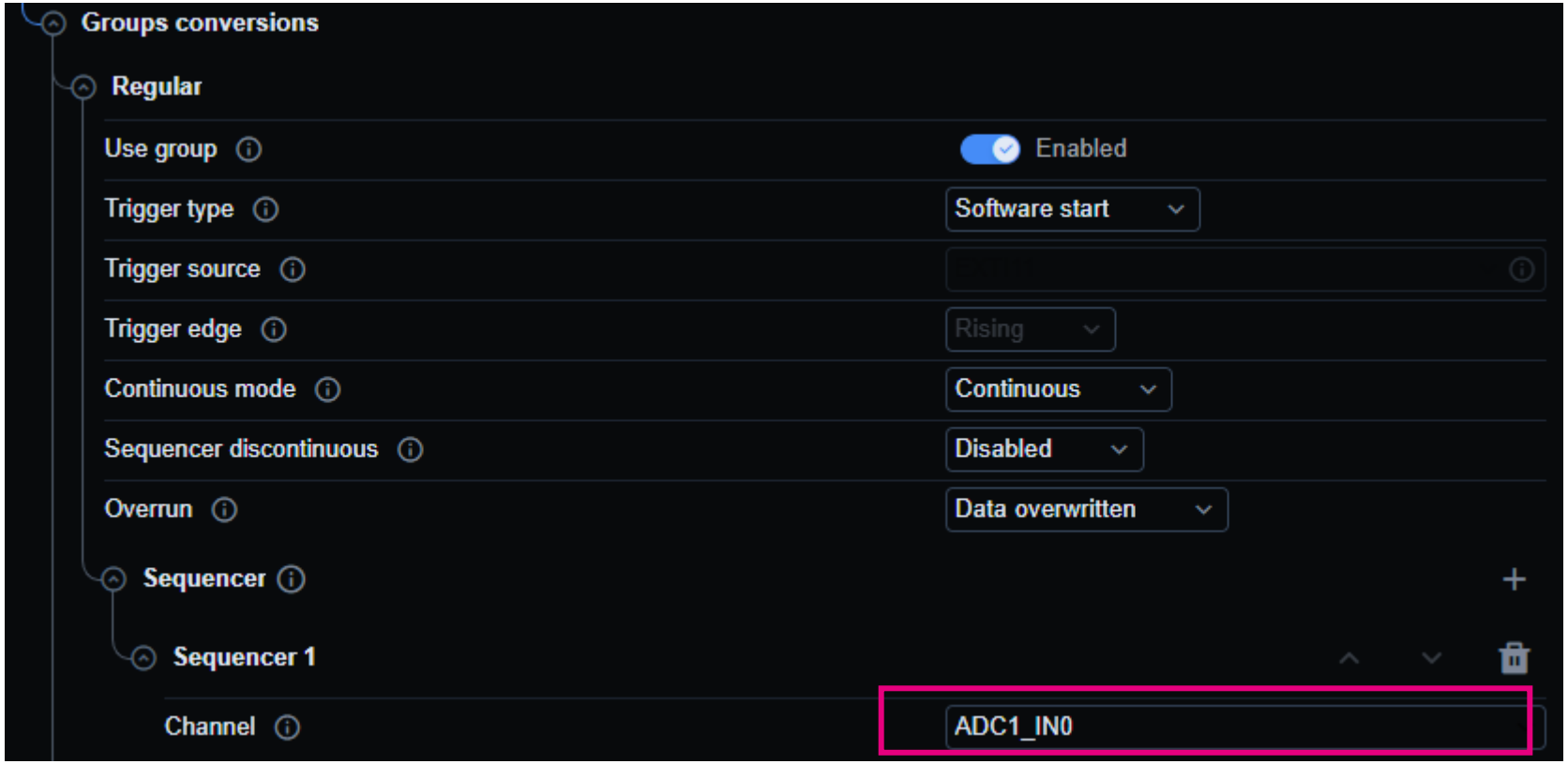
ADC Configuration

- Configure **ADC1_IN0**:
  - Use channel : **Enabled**
- Configure **ADC groups conversions:**
  - Continuous mode: **Continuous**



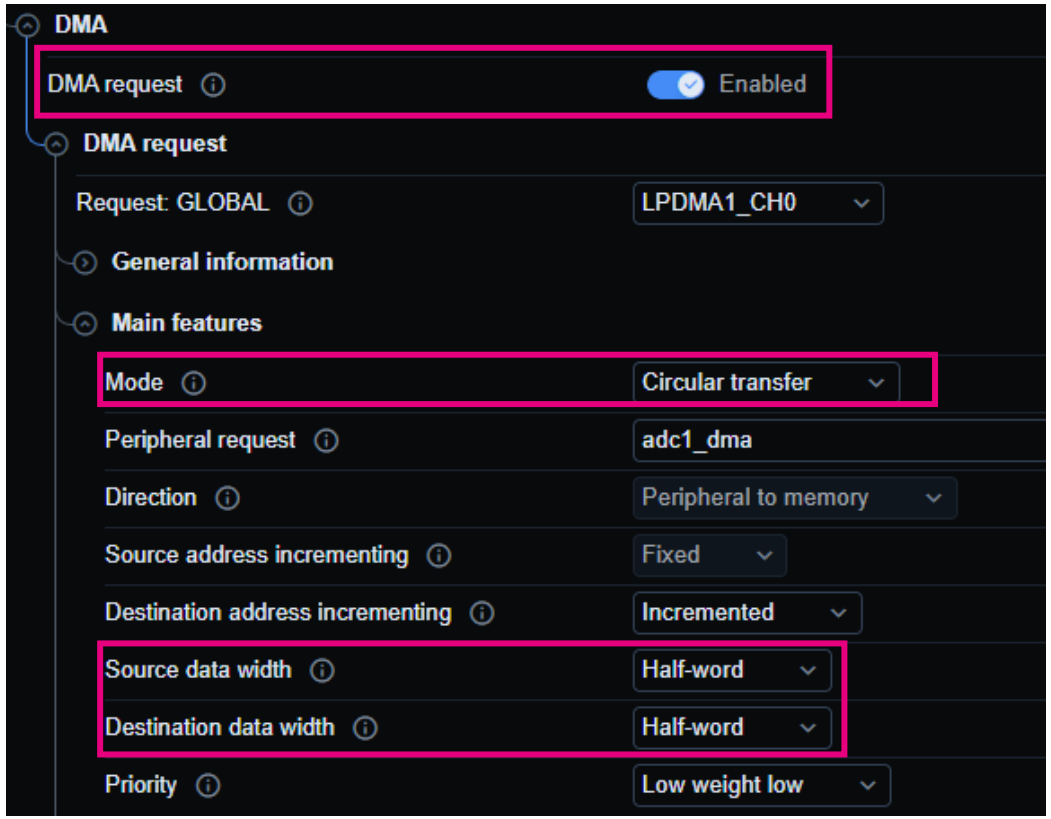
ADC Configuration

- Add **Channel0** under **ADC groups conversions > Regular sequencer**



ADC Configuration

- Configure the **DMA Request** with the following parameters:
  - DMA Request: **Enabled**
  - Mode: **Circular Transfer**
  - Source data width: **Half-word**
  - Destination data width: **Half-word**



Generating CMake Project

- Return to project settings, make sure **CMake** is selected as the format and click **Generate IDE project**





Adding Application Code

- Before **main()**
  - Add the ADC handle declaration
  - Add the ADC buffer declaration

```
18  #include "main.h"
19
20  /* Private typedef -----------------------------------------------------------*/
21  /* Private define ------------------------------------------------------------*/
22  /* Private macro -------------------------------------------------------------*/
23  /* Private variables ---------------------------------------------------------*/
24  hal_tim_handle_t* htim2;
25  hal_adc_handle_t* hadc1;
26  uint16_t adc_buffer[8];
27  /* Private functions prototype -----------------------------------------------*/
28
29  /**
30    * brief:  The application entry point.
31    * retval: none but we specify int to comply with C99 standard
32    */
33  int main(void)
34  {
```

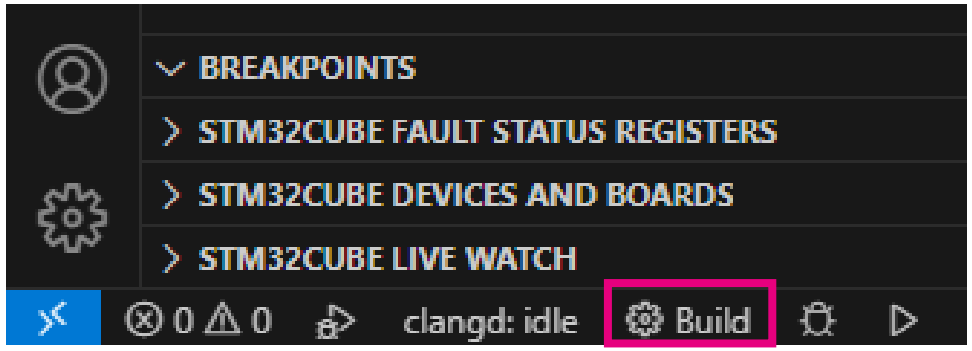
Adding Application Code

- In **main()**
  - Define the ADC handle using gethandle()
  - Start the ADC
  - Calibrate the ADC
  - Start ADC conversions

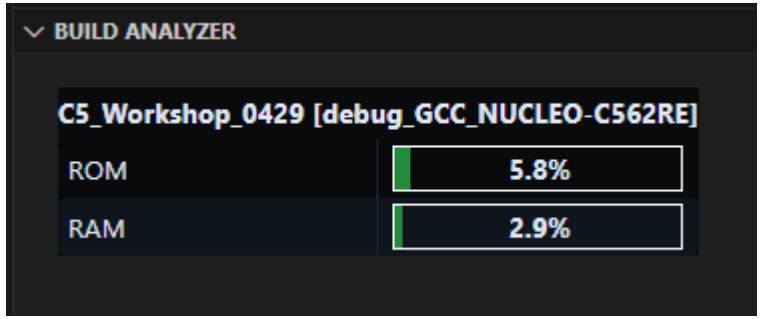
```
33  int main(void)
34  {
35    /** System Init: this code placed in targets folder initializes your system.
36      * It calls the initialization (and sets the initial configuration) of the pe
37      * You can use STM32CubeMX to generate and call this code or not in this proj
38      * It also contains the HAL initialization and the initial clock configuratio
39      */
40    if (mx_system_init() != SYSTEM_OK)
41    {
42      return (-1);
43    }
44    else
45    {
46      /*
47        * You can start your application code here
48        */
49      htim2 = mx_tim2_gethandle();
50      hadc1 = mx_adc1_gethandle();
51      HAL_TIM_OC_StartChannel(htim2, HAL_TIM_CHANNEL_1);
52      HAL_TIM_Start(htim2);
53
54      HAL_ADC_Start(hadc1);
55      HAL_ADC_Calibrate(hadc1);
56      HAL_ADC_REG_StartConv_DMA(hadc1, (uint8_t*)adc_buffer, 8*sizeof(uint16_t));
57
58      while (1) {}
59    }
60  } /* end main */
61
```

Building your project

- Select **Build** in the bottom left-hand corner of the screen

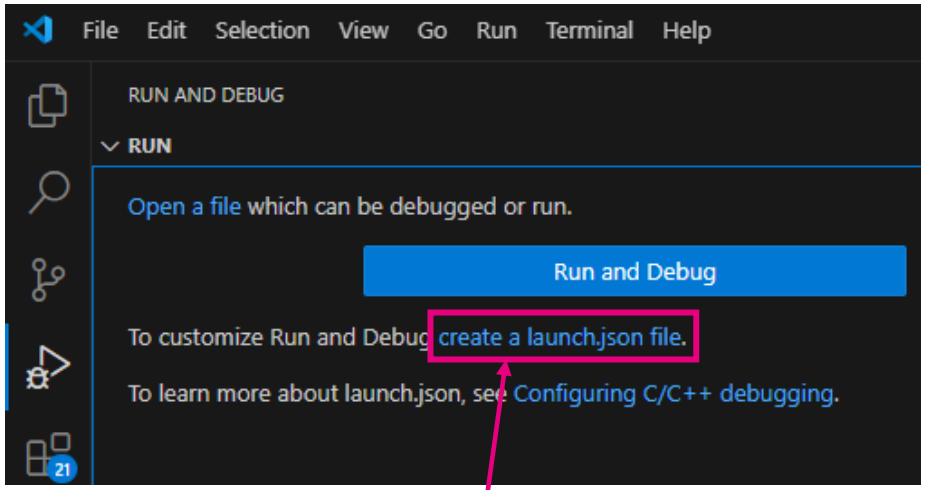


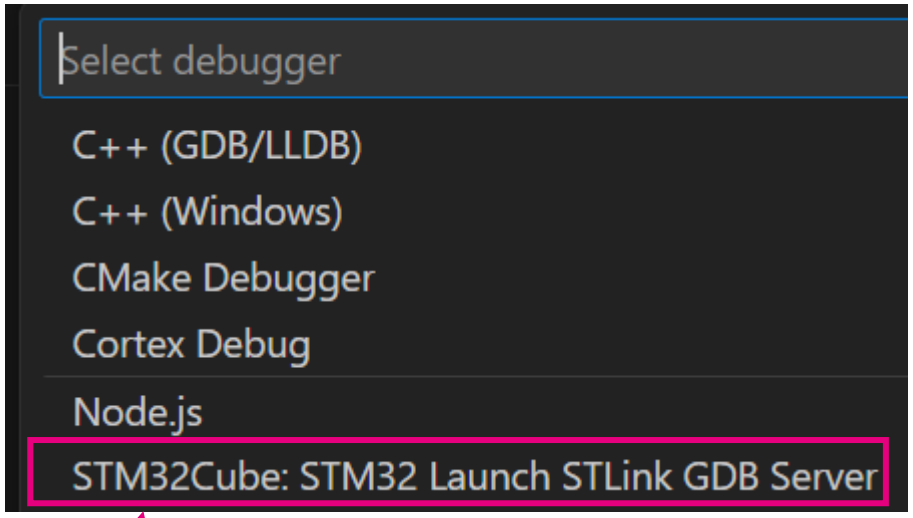
- Once you build you will see the Build Analyzer display ROM and RAM usage

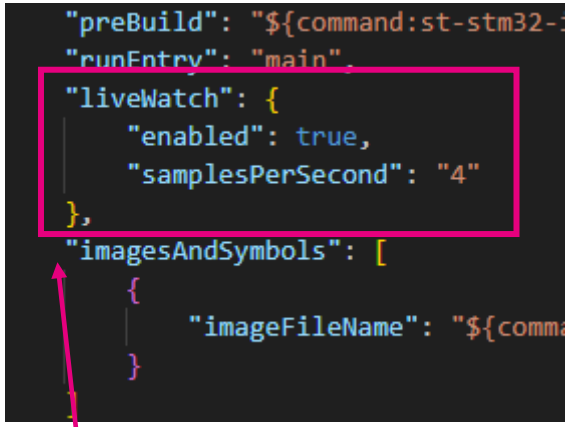


Live Watch

- If you want to use Live Watch, it must be enabled in your launch.json file







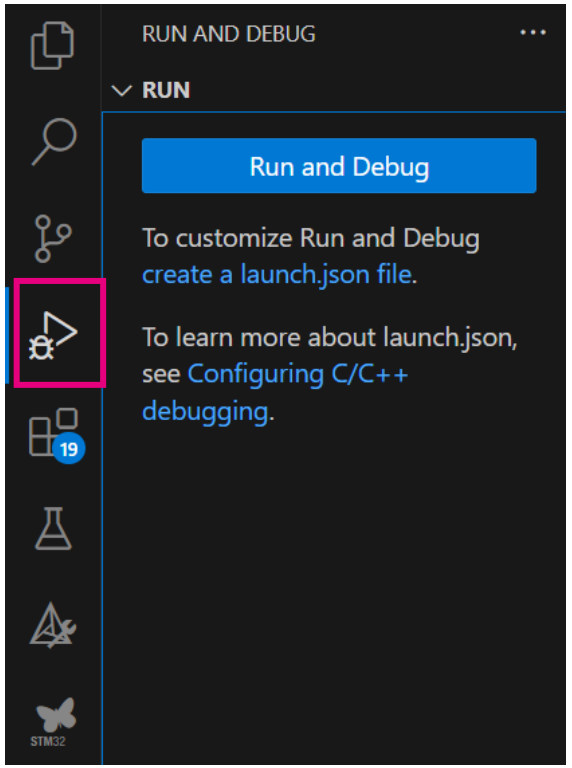
1

2

3

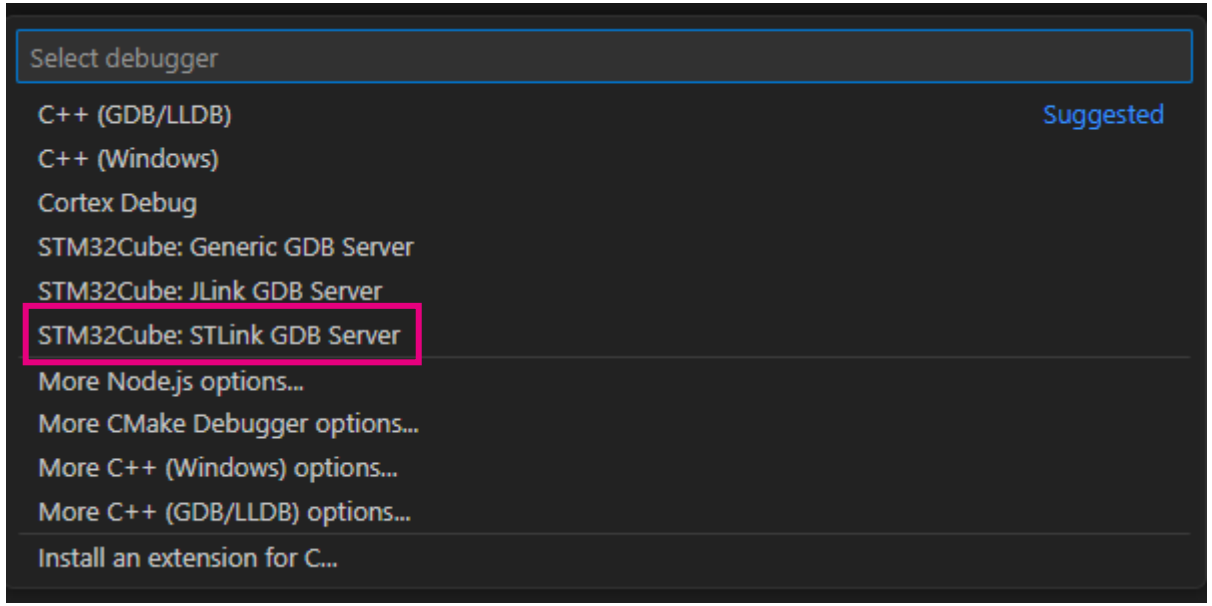
Running and debugging your project

- To run and debug your project select **Run and Debug** on the left-side toolbar and click the blue button



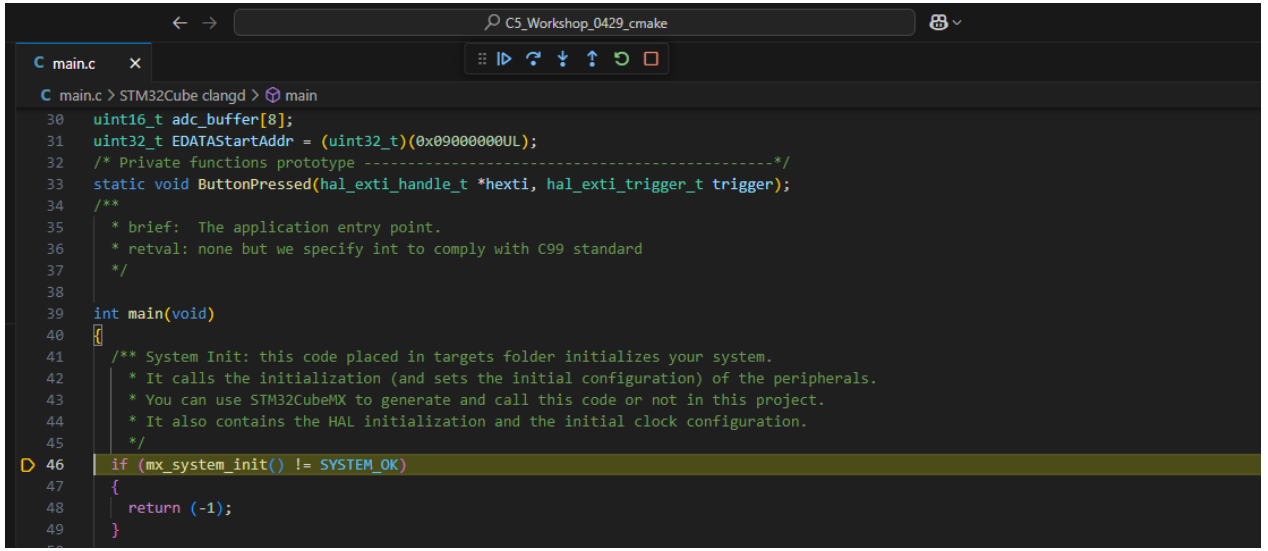
Running and debugging your project

- Select **STM32Cube: STLink GDB Server** as the debugger



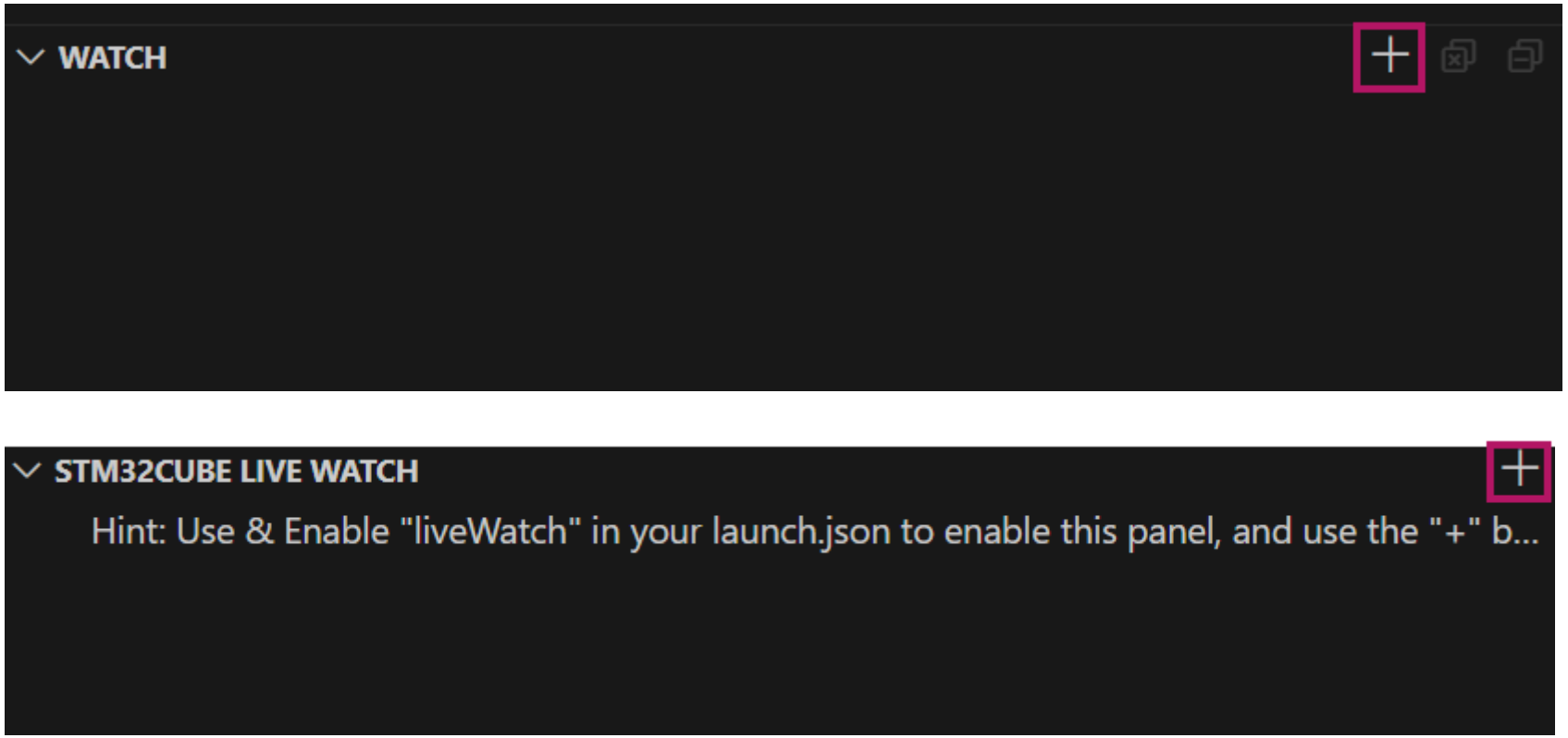
Running and debugging your project

- Your project will stop at the initial breakpoint
- Hit **Continue** to resume execution



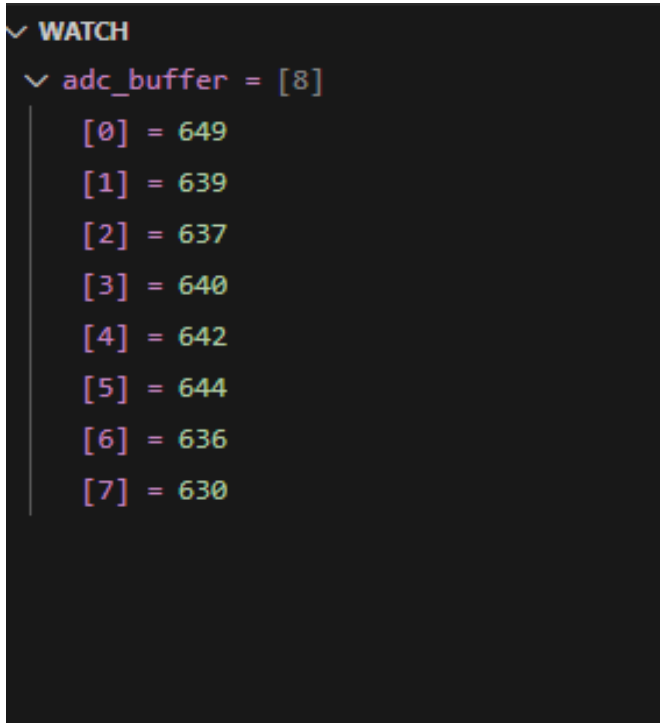
Running and Debugging

- You can add variables to the Watch and Live Watch windows



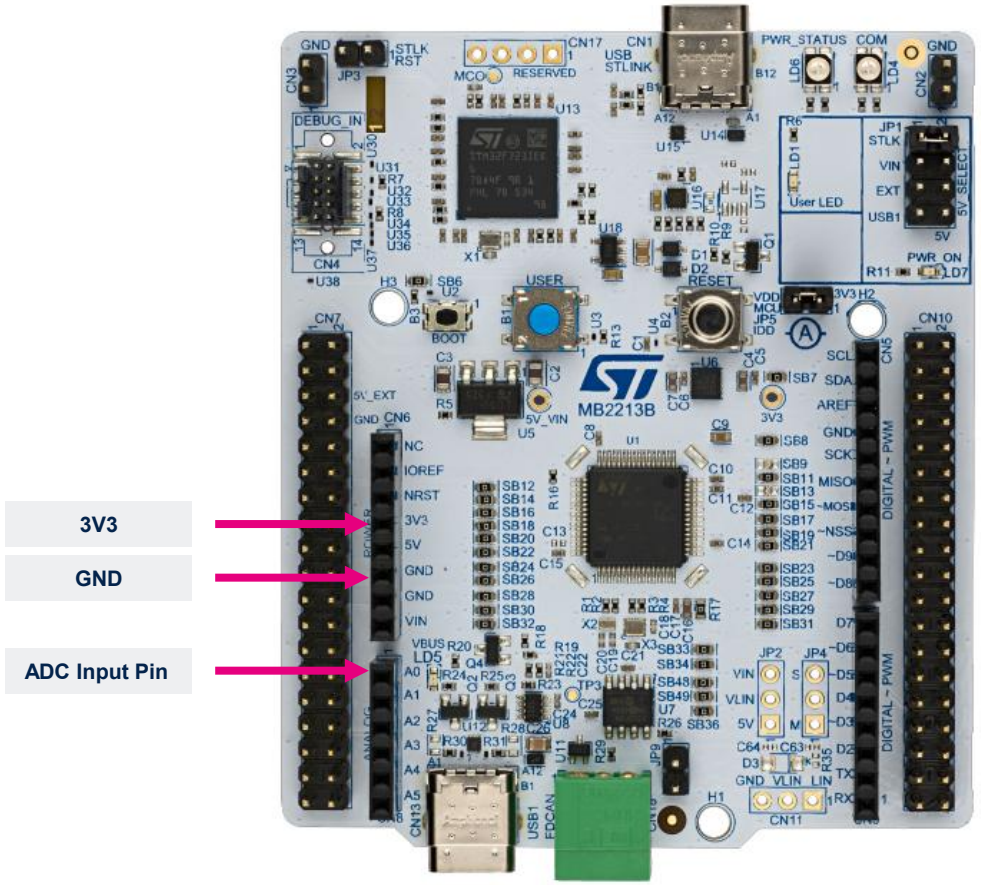
Debug Features

- Add **adc_buffer** to the watch window to see the stored values



Changing ADC Value

- The ADC value can be changed by connecting the ADC input pin to 3V3 or GND with a jumper cable

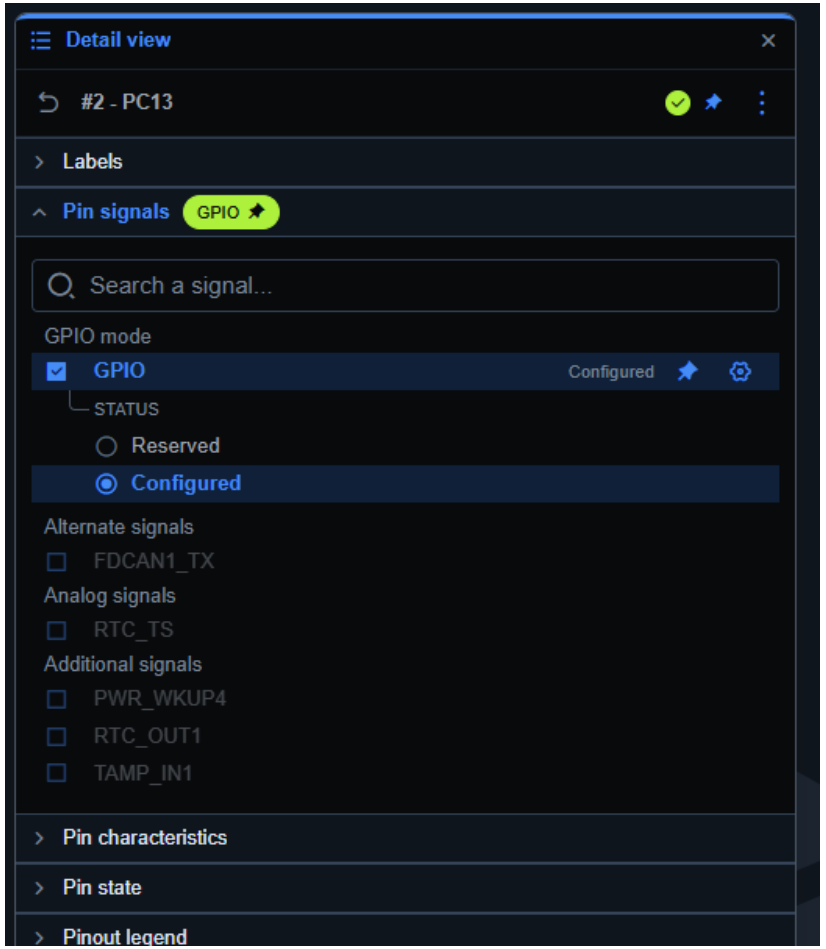


Lab 3: EXTI and USART

STM32CubeMX2 Button Configuration

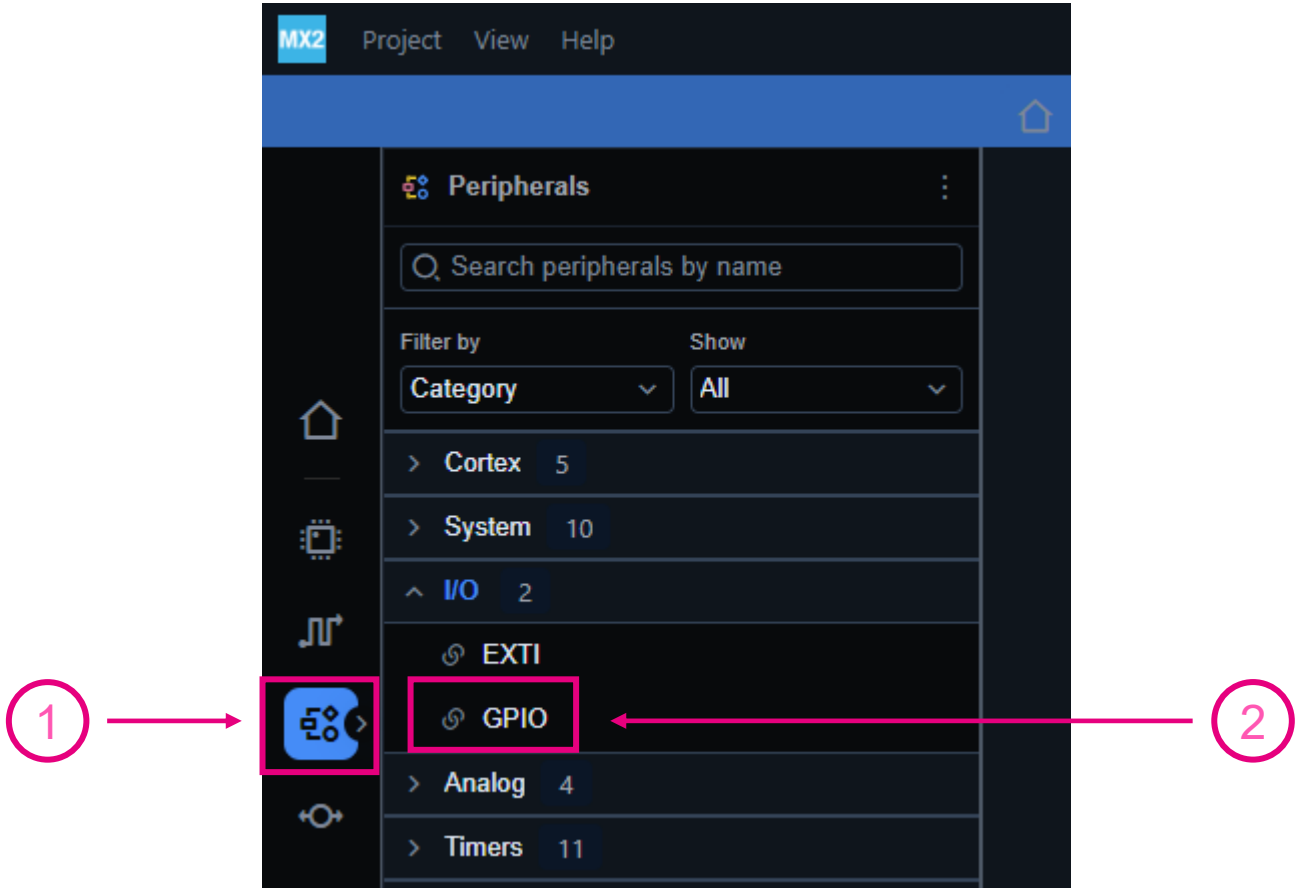
- Right-click on **PC13** and select **GPIO** under GPIO mode





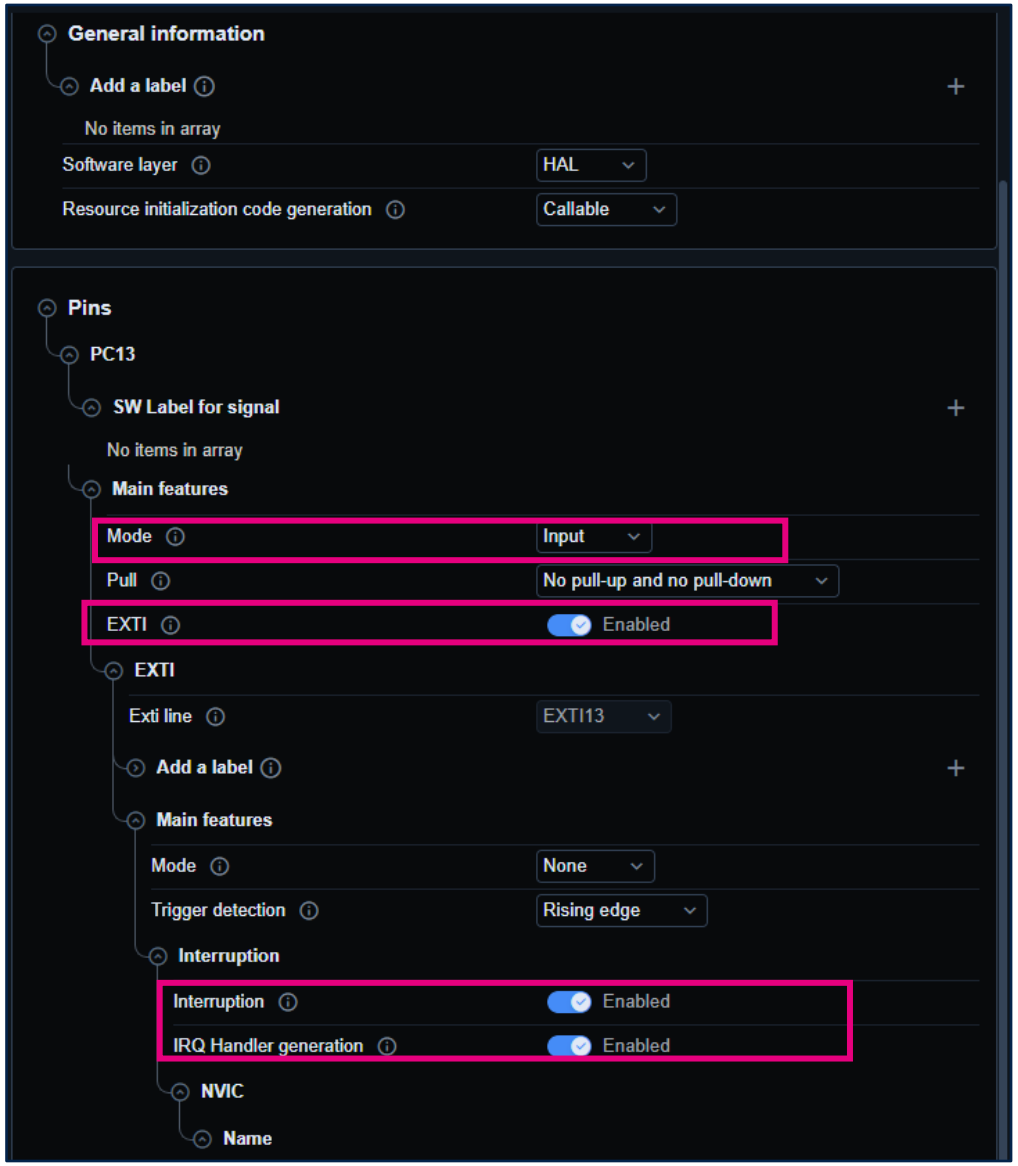
Button Configuration

- Select **GPIO** under **Peripherals > I/O** to configure **PC13**



Button Configuration

- Configure **PC13** with the below settings
  - Mode: **Input**
  - EXTI: **Enabled**
  - EXTI > Interruption: **Enabled**
  - EXTI > IRQ Handler generated: **Enabled**



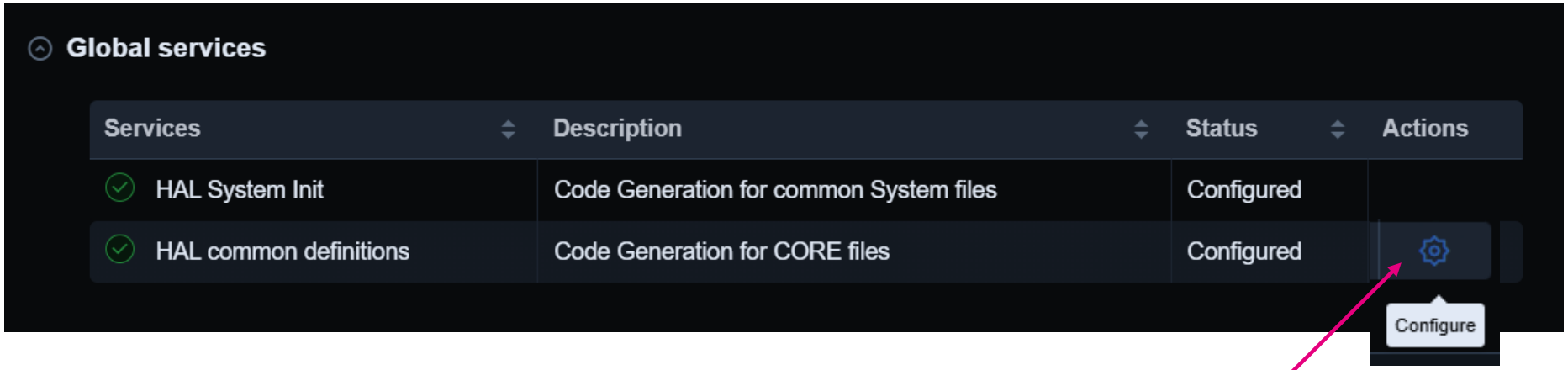
Project Settings Configuration

Configure the project settings to enable the correct HAL driver generation

- On the left toolbar, navigate **to Project Settings > HAL common definitions**



1



2

Hover your cursor in Actions area and then click on gear

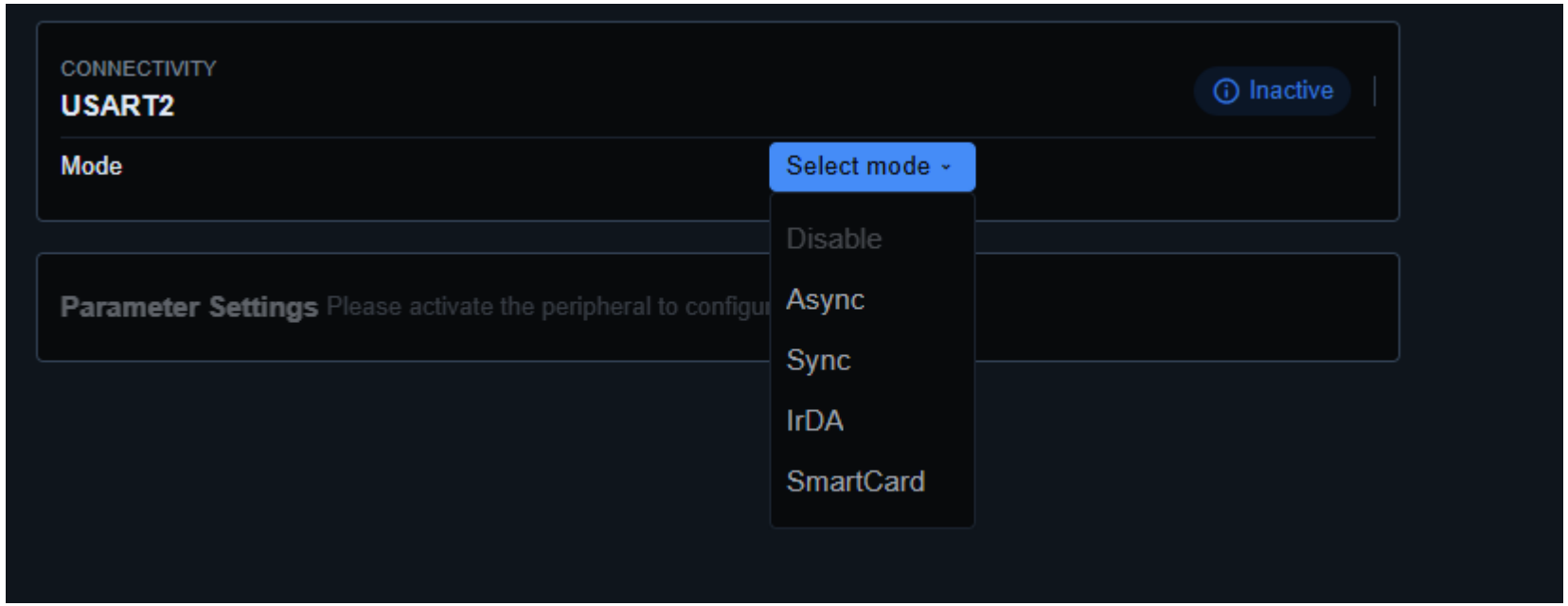
Project Settings Configuration

- Ensure **Use register callback** is enabled under **HAL EXTI**



STM32CubeMX2 USART Configuration

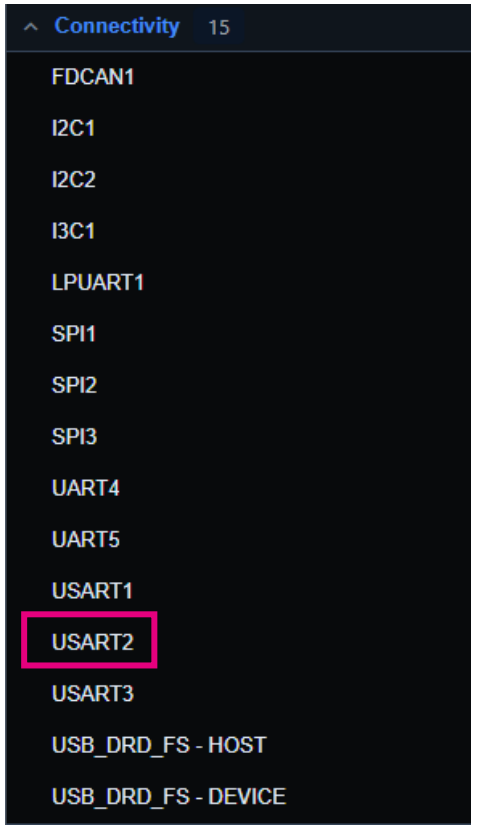
- Configure USART2 with the default baud rate of 115200 on pins PA2 and PA3



USART Configuration

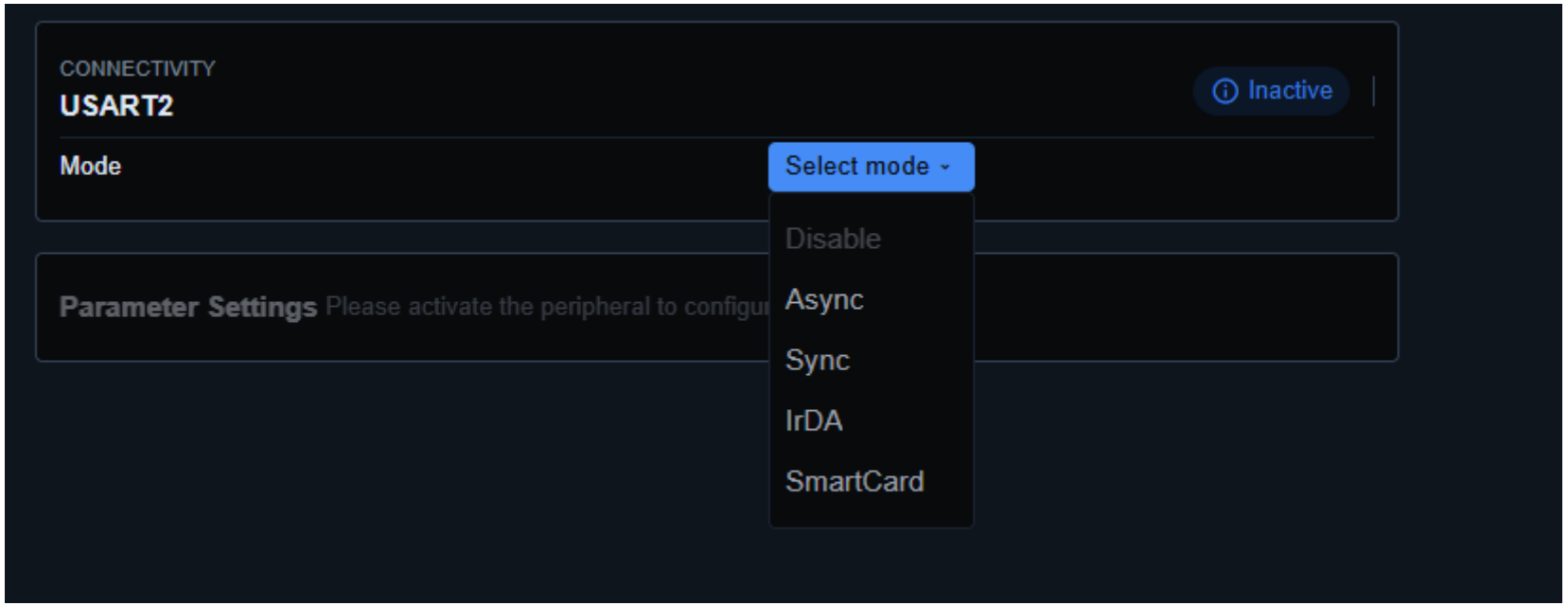
- Select **USART2** under **Peripherals > Connectivity** to configure the USART





USART Configuration

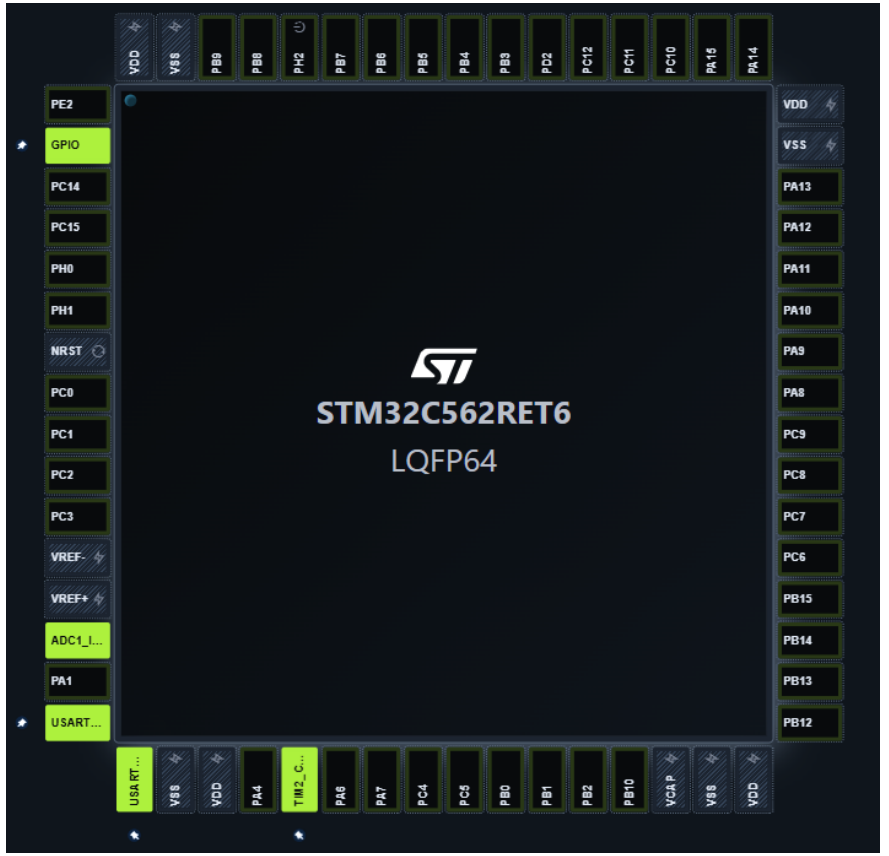
- Select **Async Mode.**
  Accept default settings 115200 baud, 8bit, no parity,1 stop bit.



USART Configuration

- Move UART pins to PA2 & PA3.
  While holding ctrl key down select USART pins then slide to new pin locations.





Generating CMake Project

- Return to project settings, make sure **CMake** is selected as the format and click **Generate IDE project**





Adding Application Code

- Before **main()**
  - Add #include <stdio.h>
  - Add the UART and EXTI handle declarations
  - Add the function prototype for the EXTI callback function

```
17  /* Includes ------------------------------------------------------------------*/
18  #include "main.h"
19  #include <stdio.h>
20  /* Private typedef -----------------------------------------------------------*/
21  /* Private define ------------------------------------------------------------*/
22  /* Private macro -------------------------------------------------------------*/
23  /* Private variables ---------------------------------------------------------*/
24  hal_tim_handle_t* htim2;
25  hal_adc_handle_t* hadc1;
26  hal_uart_handle_t* huart2;
27  hal_exti_handle_t* hexti13;
28  uint16_t adc_buffer[8];
29  /* Private functions prototype -----------------------------------------------*/
30  static void ButtonPressed(hal_exti_handle_t *hexti, hal_exti_trigger_t trigger);
31
```

Adding Application Code

- In **main()**
  - Define the handles for UART and EXTI
  - Register the EXTI Callback
  - Enable the EXTI

```
int main(void)
{
  /** System Init: this code placed in targets folder initializes your system.
    * It calls the initialization (and sets the initial configuration) of the peri
    * You can use STM32CubeMX to generate and call this code or not in this projec
    * It also contains the HAL initialization and the initial clock configuration.
    */
  if (mx_system_init() != SYSTEM_OK)
  {
    return (-1);
  }
  else
  {
    /*
      * You can start your application code here
      */
    htim2 = mx_tim2_gethandle();
    hadc1 = mx_adc1_gethandle();
    hexti13 = mx_gpio_default_exti13_gethandle();
    huart2 = mx_usart2_uart_gethandle();

    HAL_TIM_OC_StartChannel(htim2, HAL_TIM_CHANNEL_1);
    HAL_TIM_Start(htim2);

    HAL_ADC_Start(hadc1);
    HAL_ADC_Calibrate(hadc1);
    HAL_ADC_REG_StartConv_DMA(hadc1, (uint8_t*)adc_buffer, 8*sizeof(uint16_t));

    HAL_EXTI_RegisterTriggerCallback(hexti13, ButtonPressed);
    HAL_EXTI_Enable(hexti13, HAL_EXTI_MODE_INTERRUPT);
```

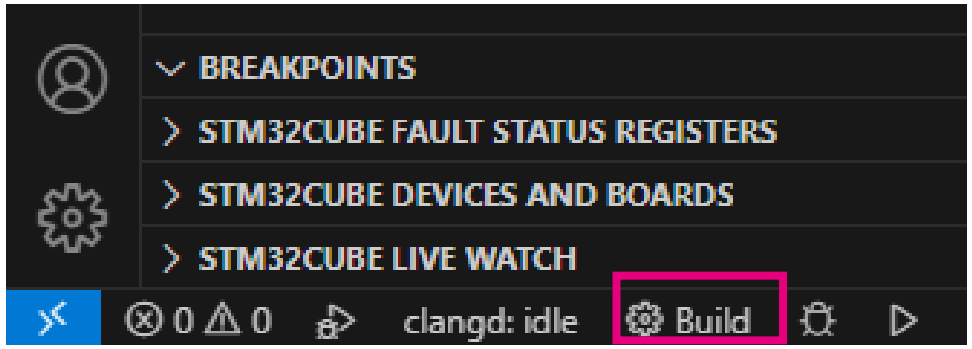
Adding Application Code

- Retarget printf() to UART2 by defining the **__io_putchar** function
- Create the EXTI callback **ButtonPressed** such that a message is printed when the callback is reached

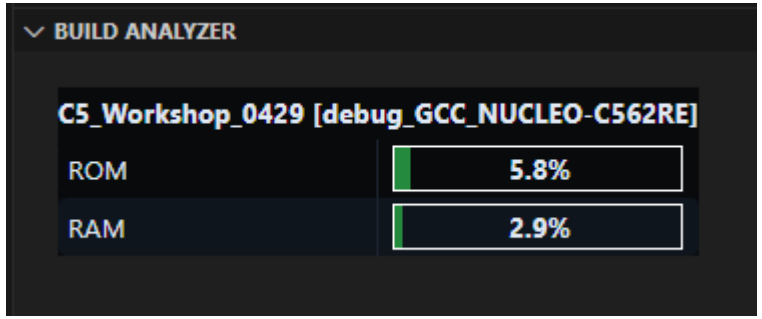
```
74  int __io_putchar(int ch){
75    HAL_UART_Transmit(huart2, (uint8_t*)&ch, 1, 0xFFFF);
76    return 0;
77  }
78
79  static void ButtonPressed(hal_exti_handle_t *hexti, hal_exti_trigger_t trigger){
80    printf("Button has been pressed\n");
81  }
```

Building your project

- Select **Build** in the bottom left-hand corner of the screen

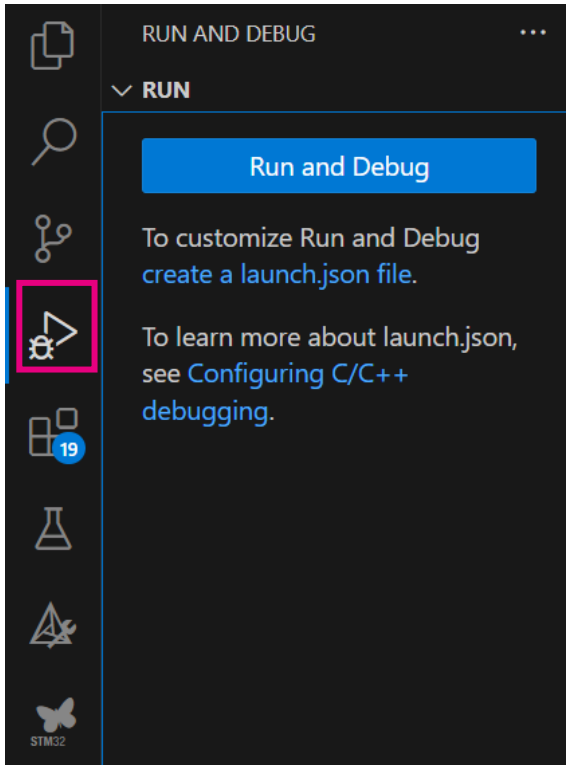


- Once you build you will see the Build Analyzer display ROM and RAM usage



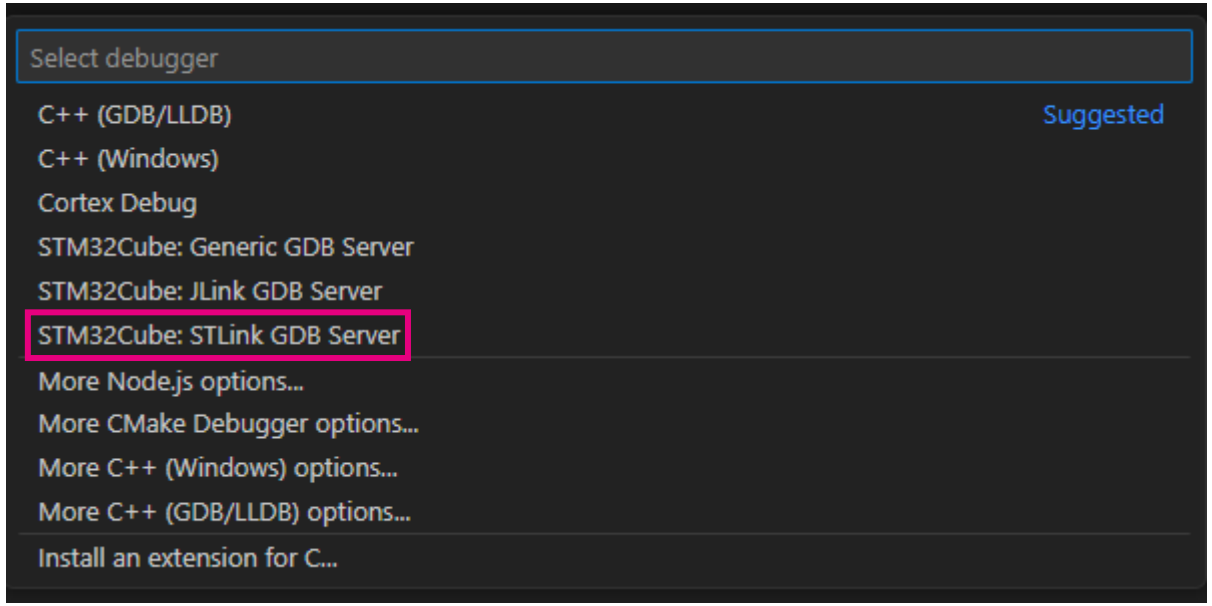
Running and debugging your project

- To run and debug your project select **Run and Debug** on the left-side toolbar and click the blue button



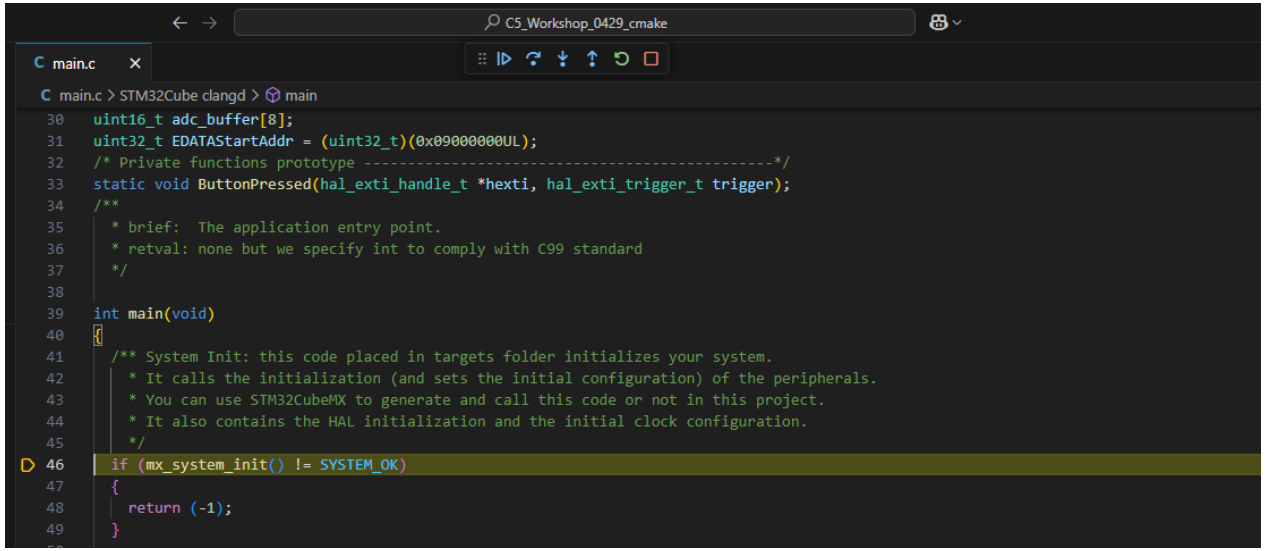
Running and debugging your project

- Select **STM32Cube: STLink GDB Server** as the debugger



Running and debugging your project

- Your project will stop at the initial breakpoint
- Hit **Continue** to resume execution

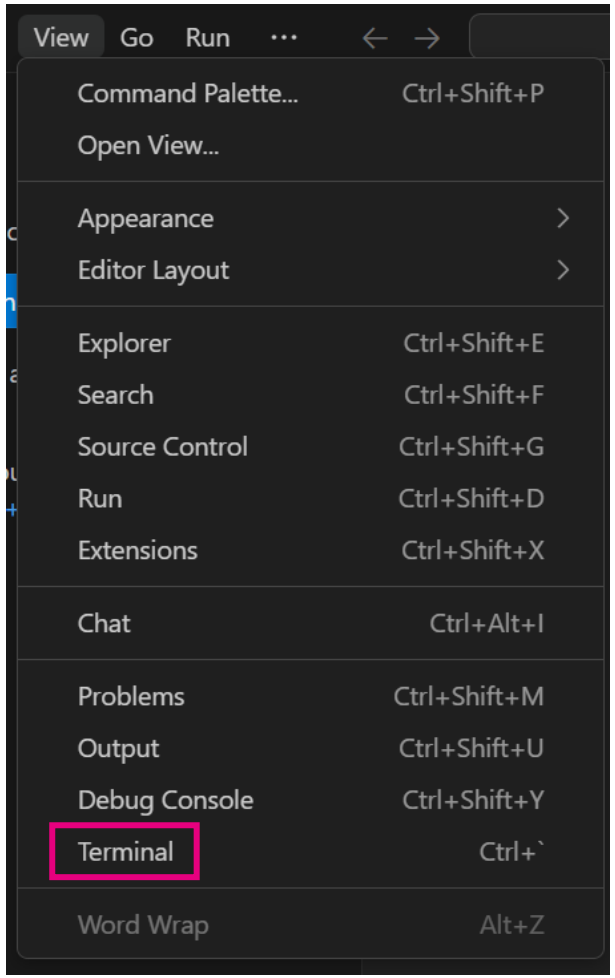


Output Message

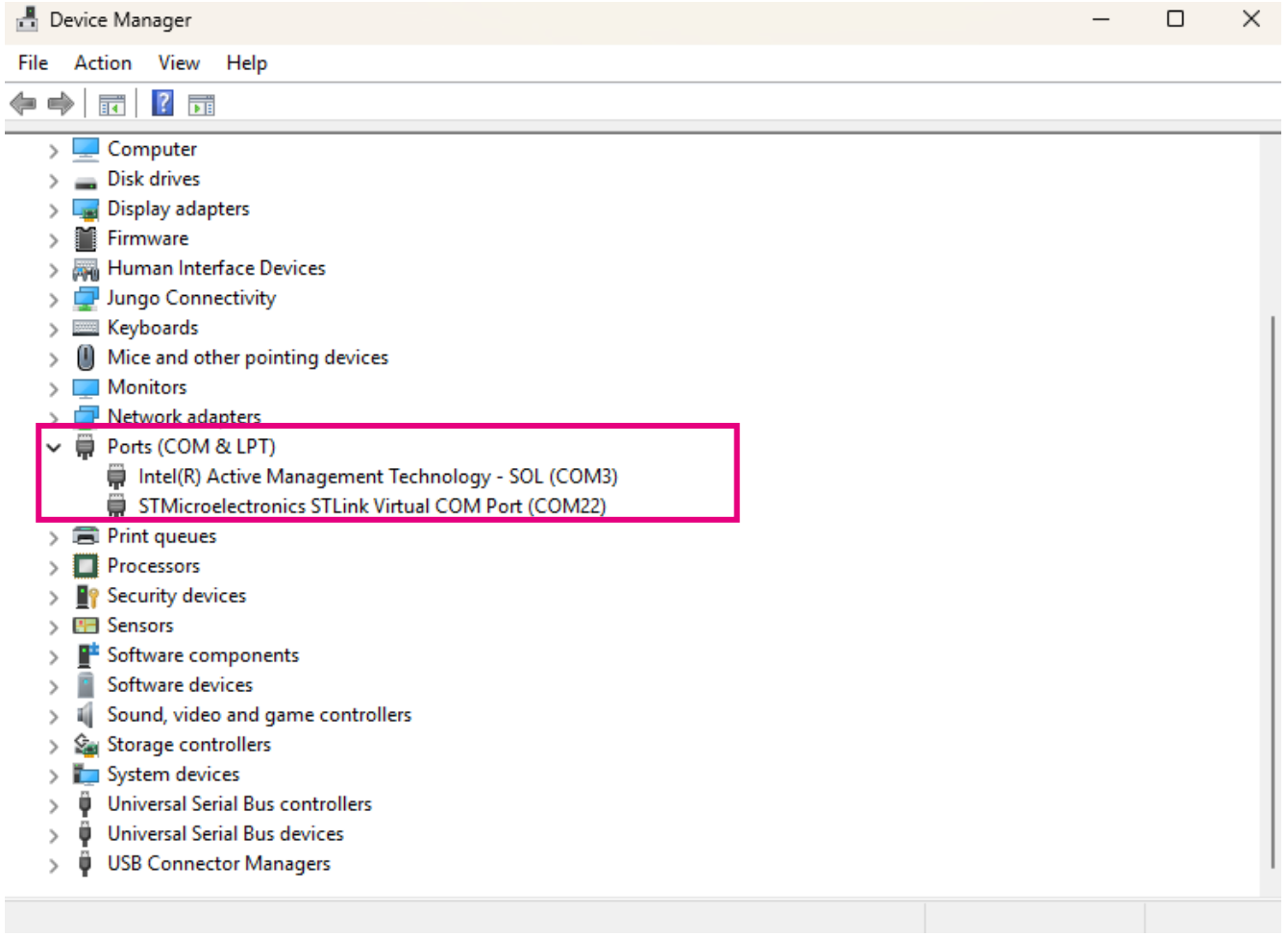
- Add these lines to the Terminal to open the COM Port or use a third-party extension or application (e.g. PuTTY)
  - $port= new-Object System.IO.Ports.SerialPort COM22,115200,None,8,one
  - $port.open()
  - while($true) { $port.ReadLine() }
- You can determine the correct COM port from your Device Manager

Output Message

- You can open the Terminal in VSCode from **View > Terminal**



Output Message



Output Message

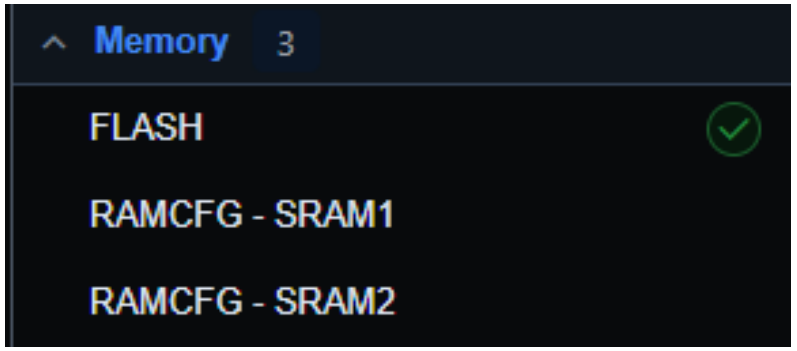
- Press the button to trigger the external interrupt
- Check for the output message in the Terminal

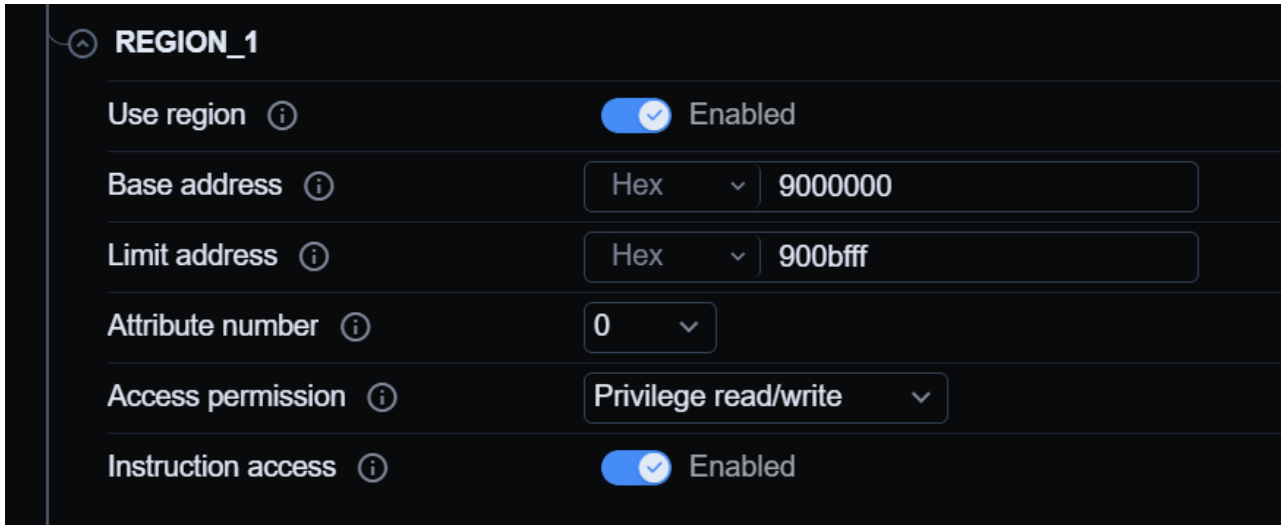


Lab 4: Emulated EEPROM

STM32CubeMX2 Emulated EEPROM Configuration

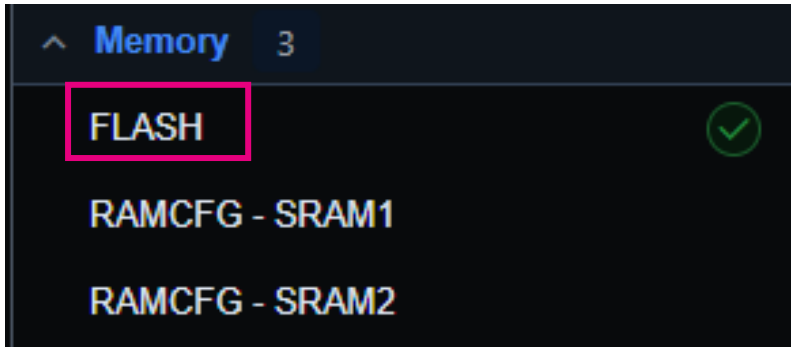
- Configure the FLASH peripheral and initialize the MPU so that data can be written to the Emulated EEPROM

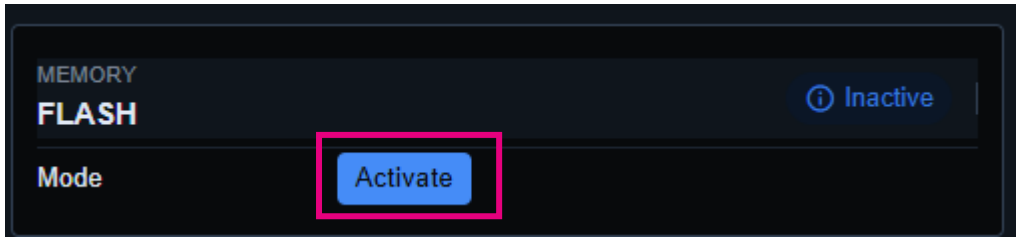


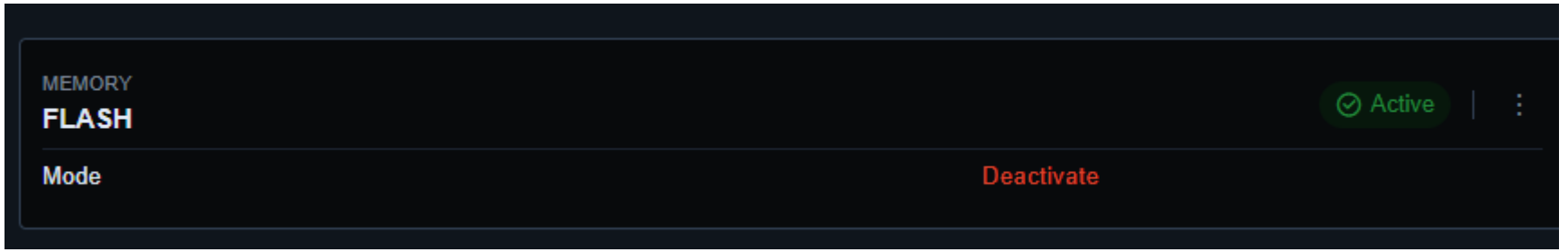


Emulated EEPROM Configuration

- Select **FLASH** under **Peripherals > Memory** to configure the FLASH

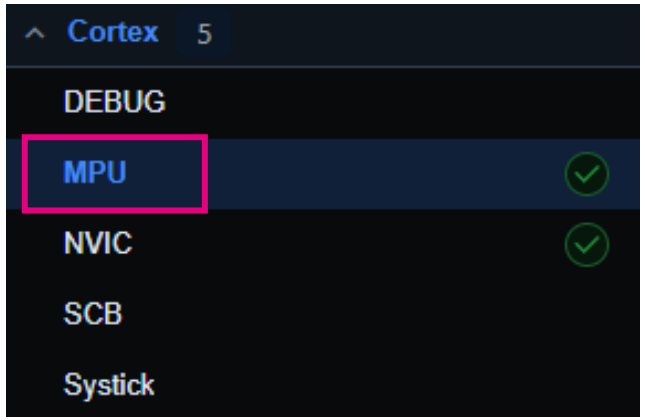






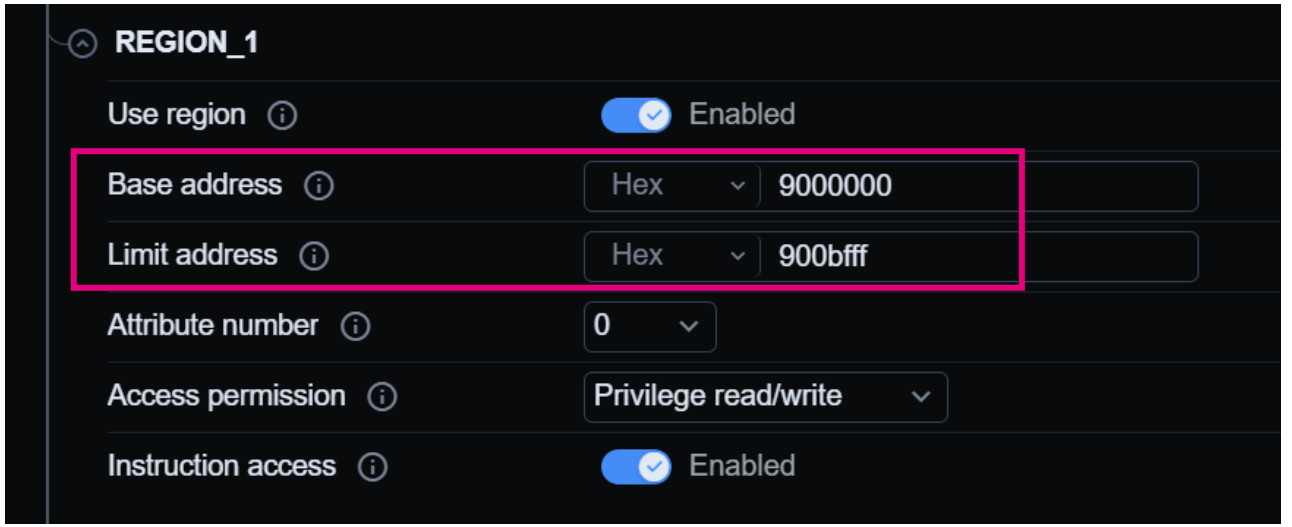
Emulated EEPROM Configuration

- Select **MPU** under **Peripherals > Cortex**



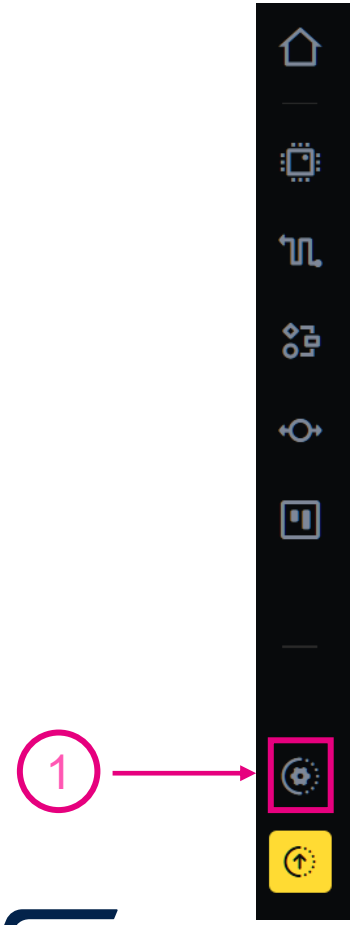
Emulated EEPROM Configuration

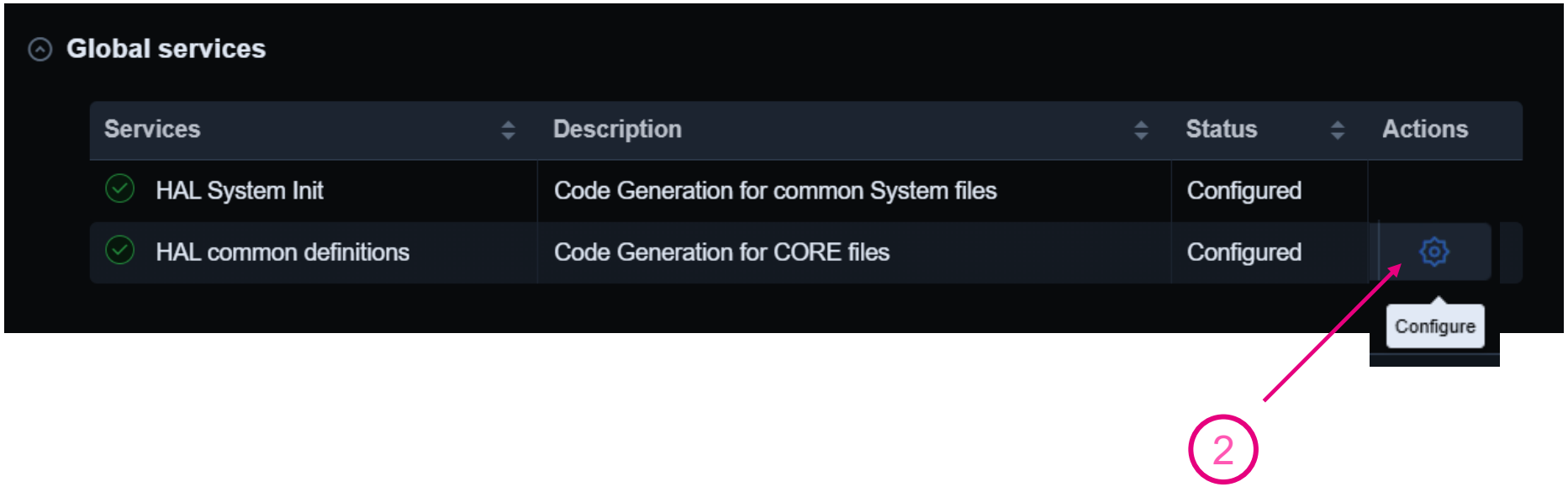
- Configure **REGION_1** with the following parameters:
  - Base Address: **0x9000000**
  - Limit Address: **0x900bfff**
  - Ensure that Instruction access is **Enabled**



Project Settings Configuration

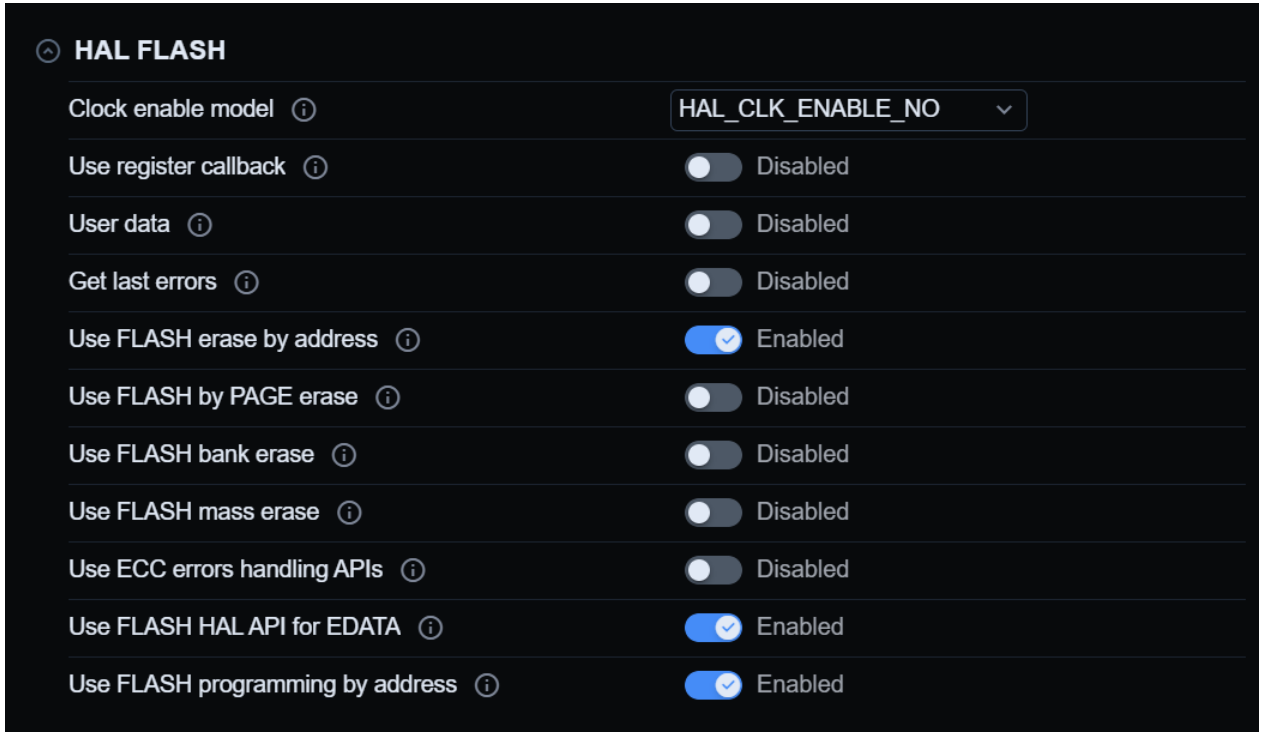
- On the left toolbar, navigate **to Project Settings > HAL common definitions**





Project Settings Configuration

- Enable the following under **HAL FLASH**
  - **Use FLASH erase by address**
  - **Use FLASH HAL API for EDATA**
  - **Use FLASH programming by address**



Generating CMake Project

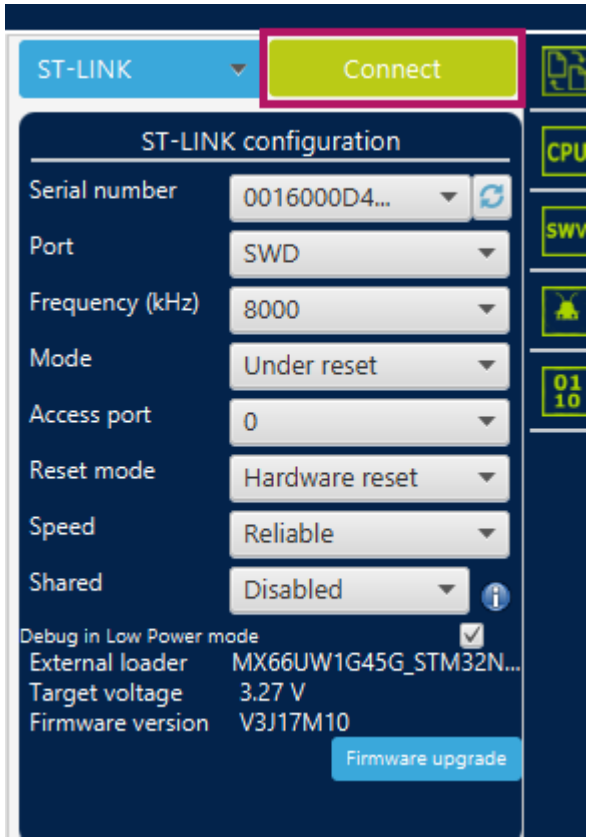
- Return to project settings, make sure **CMake** is selected as the format and click **Generate IDE project**

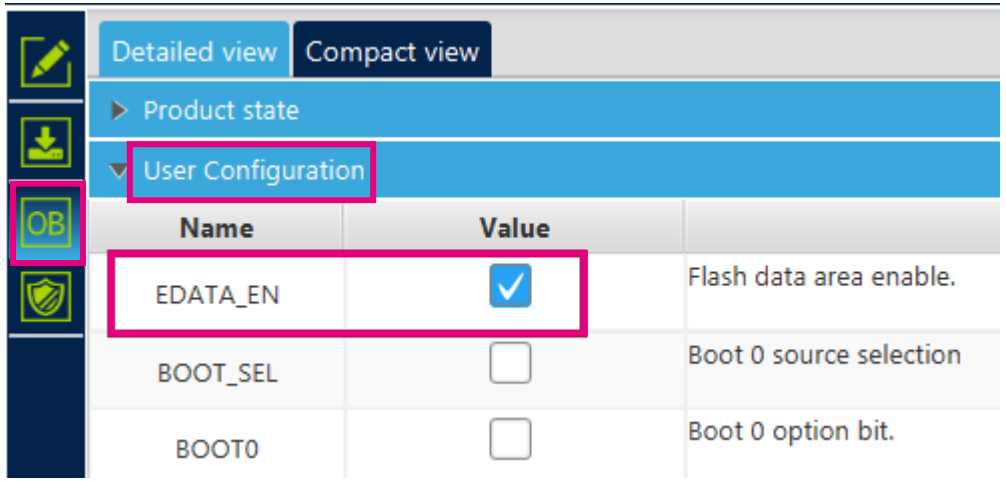




STM32CubeProgrammer

- Make sure that the EDATA_EN option byte is checked in STM32CubeProgrammer





NOTE: make sure to disconnect from STM32CubeProgrammer before trying to run and debug in VSCode

Adding Application Code

- Before **main()**
  - Add the declaration for the FLASH handle
  - Define the EDATA Start Address as 0x09000000

```
hal_tim_handle_t* htim2;
hal_adc_handle_t* hadc1;
hal_uart_handle_t* huart2;
hal_exti_handle_t* hexti13;
hal_flash_handle_t* hflash;
uint16_t adc_buffer[8];
uint32_t EDATAStartAddr = (uint32_t)(0x09000000UL);
```

Adding Application Code

- In **main()**
  - Define hflash by using the gethandle() function

```
htim2 = mx_tim2_gethandle();
hadc1 = mx_adc1_gethandle();
hexti13 = mx_gpio_default_exti13_gethandle();
huart2 = mx_usart2_uart_gethandle();
hflash = mx_flash_gethandle();
```

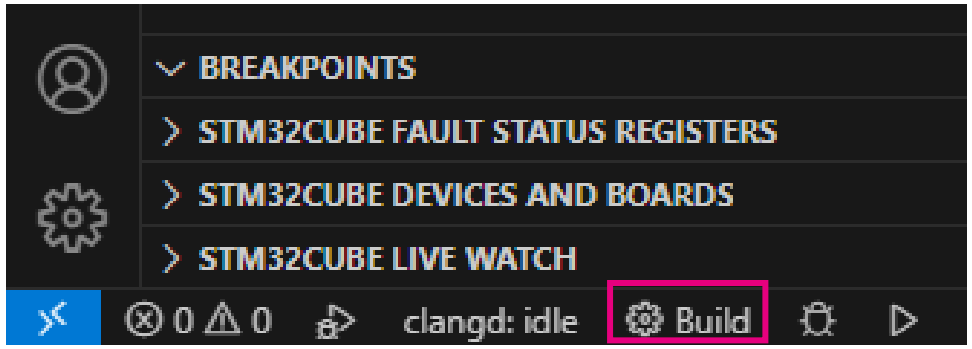
Adding Application Code

- Redefine the ButtonPressed() function so that the ADC values are stored in the Emulated EEPROM

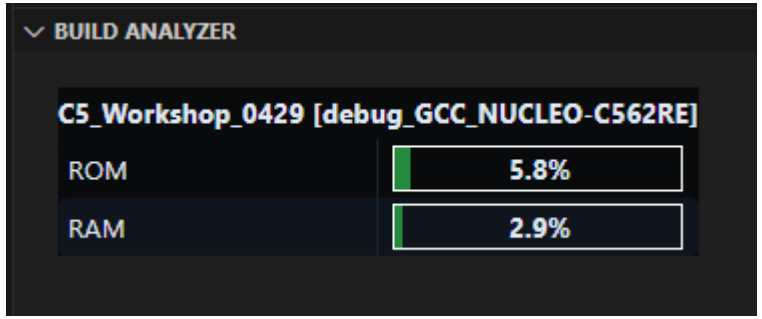
```
79  static void ButtonPressed(hal_exti_handle_t *hexti, hal_exti_trigger_t trigger){
80    HAL_FLASH_ITF_Unlock(HAL_FLASH);
81    HAL_FLASH_EDATA_EraseByAddr(hflash, EDATAStartAddr, FLASH_EDATA_PAGE_SIZE, 0xFFFF);
82    HAL_FLASH_EDATA_ProgramByAddr(hflash, EDATAStartAddr, (uint32_t*)adc_buffer, 8*sizeof(uint16_t), 0xFFFF);
83    printf("ADC Data Saved in Emulated EEPROM\n");
84  }
```

Building your project

- Select **Build** in the bottom left-hand corner of the screen

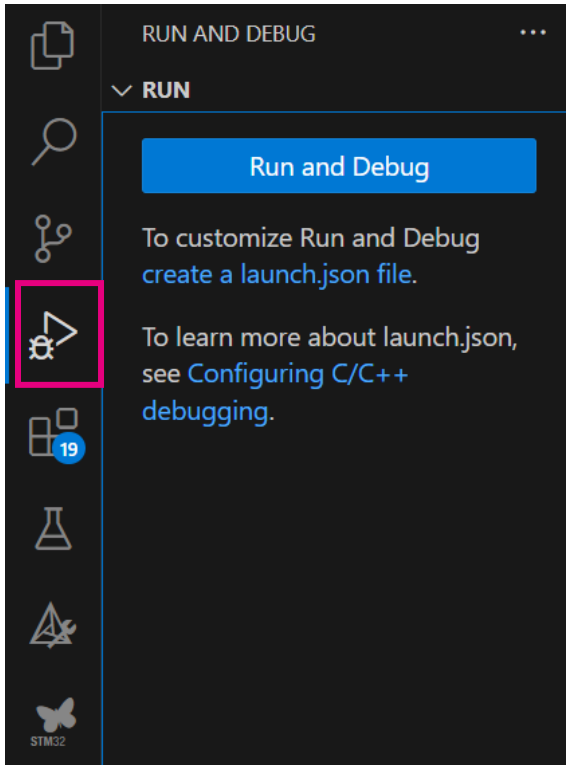


- Once you build you will see the Build Analyzer display ROM and RAM usage



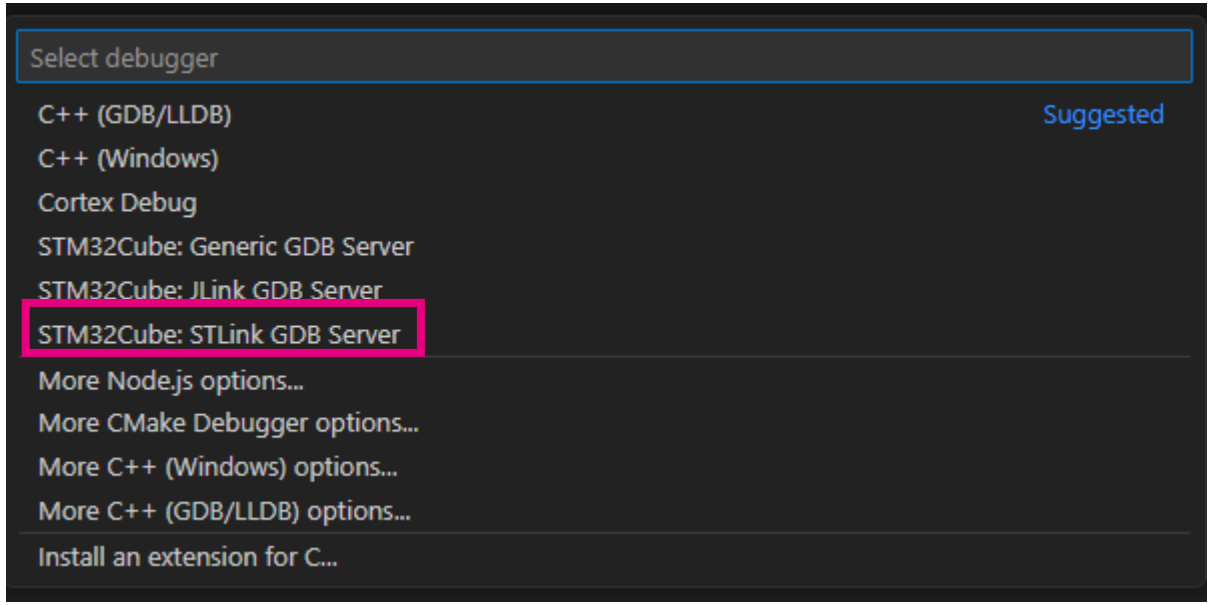
Running and debugging your project

- To run and debug your project select **Run and Debug** on the left-side toolbar and click the blue button



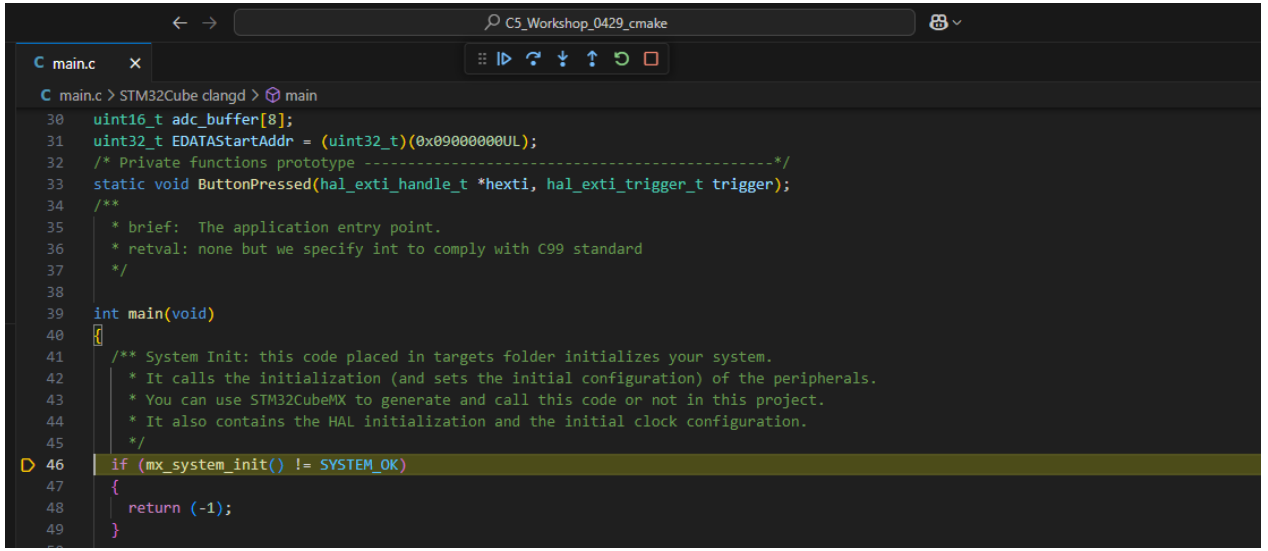
Running and debugging your project

- Select **STM32Cube: STLink GDB Server** as the debugger



Running and debugging your project

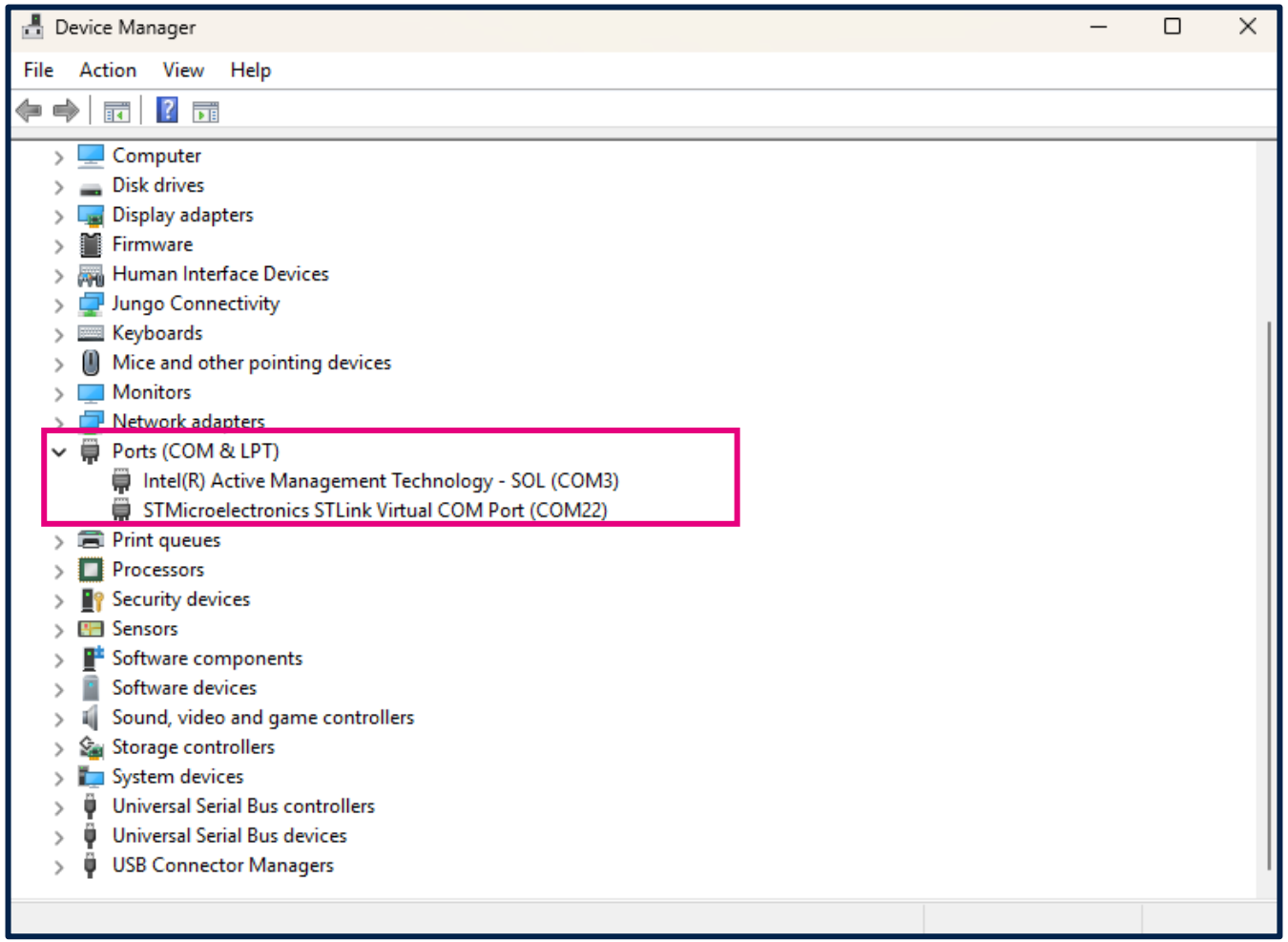
- Your project will stop at the initial breakpoint
- Hit **Continue** to resume execution



Output Message

- Add these lines to the Terminal to open the COM Port
  - $port= new-Object System.IO.Ports.SerialPort COM22,115200,None,8,one
  - $port.open()
  - while($true) { $port.ReadLine() }
- You can determine the correct COM port from your Device Manager

Output Message



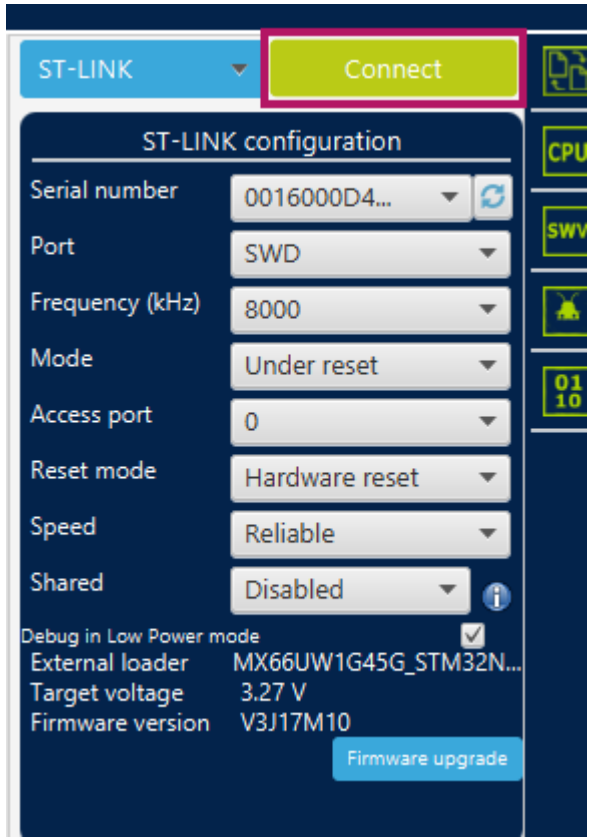
Output Message

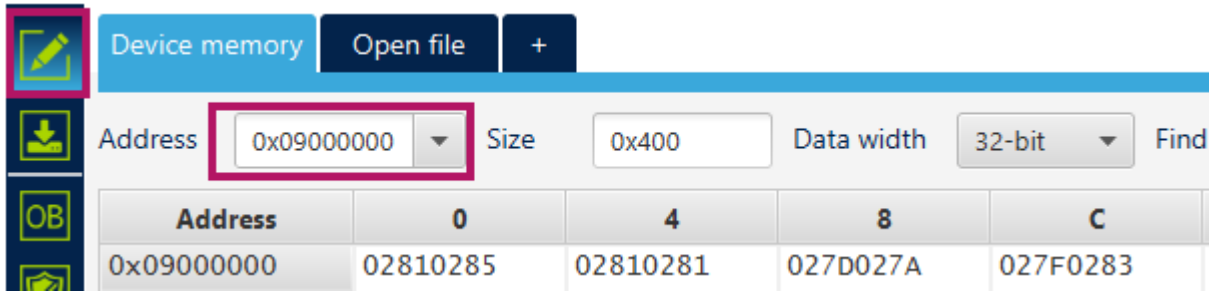
- Press the button when you are ready to save the values in the Emulated EEPROM
- Check for the output message in the Terminal



STM32CubeProgrammer

- The data stored in the emulated EEPROM can be seen from STM32CubeProgrammer

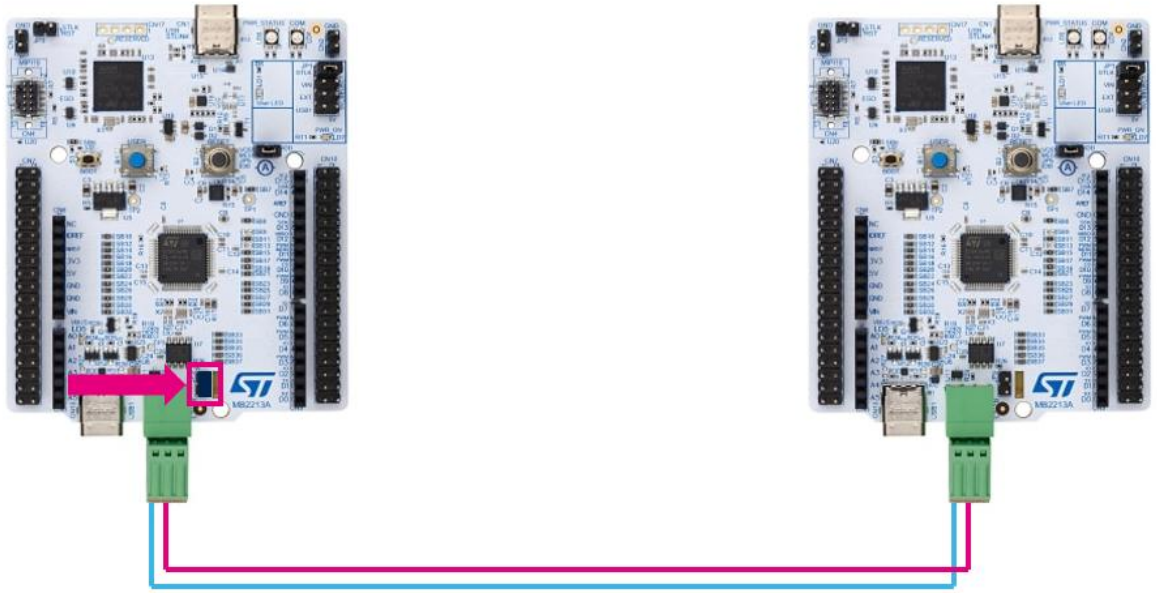




Optional Lab: FDCAN

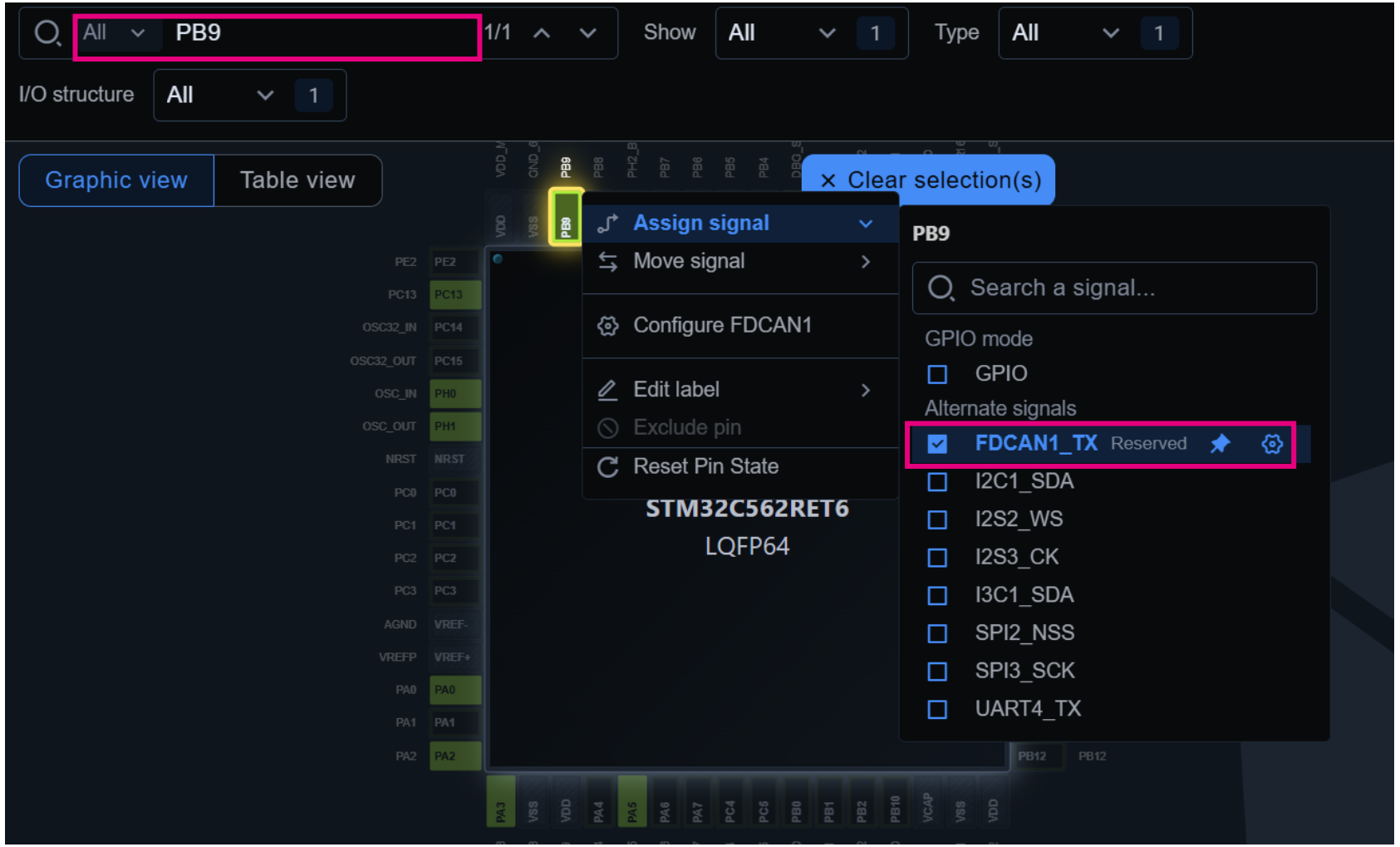
STM32CubeMX2 FDCAN Configuration

- Connect two STM32C562RE boards together to transmit and receive FDCAN messages



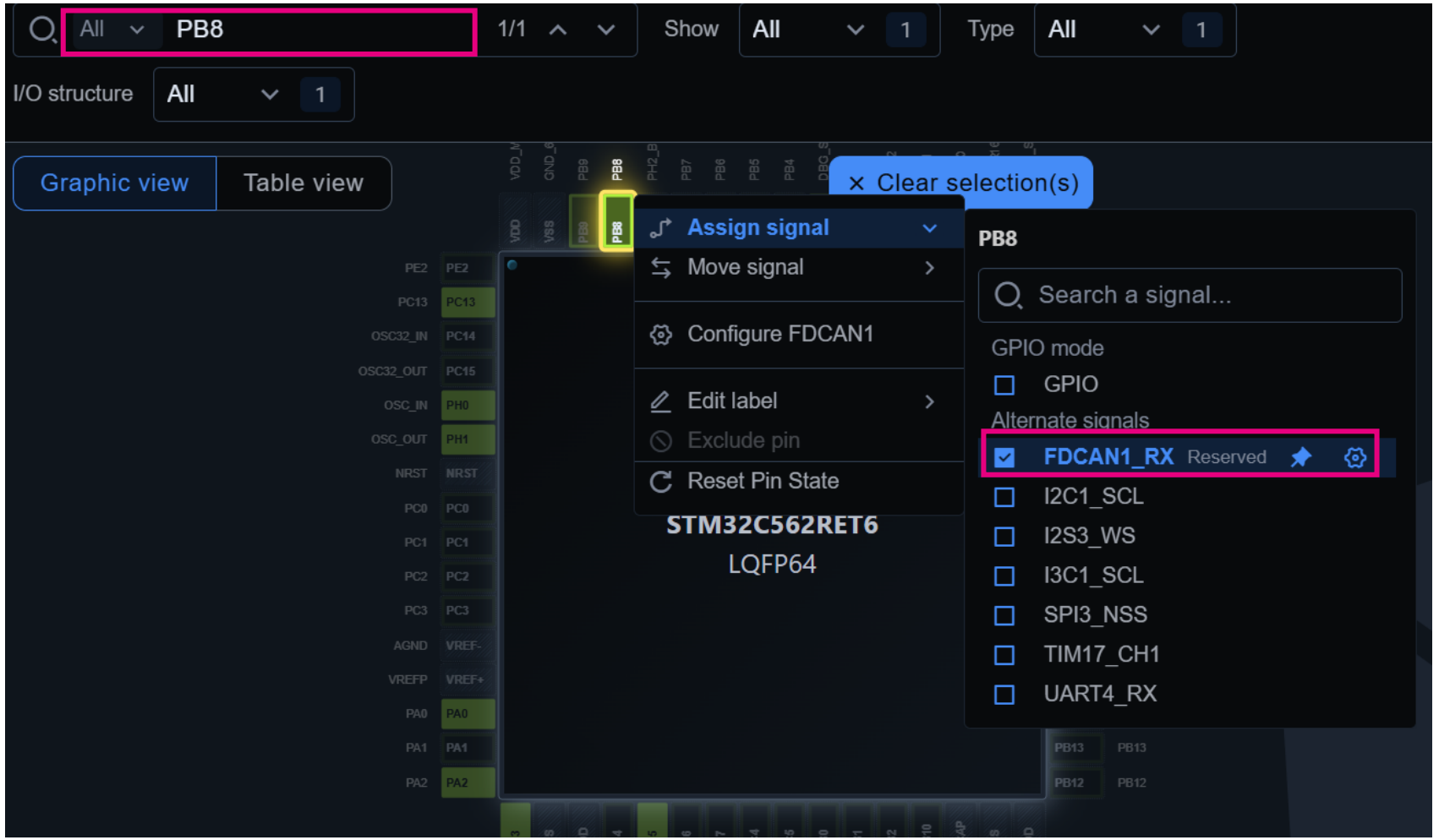
FDCAN Configuration

- Search for **PB9** and right-click on the pin to assign **FDCAN1_TX** to **PB9**



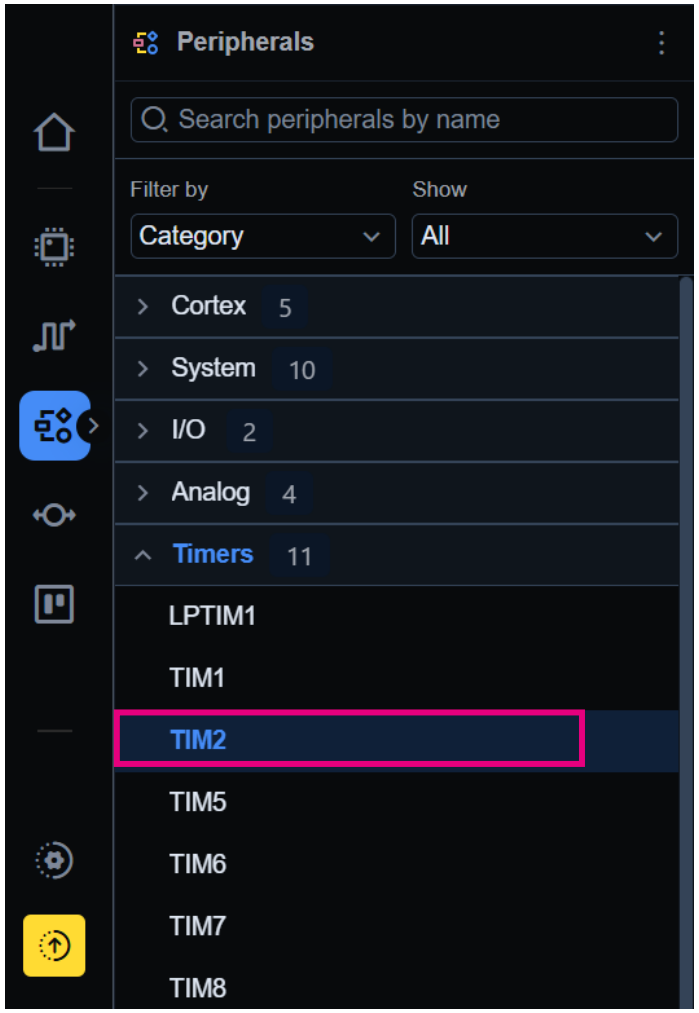
FDCAN Configuration

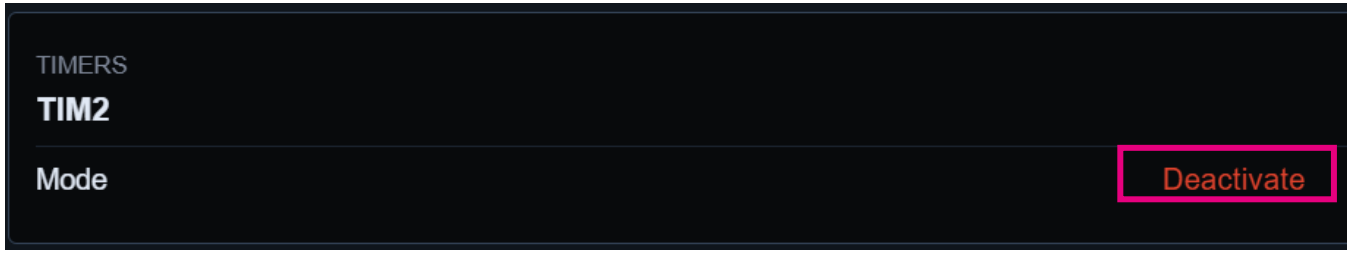
- Search for **PB8** and right-click on the pin to assign **FDCAN1_RX** to **PB8**



FDCAN Configuration

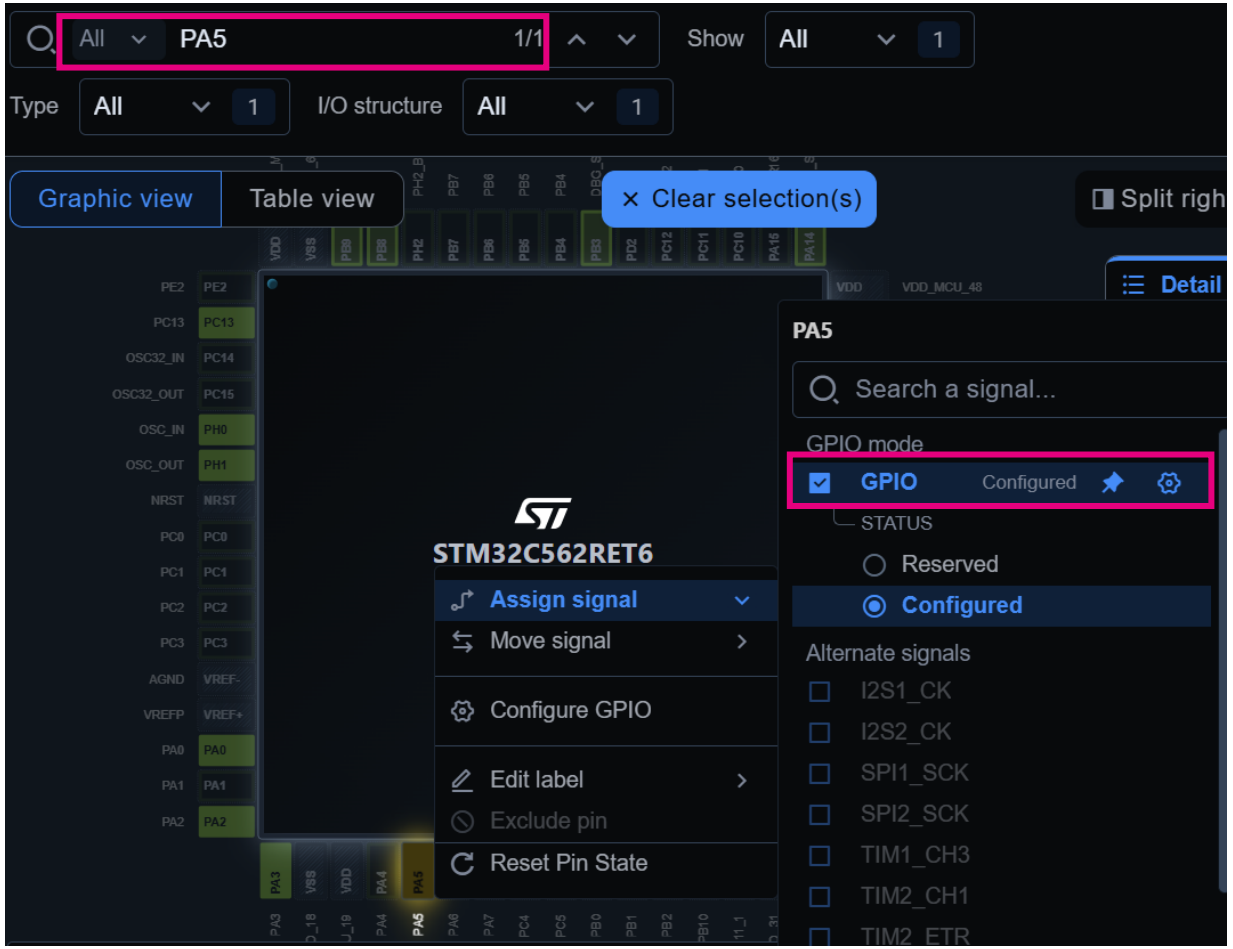
- Deactivate **TIM2** under **Peripherals > Timers**



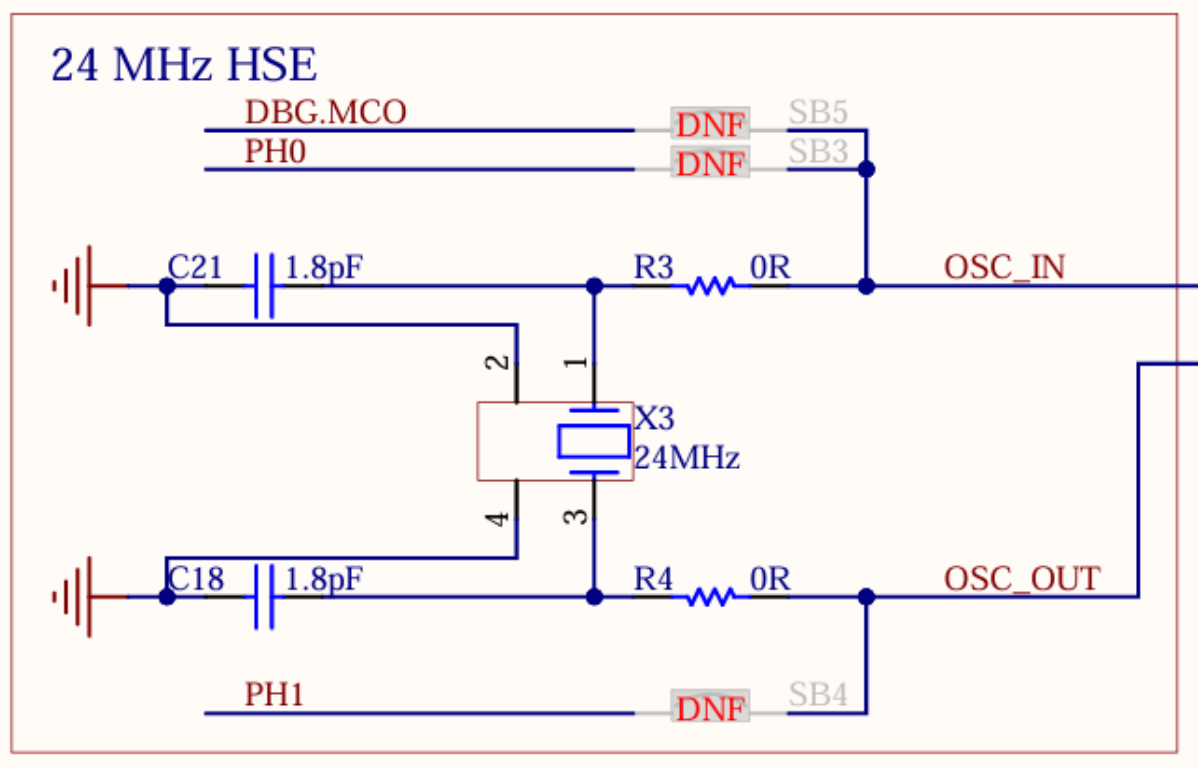


FDCAN Configuration

- Search for **PA5** and right-click on the pin to assign **GPIO** to **PA5**



External Clock

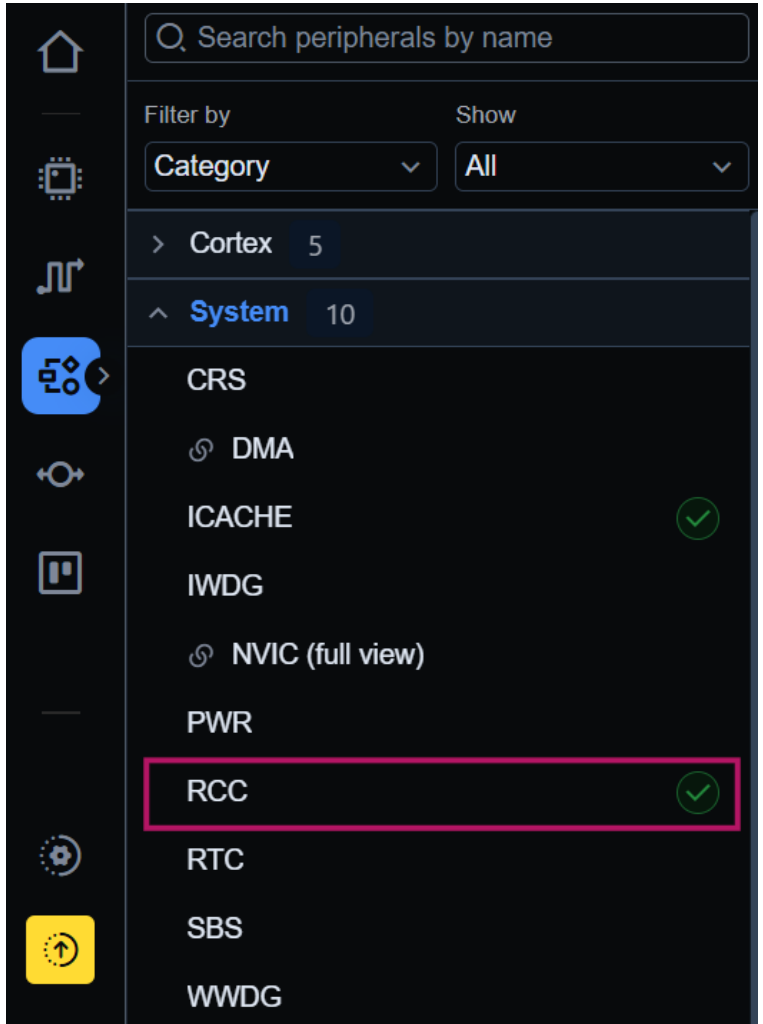


Schematics

- The NUCLEO-C562RE has an external crystal to generate a precise clock
- This resource is mandatory to meet the FDCAN timing accuracy requirements

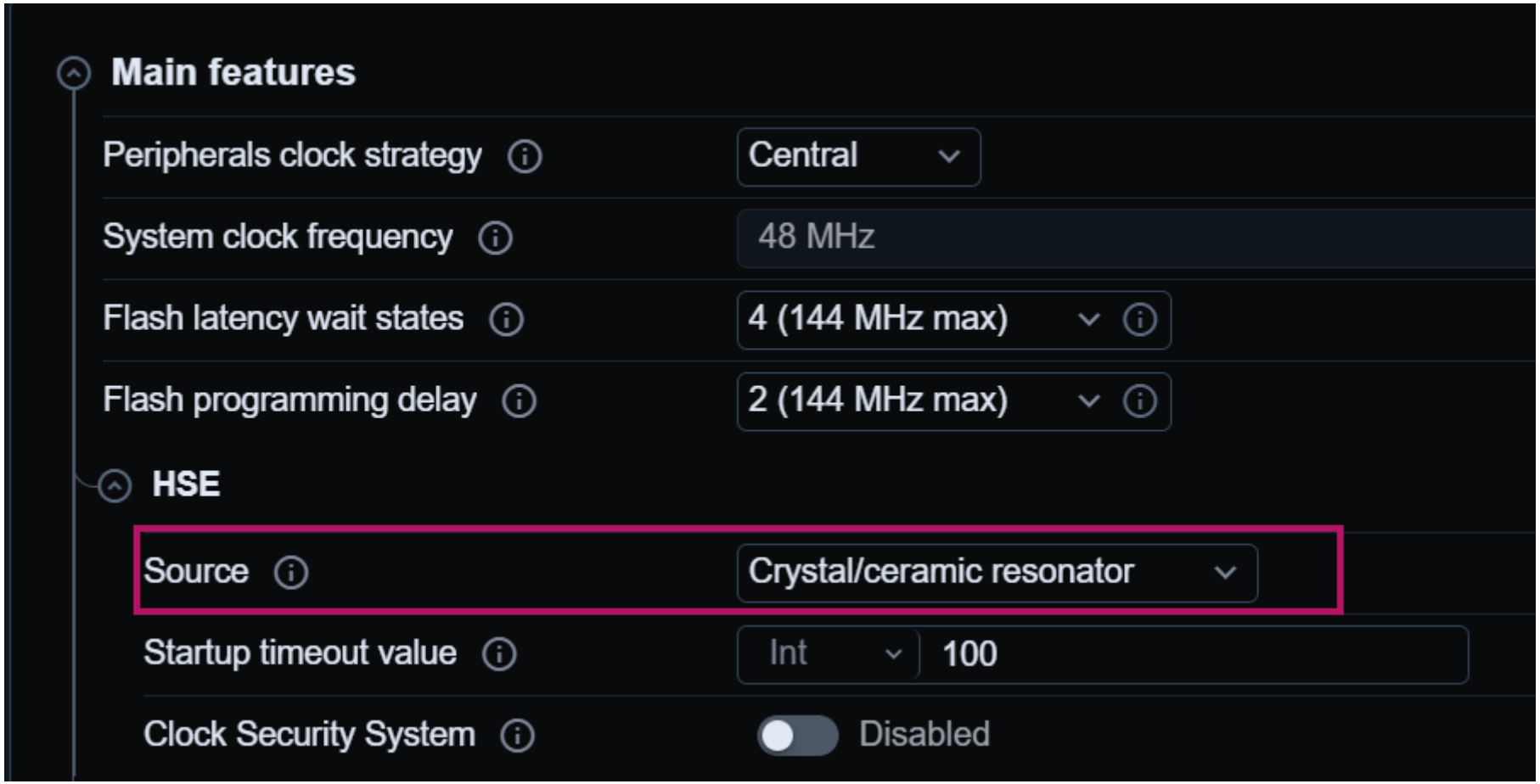
Enabling the External Clock

- We will enable the external clock crystal in **Peripherals > System > RCC**



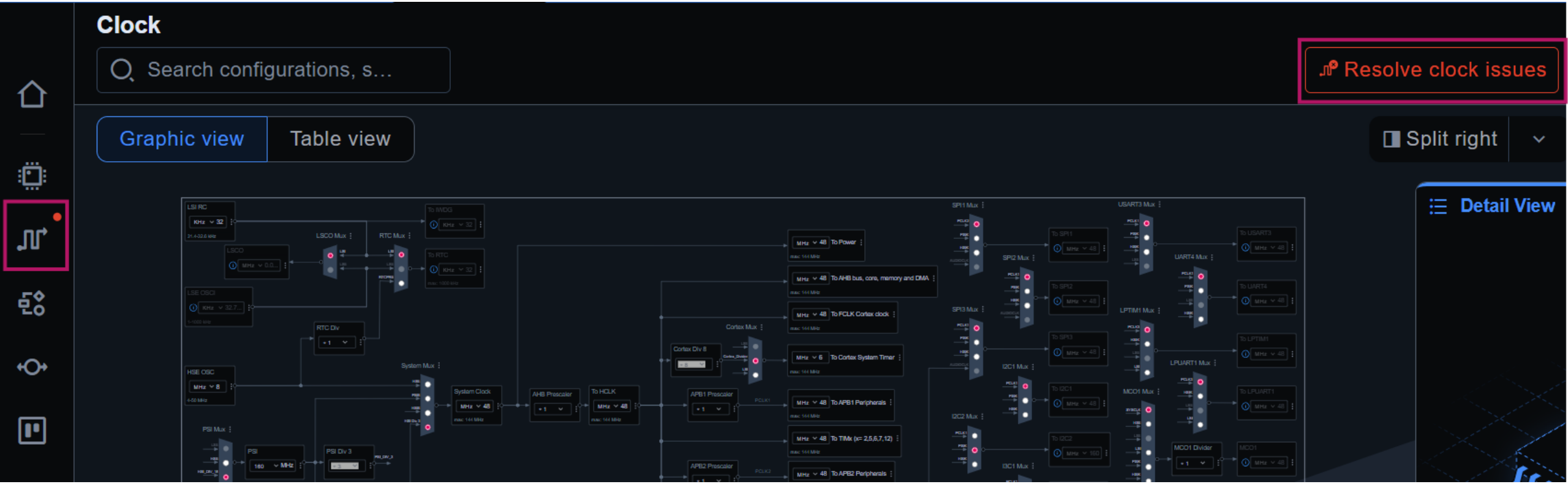
Enabling the External Clock

- Under **Main features > HSE**
  - Source: **Crystal/ceramic resonator**



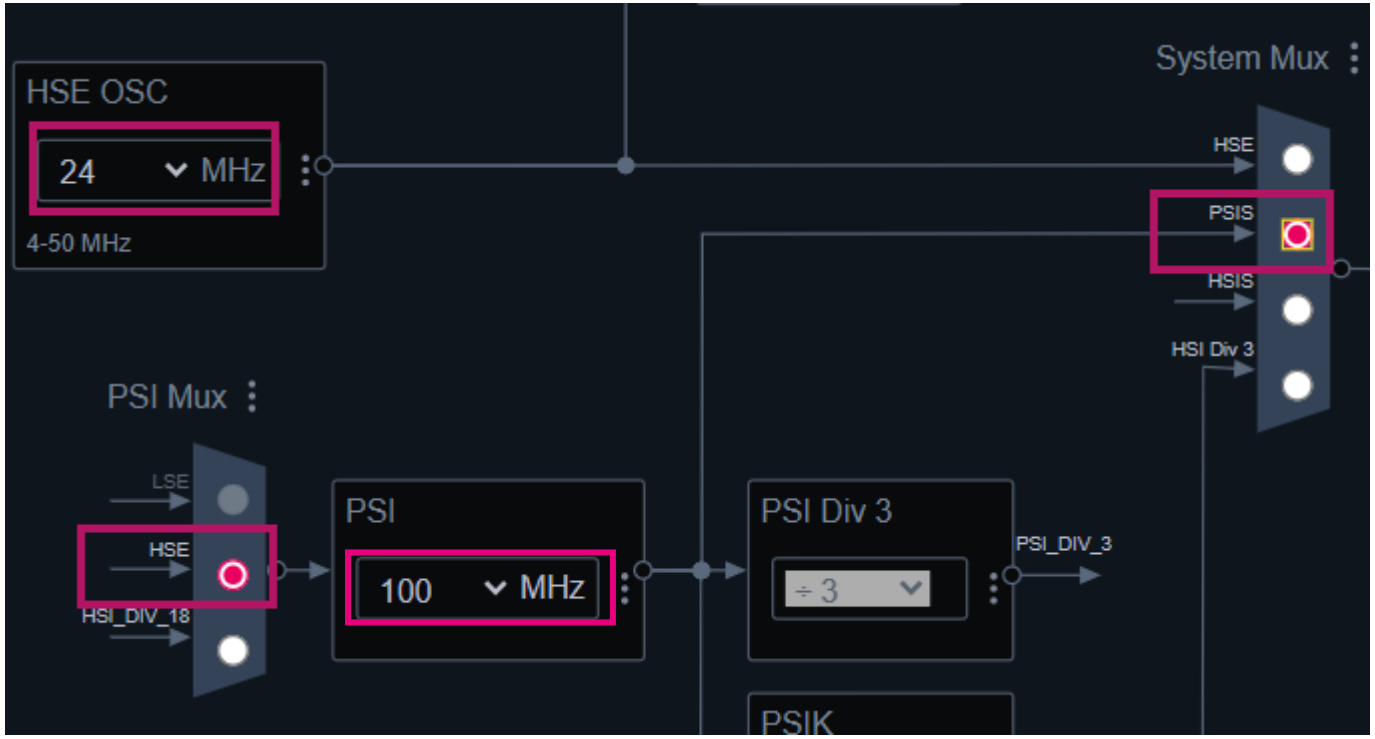
Enabling the External Clock

- If needed, open the **Clock** tab and select **Resolve clock issues**



Enabling the External Clock

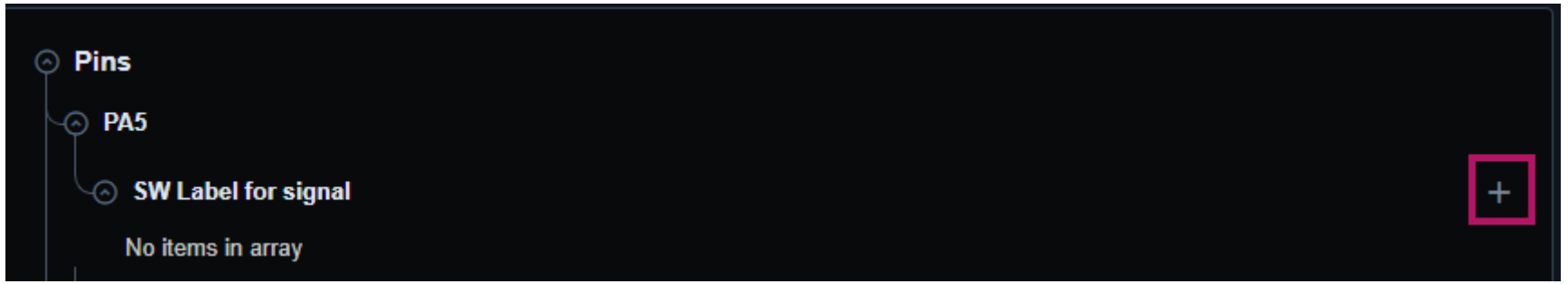
- Change the following value/selections
  - PSI Mux : **HSE**
  - PSI : **100 MHz**
  - System Mux : **PSIS**
  - HSE OSC : **24 MHz**

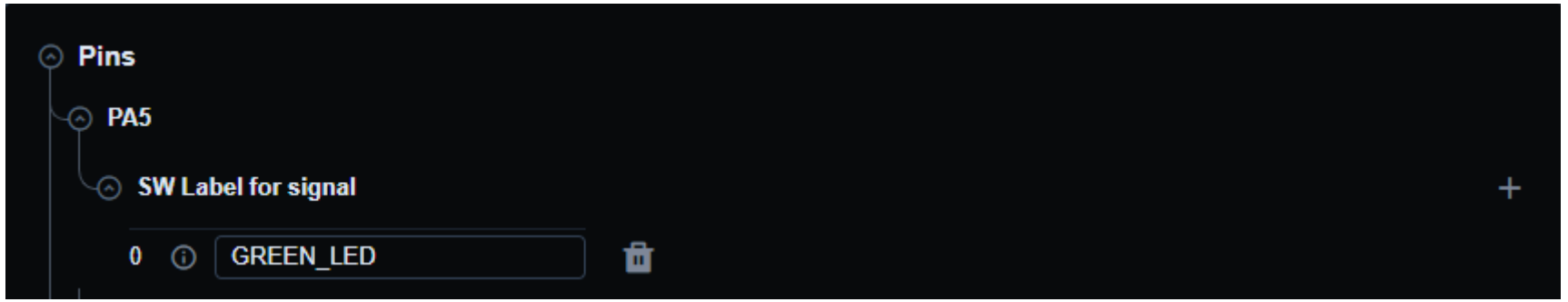


Configuring the LED GPIO

- Under **Peripherals > I/O > GPIO**
  - Navigate to **Pins > PA5** to create a SW Label for the signal with the value **GREEN_LED**

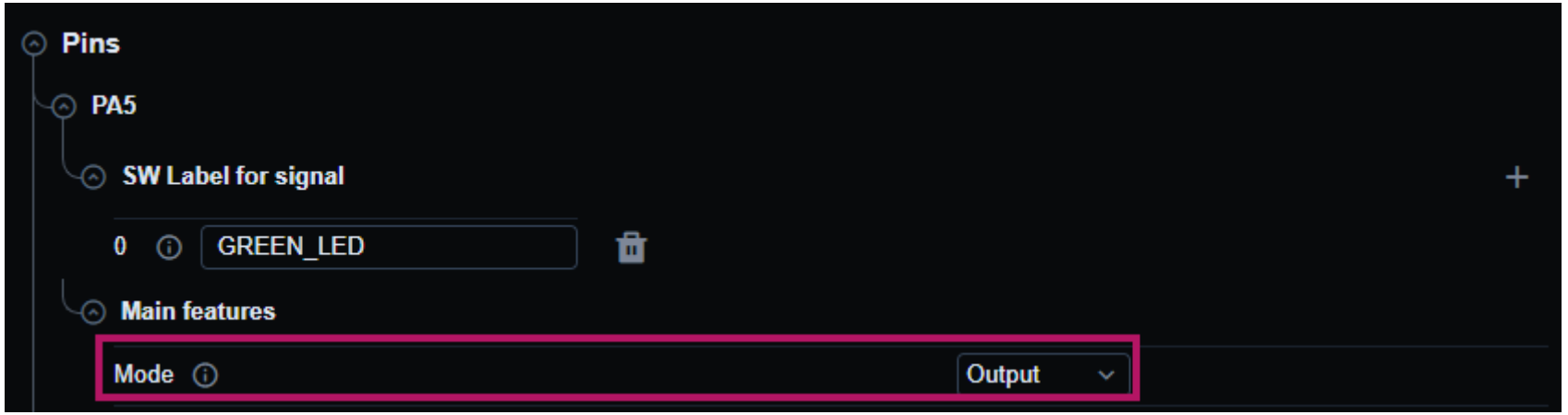






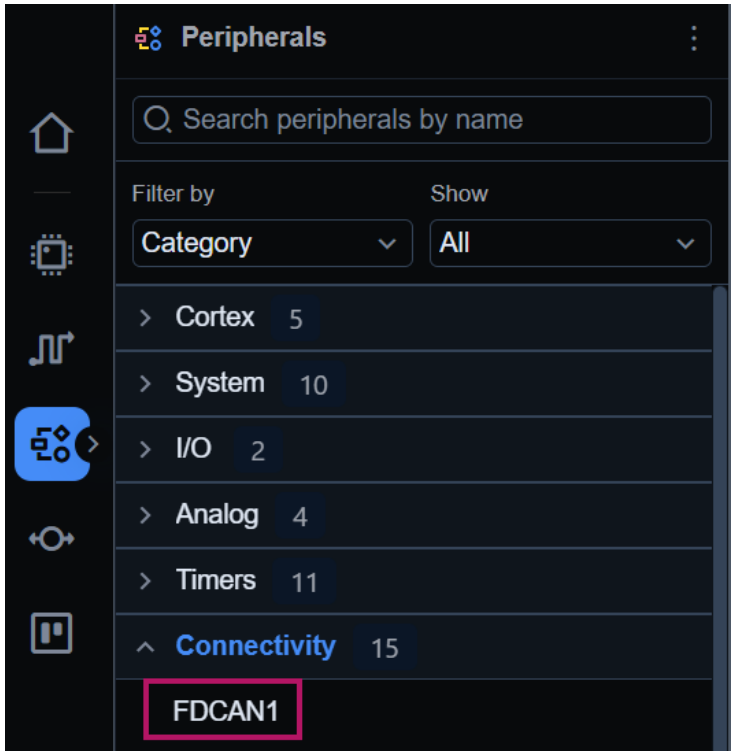
Configuring the LED GPIO

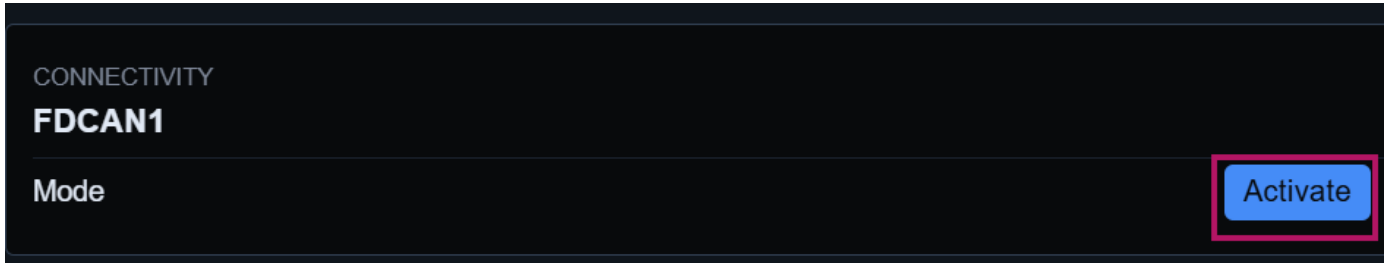
- Under **Peripherals > I/O > GPIO**
  - Navigate to **Pins > Main features > Mode**
    - Mode: **Output**



Configuring the FDCAN Peripheral

- Select **Peripherals > Connectivity > FDCAN**
  - Click **Activate** to enable the peripheral





Configuring the FDCAN Peripheral

- Under **Main Features**
  - Frame Format: **FD mode without bitrate switching**
  - Frame filters > Number of standard filters: **1**



Configuring the FDCAN Peripheral

- Under **Frame filters > Standard frame filters > Standard Filter0**
  - Configure the standard filter: **Enabled**
  - Filter type: **Dual ID: Matches only ID1 or ID2**
  - Filter ID1: **5**
  - Filter ID2: **a**



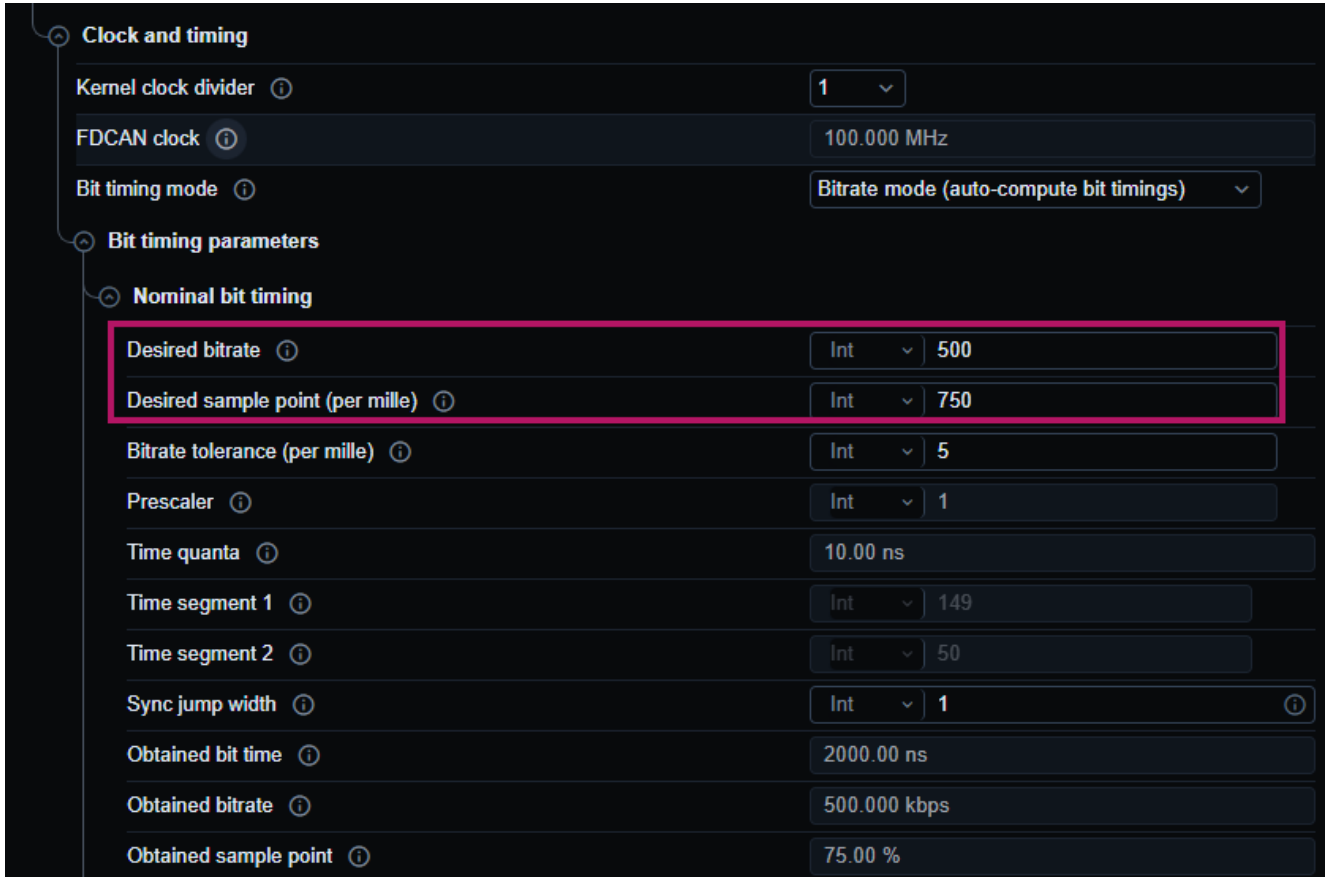
Configuring the FDCAN Peripheral

- Under **Frame filters > Acceptance or rejection rules for filters**
  - Acceptance for non-matching standard frames: **Reject non-matching frames**



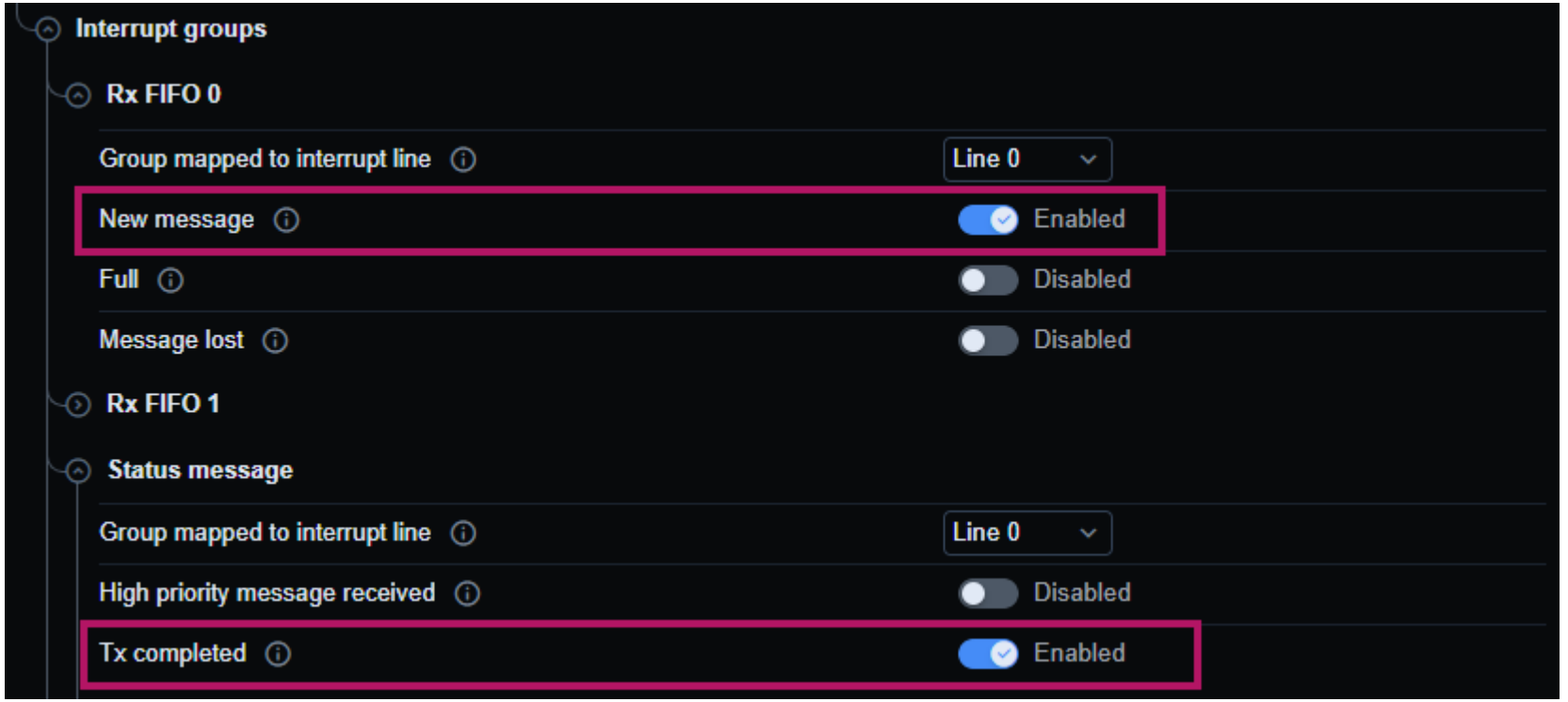
Configuring the FDCAN Peripheral

- Under **Clock and timing > Bit timing parameters > Nominal bit timing**
  - Desired bitrate: **500** (kbps)
  - Desired sample point: **750** (per mille)



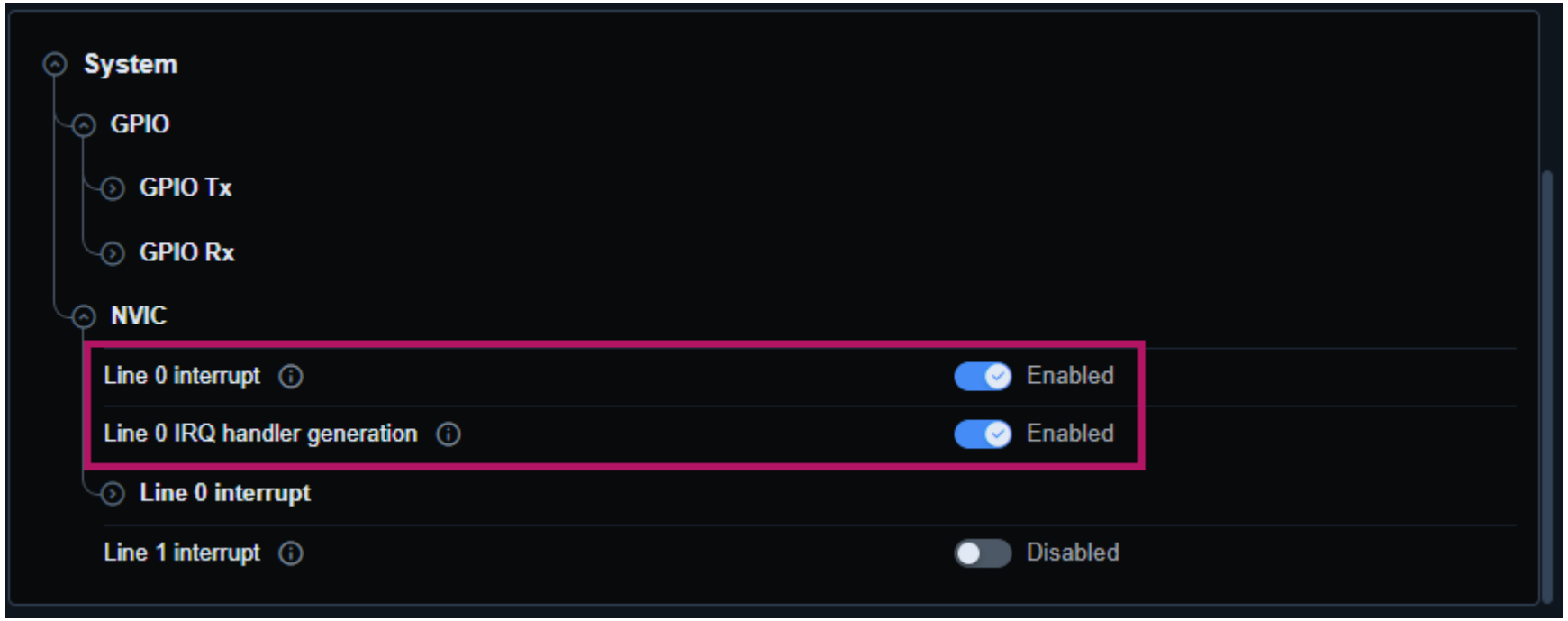
Configuring the FDCAN Peripheral

- Under **Advanced Groups > Interrupt groups**
  - RX FIFO 0 > New message: **Enabled**
  - Status message > Tx completed: **Enabled**



Configuring the FDCAN Peripheral

- Under **System > NVIC**
  - Line 0 interrupt: **Enabled**
  - Line 0 IRQ handler generation: **Enabled**



Generating CMake Project

- Return to project settings, make sure **CMake** is selected as the format and click **Generate IDE project**





Adding Application Code

NOTE: delete the existing contents of main.c before adding the FDCAN application code

Adding Application Code

- Before **main()**
  - Define txData and rxData
  - Define tx_complete_flag as 1
  - Declare the rx_element_header
  - Define the tx_element_header

```
/* Private variables ---------------------------------------------
uint8_t txData[9] = "FDCAN:12";
uint8_t rxData[8];

volatile uint8_t tx_complete_flag = 1;

hal_fdcan_rx_header_t rx_element_header;
hal_fdcan_tx_header_t tx_element_header =
{
    .b.identifier = 0x00,
    .b.frame_type = HAL_FDCAN_FRAME_DATA,
    .b.identifier_type = HAL_FDCAN_ID_STANDARD,
    .b.error_state_indicator = HAL_FDCAN_ERROR_STATE_IND_ACTIVE,
    .b.message_marker = 52U,
    .b.frame_format = HAL_FDCAN_HEADER_FRAME_FORMAT_FD_CAN,
    .b.bit_rate_switch = HAL_FDCAN_BIT_RATE_SWITCH_ON,
    .b.event_fifo_control = HAL_FDCAN_TX_EVENTS_FIFO_STORE,
    .b.data_length = HAL_FDCAN_DATA_LEN_CAN_FDCAN_8_BYTE,
};
```

Adding Application Code

- In **main()** before while loop
  - Start FDCAN peripheral

```c
int main(void)
{
  /** System Init: this code placed in targets folder initializes your system.
   * It calls the initialization (and sets the initial configuration) of the periphe
   * You can use STM32CubeMX to generate and call this code or not in this project.
   * It also contains the HAL initialization and the initial clock configuration.
   */
  if (mx_system_init() != SYSTEM_OK)
  {
    return (-1);
  }
  else
  {
    /*
     * You can start your application code here
     */

    /* Start the FDCAN Peripheral */
    HAL_FDCAN_Start(mx_fdcan1_gethandle());
```

Adding Application Code

- In **main()** in while loop
  - Add transmit application code

```
70    while (1) {
71      /* Wait for the TX complete IT */
72      if(tx_complete_flag == 1)
73      {
74        HAL_Delay(100);
75
76        /* Transmit a new message on CAN Bus */
77        HAL_FDCAN_ReqTransmitMsgFromFIFOQ(mx_fdcan1_gethandle(), &tx_element_header, txData);
78
79        /* Update the message to be transmitted */
80        txData[7]++;
81        if(txData[7] > '9')
82        {
83          txData[7] = '0';
84          txData[6]++;
85          if(txData[6] > '9')
86          {
87            txData[6] = '0';
88          }
89        }
90
91        /* Change the message ID */
92        tx_element_header.b.identifier++;
93
94        /* If the message ID is outside the range, reset it */
95        if(tx_element_header.b.identifier >= 0xB)
96        {
97          tx_element_header.b.identifier = 0;
98        }
99
100       /* Reset the TX Complete flag */
101       tx_complete_flag = 0;
102     }
103   }
104 }
105 /* end main */
```

Adding Application Code

- Outside of **main()**
  - Add RXFifo0Callback – Copy received FDCAN message to RAM and toggle LED
  - Add TxBufferCompleteCallback – Set the tx_complete_flag

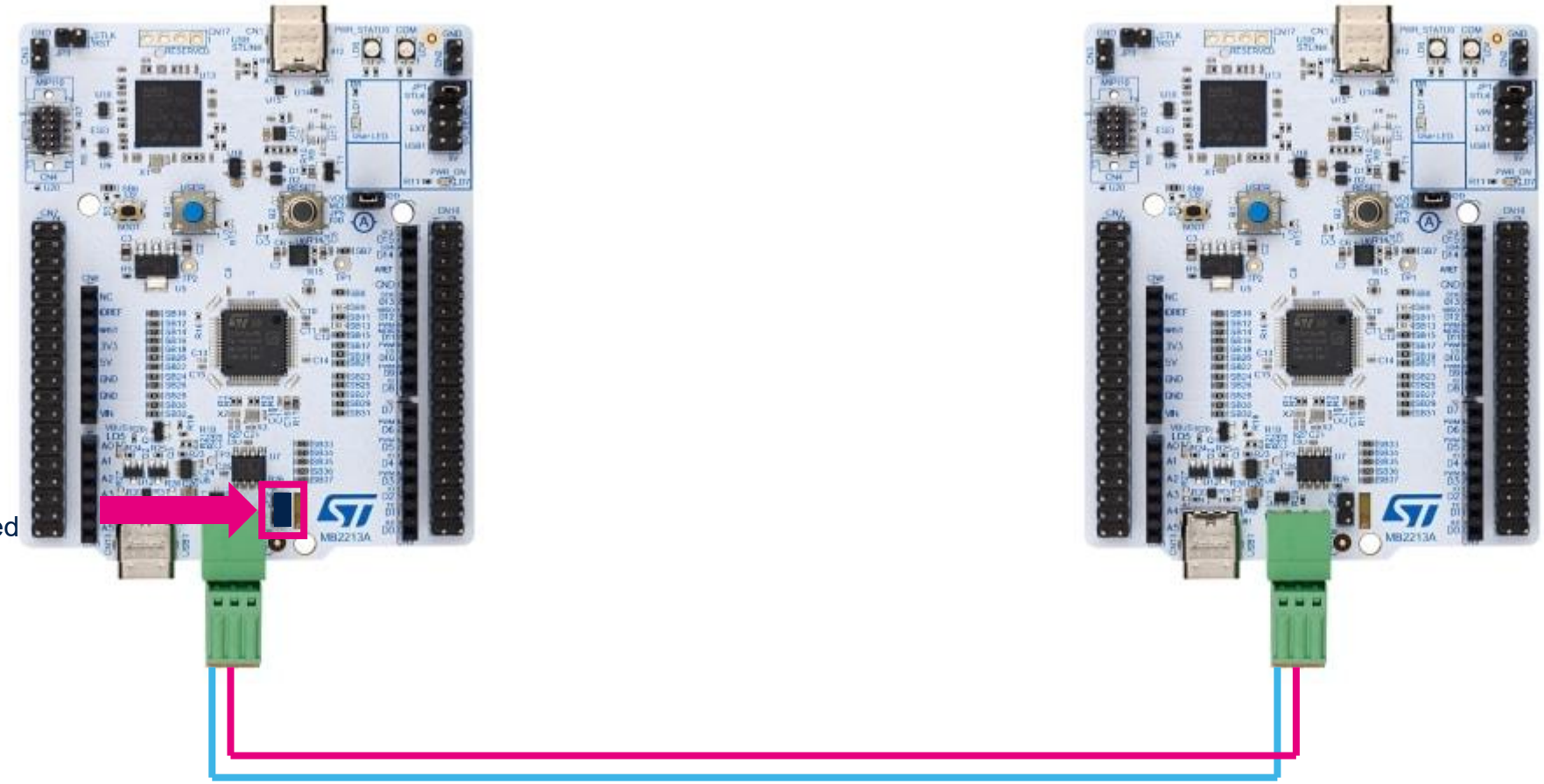
```
void HAL_FDCAN_RxFifo0Callback(hal_fdcan_handle_t *hfdcan, uint32_t rx_fifo0_interrupts)
{
  /* Copy the received FDCAN message to the MCU RAM */
  HAL_FDCAN_GetReceivedMessage(mx_fdcan1_gethandle(), HAL_FDCAN_RX_FIFO_0, &rx_element_header, rxData);

  /* Toggle the LED to indicate a message was received */
  HAL_GPIO_TogglePin(GREEN_LED_PORT, GREEN_LED_PIN);
}

void HAL_FDCAN_TxBufferCompleteCallback(hal_fdcan_handle_t *hfdcan, uint32_t tx_buffers_idx)
{
  /* Set the TX complete flag */
  tx_complete_flag = 1;
}
```

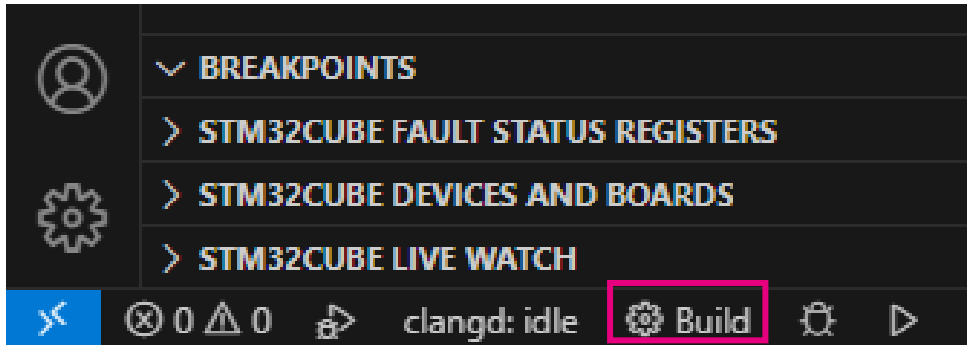
Hardware setup

Termination Resistor may not be necessary due to short cable length, but can be added by closing this header

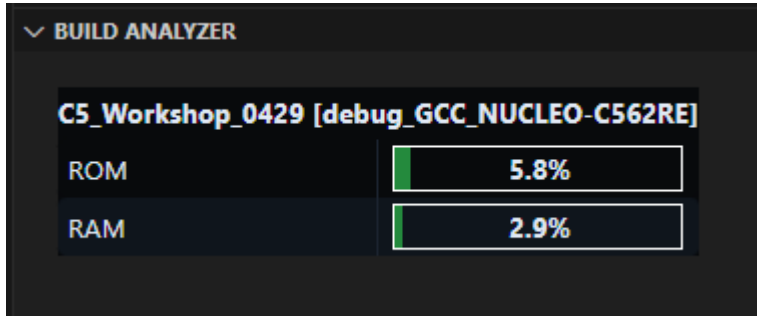


Building your project

- Select **Build** in the bottom left-hand corner of the screen

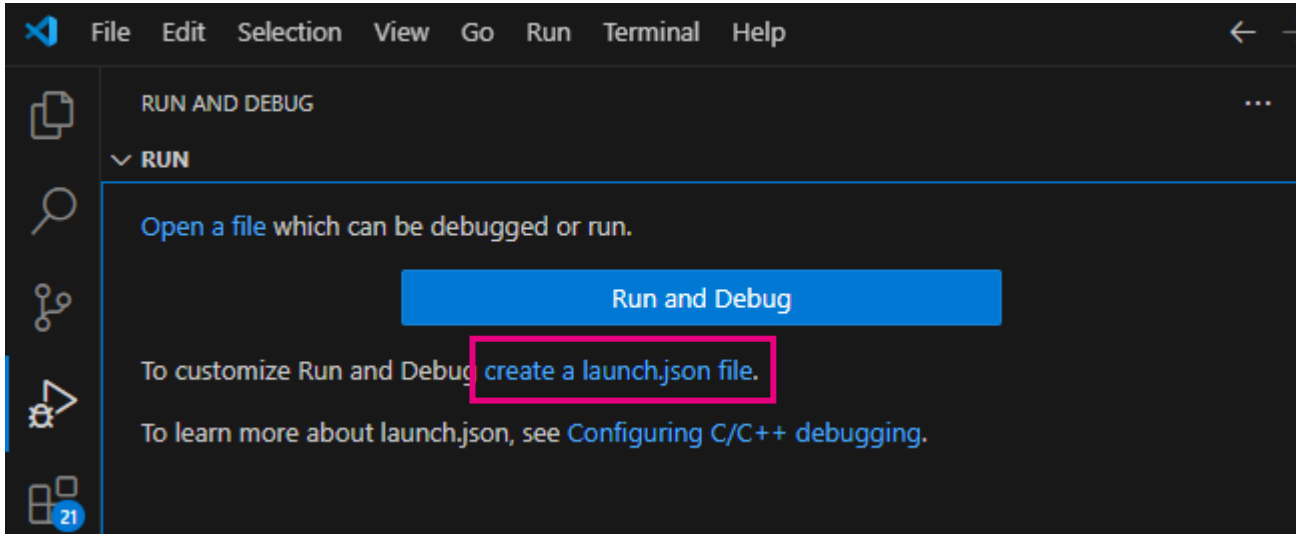


- Once you build you will see the Build Analyzer display ROM and RAM usage



Live Watch

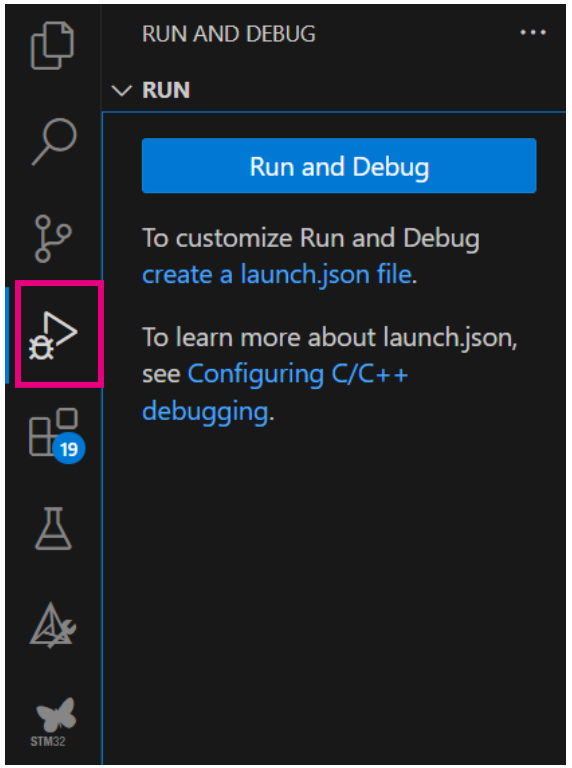
- If you want to use Live Watch, it must be enabled in your launch.json file




```
"preBuild": "${command:st-stm32-ide-debug-launch.build}",
"runEntry": "main",
"liveWatch": {
    "enabled": true,
    "samplesPerSecond": "4"
},
"imagesAndSymbols": [
    {
        "imageFileName": "${command:st-stm32-ide-debug-launch
    }
]
```

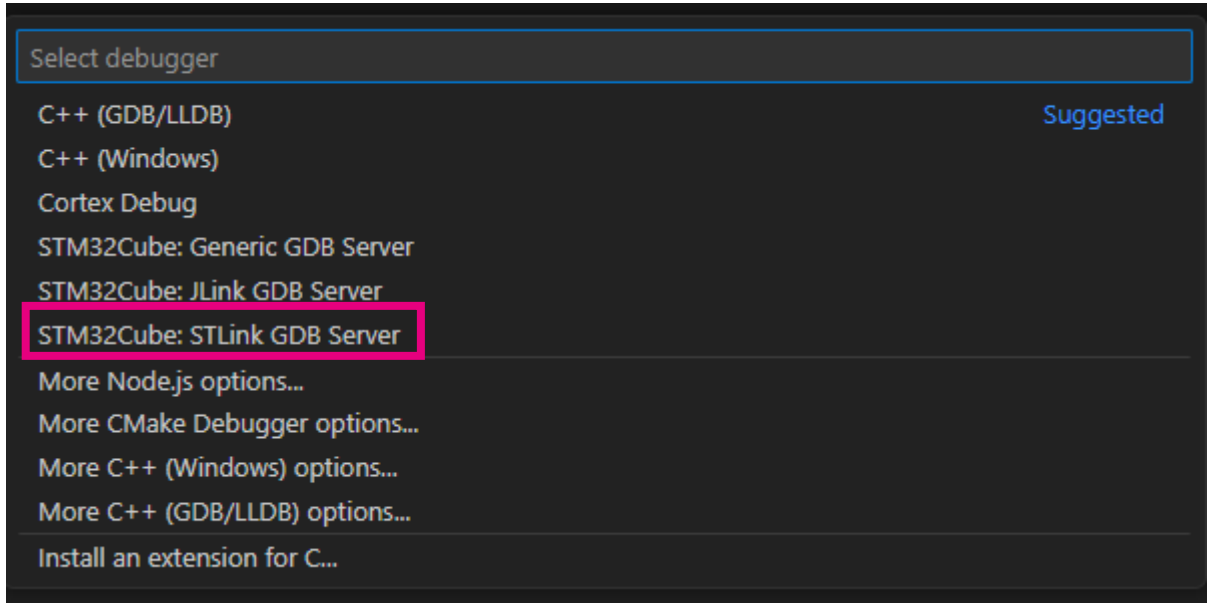
Running and debugging your project

- To run and debug your project select **Run and Debug** on the left-side toolbar and click the blue button



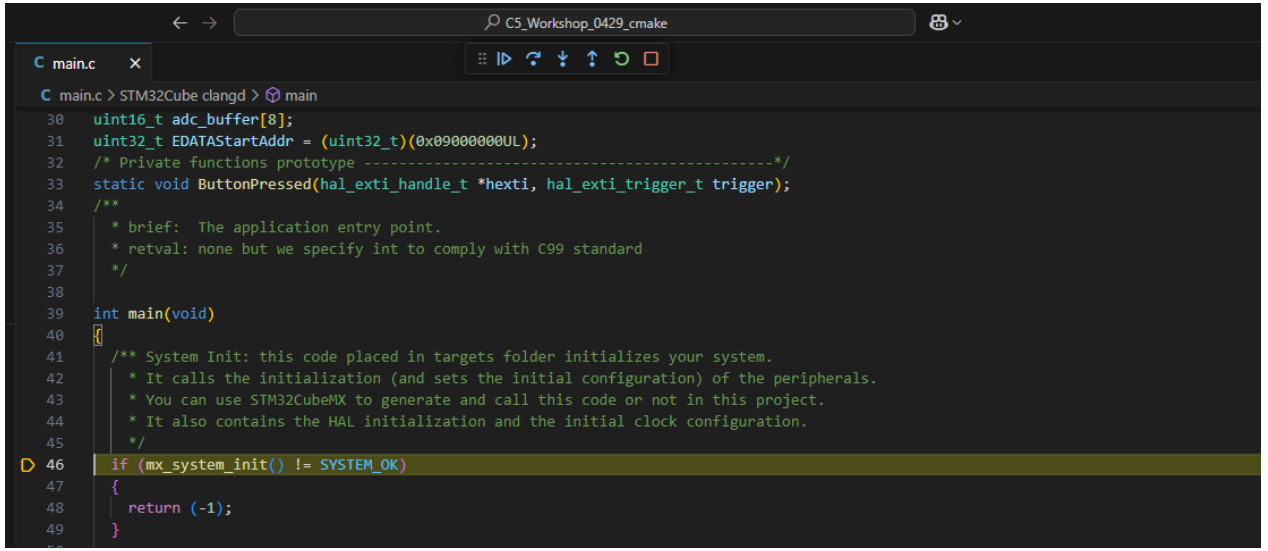
Running and debugging your project

- Select **STM32Cube: STLink GDB Server** as the debugger



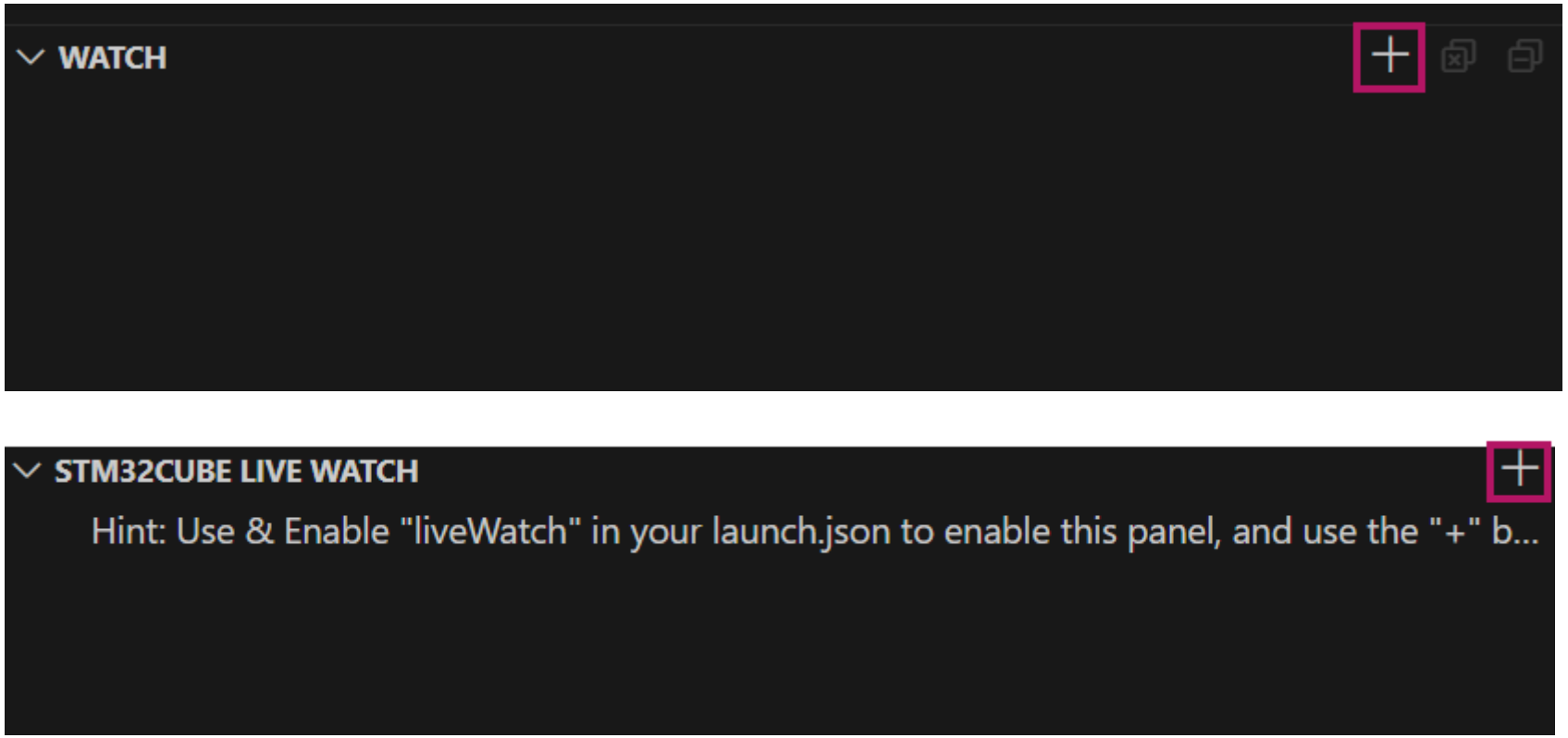
Running and debugging your project

- Your project will stop at the initial breakpoint
- Hit **Continue** to resume execution



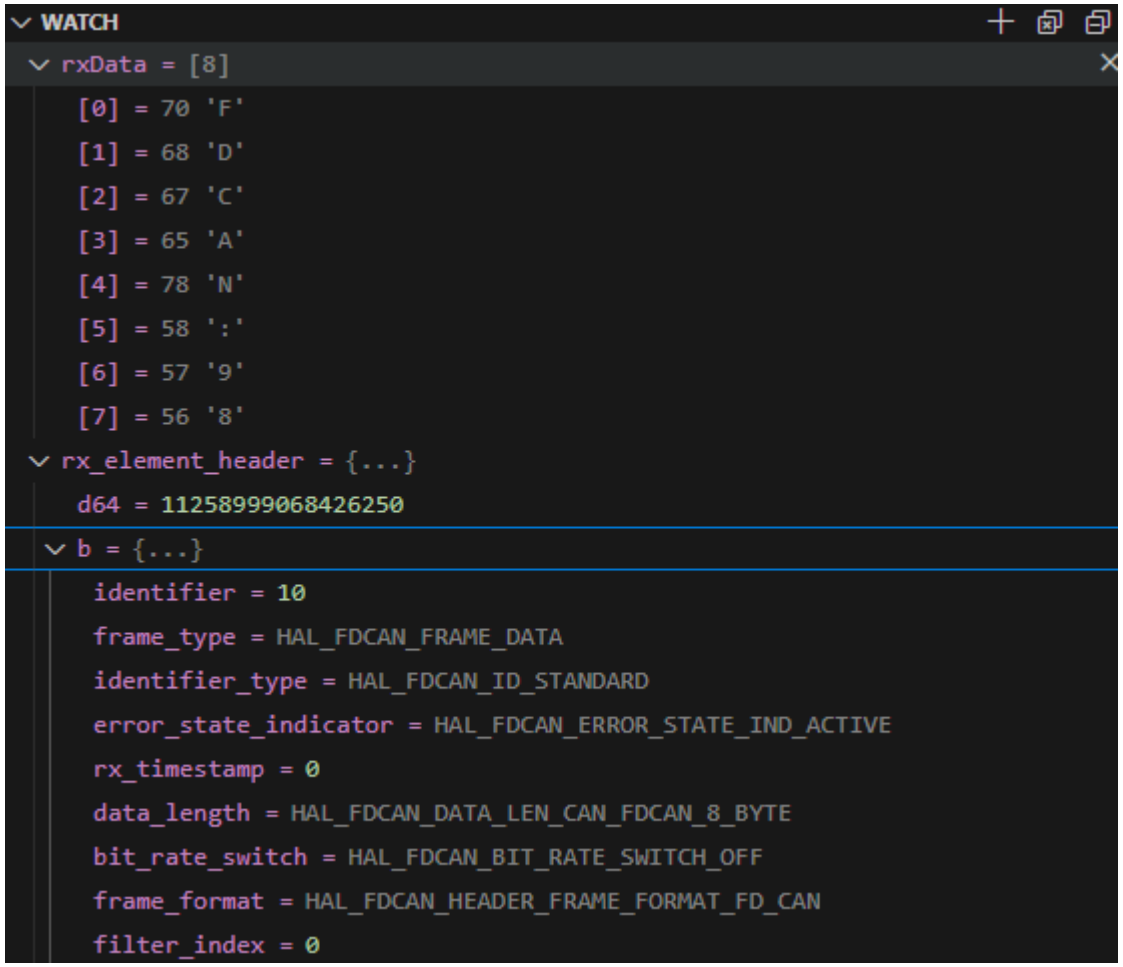
Running and Debugging

- You can add variables to the Watch and Live Watch windows



Running and debugging your project

- As you and your partner run your project, you should see:
  - LED Blinking
  - Corrrect values in watch window for **rx_element_header**
  - Values changing for **rxData**

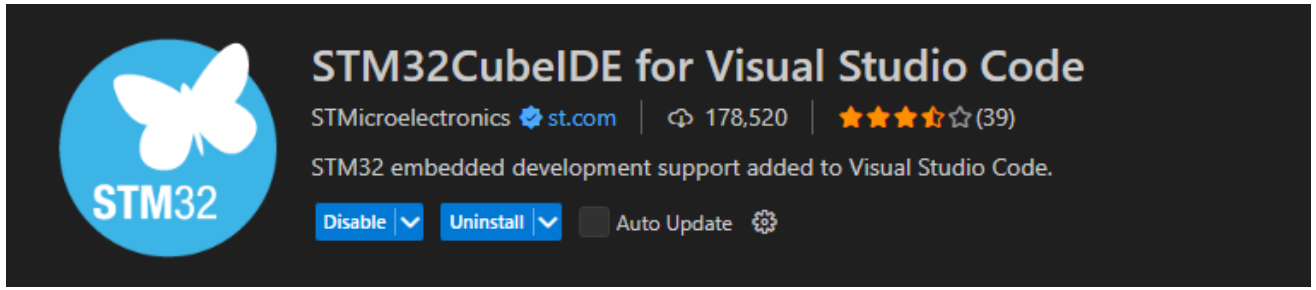


Takeaways

Now you have successfully…

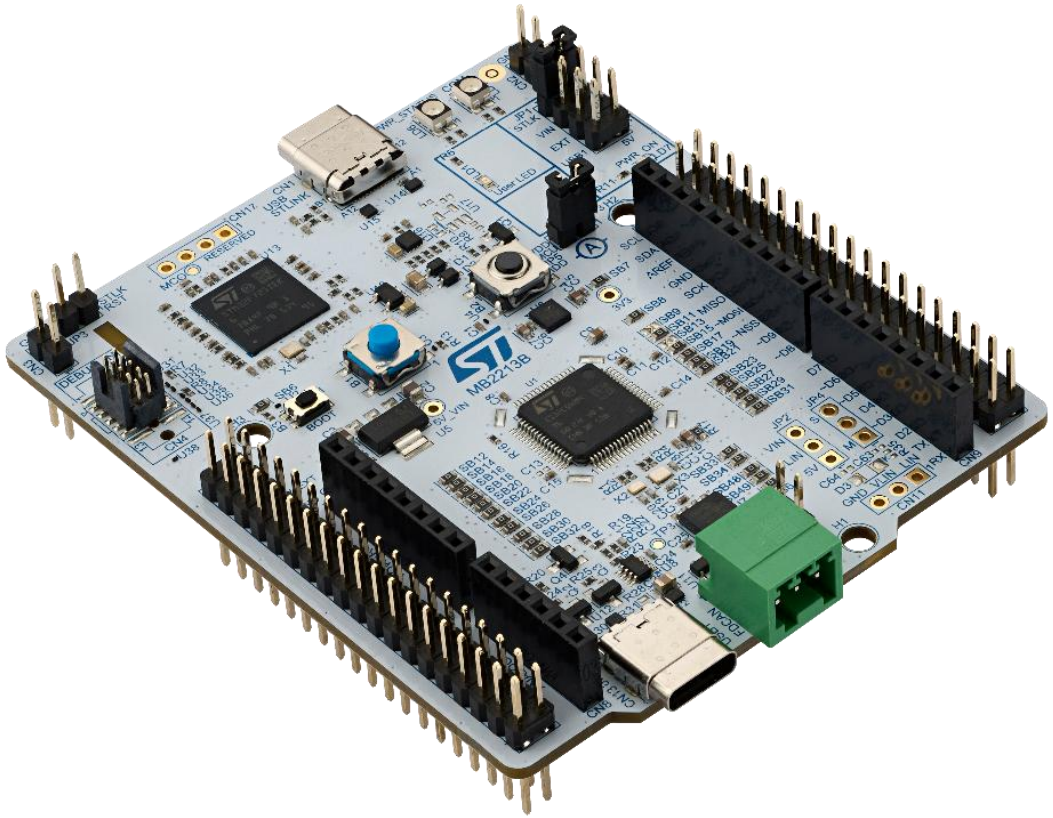
- Created an application using the STM32C5
- Configured a project with STM32CubeMX2
- Set up a CMake project using STM32CubeIDE for VSCode





Additional Resources

- [STM32CubeMX2 and HAL2 basics | Community]





Releasing your creativity

@STM32

@ST_World

community.st.com/

STM32C5 series - STMicroelectronics

wiki.st.com/stm32mcu

github.com/stm32-hotspot

www.st.com/mcu-developer-zone

life.augmented

Our technology starts with You



Find out more at www.st.com

© STMicroelectronics - All rights reserved.
ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.
For additional information about ST trademarks, please refer to www.st.com/trademarks.
All other product or service names are the property of their respective owners.